

UNIVERSITÀ
DI TORINO

University of Turin

Bachelor's Degree Program in Computer Science

Design and Implementation of a Simple Kathará-Based Framework for Network Emulation and Testing

Automated Topology Generation, Connectivity Validation and Denial-of-Service Experimentation

Bachelor's Thesis

Supervisor
Prof. Gianluca Rizzo

Candidate
Luca Bortoluzzi
Student ID: 1097723

Academic Year 2025/2026

Declaration

I declare to be responsible for the content I'm presenting in order to obtain the final degree, not to have plagiarized in all or part of the work produced by others, and to have cited original sources in a manner consistent with current plagiarism regulations and copyright. I am also aware that, in case of a false declaration, I could incur legal penalties and my admission to the final exam could be denied.

Abstract

Network laboratories are widely used to study routing, service deployment, traffic analysis, and cybersecurity. Nevertheless, the manual preparation of a network-emulation environment often requires several configuration files, repeated commands, and careful coordination among topology, addressing, routing, and application services. This thesis presents the design and implementation of a lightweight Python-based framework built on top of Kathará. The framework allows a network laboratory to be described through a structured YAML configuration containing nodes, interfaces, collision domains, IP addresses, gateways, static routes, routing protocols, and optional services. Starting from this declarative description, the system validates the input and automatically generates the files required to deploy the corresponding Kathará laboratory. The project also includes an automated validation component. Legitimate clients execute concurrent TCP connection tests against a target service, while the framework records the result and the elapsed time of every request in structured CSV files and execution logs. The same procedure is applied to a baseline scenario and to a controlled Denial-of-Service simulation executed inside an isolated laboratory. The objective is not to replace Kathará, but to provide a simple abstraction and orchestration layer that reduces repetitive configuration tasks and supports repeatable experiments. The resulting workflow integrates topology generation, routing configuration, service deployment, traffic inspection, connectivity testing, and the comparison of network behaviour under normal and adverse conditions.

Keywords: network emulation; Kathará; Docker; YAML; FRRouting; automated testing; Wireshark; denial of service; large language models.

Contents

Declaration **i**

Abstract **iii**

Introduction **vii**
Objectives . . . viii
Research Questions . . . viii
Thesis Structure . . . ix

1 Network Emulation and Supporting Technologies **1**
1.1 Chapter Overview . . . 1
1.2 Experimental Network Environments . . . 1
1.3 Simulation and Emulation . . . 2
1.4 Virtual Machines and Containers . . . 2
1.5 Docker as the Execution Substrate . . . 3
1.6 Kathará . . . 3
1.6.1 Nodes and Interfaces . . . 3
1.6.2 The lab.conf File . . . 3
1.6.3 Startup Files . . . 4
1.6.4 Node Directories and hostlab . . . 4
1.6.5 Laboratory Lifecycle . . . 4
1.7 Declarative Configuration with YAML . . . 5
1.8 Dynamic Routing and FRRouting . . . 5
1.8.1 OSPF . . . 5
1.8.2 RIP . . . 6
1.8.3 BGP . . . 6
1.9 Traffic Inspection with Wireshark . . . 6
1.10 Automation, Validation, and Reproducibility . . . 6
1.11 Positioning with Respect to Related Work . . . 7
1.12 Chapter Summary . . . 8

2 Requirements, Development Lifecycle and System Architecture **9**
2.1 Problem, Scope, and Requirements . . . 9
2.2 Software Development Lifecycle and Design Evolution . . . 13
2.3 System Architecture and Repository Organisation . . . 15
2.4 Operational Flow and Design Principles . . . 17
2.5 LLM-Assisted Configuration and Deterministic Validation . . . 18
2.6 Architectural Boundaries and Chapter Summary . . . 20

3 Configuration Model and Validation **23**
3.1 Configuration Model and Supported Operations . . . 23
3.2 Validation Process and Semantic Rules . . . 31
3.3 Validation Workflow and Limitations . . . 34

4 Laboratory Generation and Lifecycle Management **37**
4.1 Laboratory Generation Process . . . 37
4.2 Generated Laboratory Structure and Resources . . . 40
4.3 Laboratory Execution and Lifecycle Management . . . 45
4.4 Chapter Summary . . . 49

5 Experimental Evaluation and Controlled DoS Experiment **51**
5.1 Experimental Methodology and Testbed . . . 53
5.2 Measurement Infrastructure and Baseline Evaluation . . . 54
5.3 Controlled DoS Experiment and Observed Results . . . 58
5.4 Reverse-Proxy Mitigation Results . . . 62
5.5 Repeated-Run Repeatability Analysis . . . 65
5.6 Threats to Validity . . . 66
5.7 Chapter Summary . . . 67

6 Robustness, Security Limitations and Scalability Evaluation **71**
6.1 Stress-Test Methodology . . . 71
6.2 Security and Robustness Weaknesses . . . 73
6.3 Translator Model Limitations . . . 76
6.4 Possible Improvements . . . 77
6.5 Discussion . . . 78

7 Framework Evolution and Future Developments **79**
7.1 Extension to Network Attack Testing . . . 79
7.2 Runtime Verification and Automated Testing . . . 82
7.3 Experiment Management and Usability . . . 83
7.4 Using LLM-Based Coding Alongside the Framework . . . 85
7.5 Towards a More General Network Emulation Framework . . . 87

8 Conclusions **89**
8.1 Summary of Contributions . . . 89
8.2 Main Findings and Overall Assessment . . . 91
8.2.1 Answers to the Research Questions . . . 93
8.3 Final Remarks . . . 93

Acknowledgements **95**

References **97**

Introduction

Computer networks are increasingly complex systems composed of heterogeneous devices, protocols, and services. Even a limited experimental scenario can include several end hosts, multiple routers, dynamic routing, a demilitarised zone, traffic-monitoring tools, and one or more application services. Creating such an environment requires not only a logical topology, but also a coherent set of executable configurations.

Experimentation on physical hardware is not always practical. Dedicated devices may be expensive or unavailable, while changes to existing equipment may interfere with operational networks. Physical laboratories can also be difficult to reproduce exactly. Network emulation addresses these limitations by creating isolated environments in which real networking software can be executed without requiring a separate physical device for every node [14].

Kathará is an open-source, container-based network emulation system intended for teaching, research, sandbox testing, and protocol development [10, 26, 1]. It represents network devices as containers and connects their virtual interfaces through Layer 2 collision domains. This approach makes it possible to reproduce multi-node network scenarios on a single workstation while retaining access to real protocol stacks and routing software.

Although Kathará reduces the infrastructure required for network experiments, the creation of a laboratory still involves several files and repetitive operations. The user must normally define the topology, prepare startup files, assign addresses, configure gateways and routes, initialise routing daemons, copy application scripts, and verify that the generated environment behaves as expected.

These tasks become increasingly difficult to coordinate as the topology grows. The same information may appear in several artefacts, and a modification to one file may not be reflected in the others. A laboratory can also start without immediately revealing an incorrect route, an inactive application service, or an inconsistent address.

This thesis presents a framework that introduces a declarative configuration layer above Kathará. A network laboratory is described through a YAML file, which is parsed and validated before the required Kathará files are generated. The framework supports several node roles, interface and gateway configuration, static and dynamic routing, application services, optional traffic inspection, and custom node commands.

The work also addresses functional validation. A dedicated testing component discovers legitimate clients and performs simultaneous TCP connection attempts against a selected service. Each attempt is classified according to the application response and is recorded together with its elapsed time. The results are stored in a form that can be aggregated and compared.

The framework is finally used to execute two phases across two related service architectures. Each architecture is measured first under baseline conditions and then while attacker-controlled nodes generate a bounded Denial-of-Service workload. The first topology exposes the application server directly; the second places a rate-limiting reverse proxy in front of a private backend. Reusing the same client-side measurement procedure makes it possible to observe both the effect of the additional workload and the outcome of the mitigation in the tested configuration.

The implemented system was not obtained from a completely fixed specification in a single development step. The project followed an iterative and incremental software-

development lifecycle. A further practical question arises from the current use of LLM-based assistants in programming and network configuration: which parts of the workflow can be delegated to a general-purpose LLM, and which parts still benefit from an explicit domain-specific validator and generator? This question is addressed explicitly in Section 2.5 and revisited as a future integration direction in Section 7.4. The development lifecycle began with the narrower objective of reducing the repetitive preparation of Kathará laboratories and progressively acquired new requirements as generated topologies were executed and inspected. Problems observed during these iterations motivated the introduction of semantic validation, richer routing support, optional diagnostic components, explicit lifecycle management, application-level connectivity checks, concurrent measurements, and finally robustness and scalability evaluation. The thesis therefore discusses not only the final implementation, but also the reasons for important design choices, the alternatives they replaced, and the limitations revealed during verification.

Objectives

The principal objectives of the work are:

- to define a readable declarative format for network laboratories;
- to validate the configuration before generating deployment files;
- to automate the generation of Kathará topology and startup files;
- to generate FRRouting configurations for supported routing protocols;
- to integrate optional packet inspection with Wireshark;
- to support deployment of node-specific application resources and startup commands;
- to execute concurrent application-level connectivity tests;
- to store reproducible, machine-readable experimental results;
- to compare normal service behaviour with behaviour under a controlled Denial-of-Service workload and evaluate a reverse-proxy mitigation using the same measurement procedure;
- to assess the role of the framework in an LLM-assisted network experimentation workflow and identify which responsibilities should remain deterministic and explicitly verifiable.

Research Questions

The thesis is guided by the following questions:

RQ1 How can a declarative topology description be translated into the files and directories required by Kathará?

RQ2 How can the generated laboratory automatically include addressing and routing configuration, deploy node-specific application resources and startup commands, and attach monitoring components?

RQ3 How can end-to-end service availability and application response time be measured repeatedly from several legitimate clients?

RQ4 Can the same workflow support a controlled comparison between baseline and Denial-of-Service conditions, and can it evaluate a reverse-proxy mitigation without changing the client measurement method?

RQ5 What additional value does a deterministic validation and generation framework provide when LLMs can already assist with network configuration, and how can the two approaches be combined in a practical experimentation workflow?

RQ1–RQ4 are answered through implementation evidence and experimental evaluation. RQ5 is a design and discussion question addressed through architectural analysis and comparison with the literature; this thesis does not claim an empirical benchmark of LLM-generated configurations.

Thesis Structure

Chapter 1 introduces network emulation and the supporting technologies. Chapter 2 defines the requirements, reconstructs the development lifecycle of the project, presents the final system architecture, and positions the framework relative to LLM-assisted configuration. Chapter 3 describes the YAML model and validation subsystem, including the implementation choices that emerged as the configuration language evolved. The following chapters present laboratory generation and lifecycle management, the experimental scenarios and results, robustness and scalability evaluation, possible future developments—including an LLM-assisted configuration front end—and the conclusions of the work.

Chapter 1

Network Emulation and Supporting Technologies

1.1 Chapter Overview

This chapter introduces the technical concepts and software technologies on which the developed framework is based. The purpose is not to provide an exhaustive description of each tool, but to identify the functions that each technology performs in the proposed workflow. The framework combines container-based execution, network emulation, declarative configuration, dynamic routing, and packet inspection. These elements must cooperate while remaining sufficiently transparent for the user to inspect and debug a generated laboratory.

The chapter first discusses the role of experimental network environments and the distinction between simulation and emulation. It then presents containers and Docker as the execution substrate used by Kathará. The Kathará laboratory model is described through nodes, interfaces, collision domains, startup files, and node directories. The final sections introduce YAML, FRRouting, Wireshark, and the role of automated validation in a reproducible laboratory-generation process.

1.2 Experimental Network Environments

Network experimentation is required whenever a topology, protocol, service, or configuration must be studied without modifying an operational network. A small experiment may contain only two hosts and one router. A more complete scenario may include several access networks, a routing core, a server network, multiple routing protocols, and monitoring components. Reproducing such a scenario on physical equipment is possible, but it introduces costs and practical limitations.

Physical laboratories require dedicated devices, cabling, power, and space. They also make repeated experiments more difficult because the initial state must be restored before every execution. The configuration of one device may remain from a previous test, while a replacement device may use a different software version. These factors reduce reproducibility and make the experimental procedure dependent on the available hardware.

Software-based environments address these limitations by representing network nodes and links through processes, virtual machines, containers, or simulated models. A complete topology can be stored as files, recreated on demand, and shared with another user; container- and namespace-based emulation systems are commonly used to support rapid and repeatable prototyping [14, 26]. This approach is particularly appropriate for educational and research activities, where the objective is often to inspect the behaviour of protocols rather than to reproduce the exact performance of physical hardware.

The use of a software environment does not remove the need for careful design. Addresses, routes, interfaces, and services must still be mutually coherent. Automation is therefore valuable not only because it reduces typing, but because it makes it possible to derive several configuration artefacts from a single description.

1.3 Simulation and Emulation

The terms *simulation* and *emulation* are sometimes used interchangeably, but they describe different approaches. A simulator models the behaviour of a system through abstractions. Events such as packet transmission, propagation delay, queueing, or protocol transitions are represented by the simulation engine. The simulated node does not necessarily execute the same software that would run on a real host or router.

An emulator instead provides an execution environment in which real network stacks and applications can operate. The nodes may be implemented as virtual machines, containers, namespaces, or other isolated processes, but they execute real commands and exchange real packets. This gives the user access to standard Linux tools and routing daemons.

The distinction affects the interpretation of results. Simulation is suitable when the experiment requires controlled mathematical models, large synthetic networks, or repeatable event timing. Emulation is useful when the experiment focuses on configuration, interoperability, command-line behaviour, packet formats, and software services. The framework developed in this thesis uses emulation because its primary goal is to generate executable Kathará laboratories rather than abstract protocol models.

The use of emulation also explains why generated files remain important. A laboratory is not only a graph of nodes and links. Each node must receive Linux commands, files, and, where required, routing-daemon configuration. The framework must therefore translate an abstract topology description into a filesystem structure that can be executed by the emulator.

1.4 Virtual Machines and Containers

Virtual machines and containers are two common mechanisms for creating isolated execution environments. A virtual machine emulates or virtualises a complete computer and normally runs its own operating-system kernel. This provides strong isolation and makes it possible to run different operating systems on the same host. The cost is a comparatively large memory footprint, longer startup time, and additional management overhead.

Containers isolate processes while sharing the host kernel. Each container can have its own filesystem view, process namespace, network namespace, and resource limits [2, 4]. Since a container does not normally boot a complete guest kernel, it can be created and destroyed more quickly than a virtual machine. This property is useful for network laboratories, where a topology can contain many nodes that execute similar Linux environments.

Containers should not be interpreted as lightweight virtual machines in every respect. Their execution model and isolation boundaries are different. For the purposes of this thesis, however, the relevant property is that each container can act as an independent network node. It can own virtual interfaces, an IP routing table, processes, and configuration files. Multiple containers can therefore represent hosts and routers on a single workstation.

Container-based emulation also supports rapid regeneration. When the source configuration changes, the old laboratory can be stopped and a new set of containers can be created from the updated files. This matches the intended workflow of the framework: the YAML description is treated as the persistent source, while the running containers are temporary instances.

1.5 Docker as the Execution Substrate

Docker is the container runtime used by Kathará in the developed project [4]. A Docker image contains a filesystem and metadata from which containers are created. A container is a running or stopped instance of an image [2, 3]. Images make it possible to provide a consistent set of tools across nodes, while containers provide isolated runtime instances.

Two image categories are relevant to the generator. Ordinary end hosts use the `kathara/base` image by default. Nodes that require routing functions use `kathara/frr`. The generator may also accept an explicit image in the YAML configuration, allowing a node to override the default selection. This decision is implemented by considering both the declared node type and the presence of an enabled FRRouting section.

Docker supplies the low-level execution environment, but the framework does not directly create Docker networks or containers. That responsibility is delegated to Kathará. This separation avoids duplicating the emulator's functionality. The framework generates standard Kathará artefacts, and Kathará subsequently translates them into Docker operations.

This architecture has an additional advantage: the generated laboratory can be inspected and started without using the Python generator again. Once the files exist, standard Kathará commands are sufficient to operate the laboratory. The output therefore remains portable within the Kathará ecosystem.

1.6 Kathará

Kathará is a container-based network emulation system designed for teaching, research, protocol development, and controlled experimentation [10, 26, 1]. It represents network devices as containers and interconnects their interfaces through named Layer 2 collision domains. A laboratory is stored as a directory whose files describe the network and the configuration of each node [11].

1.6.1 Nodes and Interfaces

A Kathará node is identified by a name. The node can represent an end host, a router, a firewall, a server, or another Linux-based device. Interface indices are explicit in `lab.conf`: index 0 maps to `eth0`, index 1 to `eth1`, and so on [9]. In this framework, those indices are generated from the order of the interfaces in the YAML list.

A topology entry has the following form:

```
r1[0]="lan_a"
r1[1]="backbone"
```

Listing 1.1: Basic Kathará interface declaration.

The first line attaches `r1:eth0` to the collision domain `lan_a`. The second attaches `r1:eth1` to `backbone`. Nodes that refer to the same collision-domain name are connected at Layer 2.

The declaration does not assign an IP address. Layer 2 connectivity and Layer 3 configuration are deliberately separated. The topology file determines which interfaces share a network segment, while startup commands configure addresses and routes inside the containers.

1.6.2 The lab.conf File

The `lab.conf` file is the central topology file of a Kathará laboratory. In addition to interface declarations, it can contain laboratory metadata and node properties such as the selected Docker image. The generator produces entries such as:

```
LAB_DESCRIPTION="Example routed laboratory"
LAB_VERSION=2.0
LAB_AUTHOR="Luca-Bortoluzzi"

pc_a[0]="lan_a"
pc_a[image]="kathara/base"

r1[0]="lan_a"
r1[1]="backbone"
r1[image]="kathara/frr"
```

Listing 1.2: Simplified generated lab.conf content.

This file is derived from the YAML node and interface declarations. The order of interfaces in YAML determines the corresponding interface index in `lab.conf`. Preserving this relationship is important because the startup file assigns addresses to `eth0`, `eth1`, and the subsequent interfaces in the same order.

1.6.3 Startup Files

For a node named `r1`, Kathará executes the commands stored in `r1.startup` when the laboratory is started. Startup files commonly configure addresses, default gateways, static routes, forwarding, and services. A simplified file may contain:

```
ip address add 10.10.1.1/24 dev eth0
ip address add 10.0.12.1/30 dev eth1
systemctl start frr
```

Listing 1.3: Simplified node startup file.

The generator writes one startup file for each node. It also appends custom commands declared in YAML. This mechanism keeps common configuration automatic while retaining an escape hatch for scenario-specific behaviour.

1.6.4 Node Directories and hostlab

A directory with the same name as a node contains files that are made available inside that node. In the project workflow, resources stored beside a YAML configuration can be copied to the generated laboratory when the `--import-dirs` option is used. A directory named `server`, for example, is imported only if the YAML file declares a node named `server`.

These resources can be accessed from the container through the laboratory mount, commonly under `/hostlab`. This allows the startup file to execute scripts that remain visible and editable on the host. The generator excludes cache files and repository metadata so that generated laboratories do not contain unnecessary development artefacts.

1.6.5 Laboratory Lifecycle

A generated laboratory is started with `kathara lstart` from its directory. Kathará reads `lab.conf`, creates the containers, configures their interfaces, mounts node directories, and executes startup files. A running node can be accessed using `kathara connect`.

The laboratory is stopped and removed from the runtime with `kathara lclean`. The source files remain on disk and can be used for a later execution [11]. The project provides wrapper scripts for these operations; their implementation and the complete generation lifecycle are discussed in the following chapter.

1.7 Declarative Configuration with YAML

YAML is a human-readable data-serialisation language frequently used for configuration files [30]. It supports mappings, lists, scalars, comments, and nested structures. These properties make it suitable for representing a network laboratory in a form that is easier to maintain than several independent startup and routing files.

A declarative description states the desired structure of the laboratory. It does not require the user to write every command needed to create that structure. The following fragment describes a host and a router:

```
lab_name: example_lab

nodes:
  pc_a:
    type: host
    interfaces:
      - network: lan_a
        ip: 10.10.1.10/24
    default_gateway: 10.10.1.1

  r1:
    type: router
    interfaces:
      - network: lan_a
        ip: 10.10.1.1/24
      - network: backbone
        ip: 10.0.12.1/30
```

Listing 1.4: Simplified YAML topology declaration.

The same data are later translated into several outputs: interface entries in `lab.conf`, address commands in startup files, a default route for the host, and potentially routing configuration for the router. The YAML document therefore becomes the persistent source of truth for generated files.

Using a structured input also enables validation. The framework can check whether required fields exist, addresses are valid and unique, gateways belong to a local subnet, and routing declarations refer to existing interfaces. The validation process is a central subject of Chapter 3.

1.8 Dynamic Routing and FRRouting

FRRouting, commonly abbreviated as FRR, is an open-source routing suite for Linux and Unix systems. It provides implementations of protocols including OSPF, RIP, and BGP [6]. In the generated laboratories, FRR allows containerised nodes to exchange routes dynamically and behave as software routers.

FRR is organised around a set of protocol daemons and a common configuration interface. The `daemons` file determines which processes are enabled. The integrated `frr.conf` file contains protocol configuration, while `vtysh` provides a unified command-line interface for inspection and administration.

1.8.1 OSPF

Open Shortest Path First is a link-state interior gateway protocol. Routers exchange topology information and compute paths within an autonomous routing domain [19]. The project supports both a common area declared at node level and an `ospf_area` marker associated with individual interfaces. This allows simple single-area topologies and more structured multi-area configurations.

The generator can derive OSPF network statements from interface prefixes. Validation verifies that explicitly declared OSPF networks overlap a locally connected interface. This prevents a router from advertising a prefix that is unrelated to its own configuration.

1.8.2 RIP

Routing Information Protocol is a distance-vector protocol with a simpler configuration model [17]. In the project, the generator can enable the RIP daemon, select the protocol version, disable automatic summarisation, and derive network statements from node interfaces. RIP is useful for demonstrating that the framework is not tied to one routing protocol.

1.8.3 BGP

Border Gateway Protocol exchanges reachability information between autonomous systems [25]. A BGP configuration requires an autonomous-system number, a router identifier, one or more neighbours, and the prefixes to advertise. The framework validates neighbour addresses, remote autonomous-system values, and the existence of corresponding peer nodes.

The generated configuration includes two commands that are relevant in small FRR laboratories: `no bgp ebgp-requires-policy` and `no bgp network import-check`. Their use can be controlled through the configuration model. They allow deliberately minimal educational topologies to advertise prefixes without requiring additional policy definitions or an already installed matching route. The policy-related behaviour is particularly relevant to the default eBGP propagation model standardised in RFC 8212 [18].

1.9 Traffic Inspection with Wireshark

Wireshark is a network protocol analyser used to capture and inspect packets [27]. In an emulated environment it is useful for confirming that packets traverse the expected collision domain, inspecting protocol messages, and distinguishing routing failures from application failures.

The framework supports optional real-time Wireshark integration by adding a dedicated node directly to `lab.conf`. The user can attach the node to one or more selected collision domains. This is preferable to modifying an existing host because monitoring remains separated from the logical role of the emulated devices.

Packet capture is optional. A laboratory can be generated and executed without Wireshark, while the same topology can later be regenerated with monitoring enabled. This reflects a general design principle of the framework: diagnostic features should not be mandatory components of every laboratory.

1.10 Automation, Validation, and Reproducibility

Automation is often described as a means of reducing manual effort, but its more important contribution in this project is consistency. A manually edited laboratory may contain a valid `lab.conf` and valid startup files that nevertheless disagree with one another. A declarative generator can derive both artefacts from the same interface list.

The framework applies two levels of validation before generation. Structural validation verifies the minimum document shape required by the generator. Semantic validation analyses relationships among nodes, interfaces, subnets, gateways, and routing declarations. Errors stop generation, while warnings identify suspicious but potentially intentional configurations.

Reproducibility depends on preserving the source configuration and generation parameters. The generated laboratory is treated as a derived artefact. If a persistent

change is required, it should be made in the YAML file or in the node resources stored under `configs/`, not only in the generated `labs/` directory. A clean regeneration can then reconstruct the laboratory from the maintained source.

1.11 Positioning with Respect to Related Work

The framework combines ideas that are usually addressed separately in the literature: executable network emulation, declarative configuration, deterministic validation, and collection of application-level evidence. Its contribution is not a new container runtime or routing suite. Rather, it is the integration of these responsibilities around the standard Kathará laboratory format.

Mininet demonstrated that realistic network software could be used for rapid prototyping on a single machine, with particular emphasis on software-defined network experiments [14]. Kathará provides the execution substrate selected in this thesis and represents a broader range of Linux network nodes through containers and collision domains [1, 26]. The developed framework operates one level above Kathará: it does not replace its runtime or file format, but generates those files from a validated declarative source and adds repeatable lifecycle and measurement operations.

A second line of work studies the use of LLMs for network configuration. NetConfEval evaluates whether LLMs can perform network-configuration tasks, while more recent work considers their role in zero-touch management and surveys their wider use in network operations [29, 15, 16]. These works motivate natural-language assistance, but they also address a different abstraction boundary. The present framework does not attempt to infer intent: it receives explicit YAML and applies repeatable rules before producing an executable laboratory. An LLM can therefore be used upstream as an optional authoring aid, without becoming the authority that accepts a configuration.

Approach	**Scope and abstraction**	**Validation or evidence boundary**	**Relation to this work**
Mininet [14]	Rapid prototyping, especially of SDN networks, through programmatic topologies and host/switch configuration	Experiment-specific correctness checks remain the responsibility of the experimenter	Alternative emulation approach; not the selected output target
Kathará [1, 26]	Container-based execution from laboratory files, startup scripts, and node resources	Consumes and executes its own laboratory representation	Runtime and file format used by the developed framework
LLM-based network configuration [29, 15, 16]	Translation of natural-language intent and assistance with operational tasks	Generated output remains candidate configuration requiring technical verification	Optional upstream authoring and diagnostic support
This thesis	Validated generation and repeatable evaluation from declarative YAML	Structural and semantic checks precede generation; runtime measurements test observable behaviour	Integration of source configuration, standard artefacts, lifecycle, and evidence

Table 1.1: Positioning of the developed framework relative to the main related approaches. The rows describe complementary scopes rather than a performance ranking.

The distinction in Table 1.1 also limits the claims of the thesis. The work does not compare emulator throughput, evaluate LLM accuracy, or propose an alternative routing implementation. It evaluates whether a narrow, explicit configuration model can

coordinate existing tools, reject recognised inconsistencies before generation, and support a controlled experiment through a repeatable workflow.

1.12 Chapter Summary

This chapter introduced the execution model and technologies used by the framework. Container-based emulation provides isolated Linux network nodes, Docker supplies the runtime, and Kathará creates the topology from standard laboratory files. YAML provides a declarative source representation, FRRouting supplies dynamic routing, and Wireshark provides optional packet inspection.

The chapter also established the distinction between source configuration and generated artefacts. This distinction is central to the design of the system. The next chapter translates the technological context into explicit requirements, responsibilities, and architectural components.

Chapter 2

Requirements, Development Lifecycle and System Architecture

This chapter defines the problem addressed by the framework, its scope, its requirements, the development lifecycle followed during the project, and the architecture adopted by the final implementation. The framework accepts a YAML configuration, validates it, and produces a standard Kathará laboratory. It also provides lifecycle scripts through which a generated laboratory can be started, stopped, inspected, and removed. The detailed YAML model and the validation rules are presented in Chapter 3, while the translation of a valid configuration into `lab.conf`, startup files, FRRouting files, and node resources is discussed in the following chapter. The purpose of this chapter is therefore not only to state what the framework does, but also to explain why its main components were introduced, how problems encountered during development changed the requirements, and how the resulting responsibilities cooperate as a single system.

2.1 Problem, Scope, and Requirements

Kathará provides a flexible representation of network laboratories [10, 11], but the configuration of a non-trivial topology is normally distributed across several artefacts. A node interface is declared in `lab.conf`; its IP address is assigned in a startup file; the corresponding prefix may appear again in an FRRouting configuration; and scripts or other auxiliary resources may be stored in the directory associated with the node. Each individual file is readable, but the consistency of the complete laboratory must usually be maintained manually.

This becomes progressively more difficult as the topology grows. A change to a single address may require coordinated changes to a gateway, a routing neighbour, an advertised network, a static route, or another reference used by a node. The individual files may remain syntactically valid even when the laboratory is no longer semantically coherent, and the resulting error may only become visible after the emulated environment has started. Repetition is a related problem: hosts repeatedly require interface configuration and a default route, while routers repeatedly require interface addressing, an appropriate container image, and routing configuration. Rewriting the same information in multiple files increases the probability of inconsistent node names, IP addresses, and interface indexes.

A further limitation of the standard file-based workflow is the absence of an explicit validation boundary. Kathará configuration files are executable artefacts rather than a high-level schema that can be analysed before output is written. The proposed framework introduces such a boundary by treating a single declarative YAML document as the source

configuration, validating that document before generation, and only then producing the corresponding Kathará files. The general problem addressed by the work can therefore be stated as follows:

> How can a network laboratory be described once, checked for structural and semantic consistency, translated into standard Kathará artefacts, and managed through a repeatable lifecycle?

The framework is intentionally limited in scope. It does not replace Kathará, Docker, FRRouting, or Wireshark; instead, it acts as an automation layer that prepares input for those tools. The implemented scope includes YAML-based descriptions of laboratories, nodes, interfaces, addressing, gateways, static routes, custom commands, and routing protocols; structural and semantic validation before generation; generation of `lab.conf` and one startup file per node; generation of FRRouting files for OSPF, RIP, and BGP; optional import of directories associated with declared nodes; optional integration of a real-time Wireshark node; explicit handling of pre-existing output directories; shell wrappers for starting, stopping, and deleting laboratories; and support for multiple configurations and generated laboratories. The experimental components contained in the repository use these generated laboratories, but they are not part of the core configuration architecture discussed here; their workloads, measurements, and evaluation are addressed in later chapters.

The primary user is a student, researcher, or network experimenter who already understands basic Linux networking and needs to create repeatable Kathará laboratories. The framework does not hide networking concepts: the user still chooses addresses, prefixes, gateways, routing protocols, and neighbours. Its purpose is to avoid manually distributing the same design choices among many files. A secondary stakeholder is the reviewer of an experiment. Generated files must remain readable so that the relationship between the YAML source and the resulting Kathará configuration can be inspected. This requirement motivates the generation of ordinary text files rather than the use of a private runtime data store. The expected execution environment is a Linux workstation with Python, PyYAML, Docker, and Kathará installed [24, 4, 10]. FRRouting is provided by the appropriate Kathará image, while Wireshark integration also requires the selected container image and support for the exposed interface.

The resulting workflow can be summarised through a small set of principal use cases. The user may select a configuration by supplying either an explicit path or a configuration name resolved inside a configured search directory. The framework then loads the document with PyYAML and requires the top-level value to be a mapping. The user may validate the configuration without creating or modifying a laboratory: structural validation checks mandatory fields, while semantic validation checks relationships among addresses, collision domains, gateways, roles, and routing declarations, producing a report containing errors, warnings, and summary lines. After successful validation, the user may generate the laboratory under the selected output directory; the framework writes `lab.conf`, one startup file per node, and the required FRRouting files, while existing output is handled according to the selected `--force` or `--clean` behaviour. If directory import is enabled, a directory located beside the YAML source is copied only when its name matches a declared node; unmatched directories are reported as warnings and skipped. Wireshark can be disabled, selected interactively, attached directly to one or more collision domains, or associated with the networks connected to a specific node. Finally, the generated laboratory can be started, inspected, cleaned, and removed without deleting the original YAML configuration.

These use cases are formalised by the following functional requirements.

Table 2.1: Functional requirements of the framework.

ID	Requirement
FR1	The framework shall accept either an explicit YAML path or a configuration name resolved inside a configurable directory.
FR2	The framework shall load YAML through a safe parser and reject a top-level value that is not a mapping.
FR3	The framework shall require a laboratory name and a non-empty node mapping.
FR4	The framework shall validate identifiers, node types, interfaces, collision domains, addresses, gateways, and routing declarations.
FR5	The framework shall distinguish generation-blocking errors from non-blocking warnings.
FR6	The framework shall provide a validation-only mode that does not create or modify a laboratory.
FR7	The framework shall generate a Kathará `lab.conf` file from node-interface declarations.
FR8	The framework shall select a default container image according to node capabilities and permit an explicit image override.
FR9	The framework shall generate one startup file for every declared node.
FR10	Startup generation shall support interface addresses, a default gateway, static routes, FRR startup, and custom commands.
FR11	The framework shall generate FRRouting daemon and configuration files for enabled FRR devices.
FR12	The framework shall support OSPF, RIP, and BGP configuration forms represented by the YAML model.
FR13	The framework shall optionally copy node directories located beside the source YAML file.
FR14	The framework shall optionally attach a Wireshark node to selected collision domains.
FR15	The framework shall avoid silently overwriting an existing laboratory unless the user selects an explicit overwrite or clean-regeneration option.
FR16	The framework shall support a configurable output directory containing multiple generated laboratories.
FR17	The project shall provide commands or wrappers to start, stop, inspect, and delete generated laboratories.

In addition to these functions, the framework is constrained by a set of non-functional requirements that influence both the architecture and the implementation style.

Table 2.2: Non-functional requirements of the framework.

Property	Requirement
Reproducibility	The same maintained source configuration should regenerate the same logical laboratory structure.
Transparency	Generated files must remain ordinary, readable Kathará and FRRouting artefacts.
Safety of file operations	Existing laboratory directories must not be removed or overwritten implicitly.
Modularity	Loading, validation, generation, monitoring selection, and lifecycle operations should have separate responsibilities.
Extensibility	New configuration properties or protocol forms should be addable without replacing the entire generation pipeline.

Property	Requirement
Usability	Common operations should be available through a command-line interface with explicit options and informative messages.
Traceability	Validation messages should identify the affected node, interface, domain, neighbour, or field.
Portability	The project should operate in a standard Linux, Python, Docker, and Kathará environment.
Maintainability	Persistent modifications should be made in source YAML or resource directories rather than only in generated output.
Compatibility	The generated laboratory should remain operable with standard Kathará commands.

Requirements Traceability and Verification

Table 2.3 links each functional requirement to the implementation behaviour and to the evidence presented in the thesis. "Verified" denotes that the behaviour is demonstrated by the described validation or generation workflow and, where applicable, by execution of generated laboratories. It does not imply exhaustive testing of every possible input combination.

Table 2.3: Traceability matrix for functional requirements FR1–FR17.

ID	Implementation and verification evidence	Assessment
FR1	Path and configuration-name resolution are described in Section 3.1 and exercised by the generation workflow.	Verified
FR2	Safe YAML loading and rejection of a non-mapping root are part of the structural-validation boundary in Section 3.2.	Verified
FR3	Mandatory `lab_name` and non-empty `nodes` checks are documented with the structural rules in Section 3.2.	Verified
FR4	Identifier, role, interface, addressing, gateway, collision-domain, and routing checks are described in Sections 3.2 and 3.3.	Verified
FR5	The error/warning policy and its effect on generation are reported in Table 3.2.	Verified
FR6	Validation-only execution terminates before output generation, as described in Section 3.3; the large invalid stress case also produced no files in Section 6.1.	Verified
FR7	Translation from interface declarations to `lab.conf` is shown in Section 4.2 and Table 4.2.	Verified
FR8	Capability-based default-image selection and explicit overrides are documented in Sections 4.1 and 4.2.	Verified
FR9	Per-node startup-file generation is described and illustrated in Section 4.2.	Verified
FR10	Generated address, gateway, route, FRR-startup, and custom-command entries are mapped in Table 4.2.	Verified
FR11	Generation of `daemons` and `frr.conf` for FRR-enabled nodes is described in Section 4.2.	Verified
FR12	The supported OSPF, RIP, and BGP forms and their validation are documented in Sections 3.1 and 3.2; routed laboratories provide runtime use evidence.	Verified
FR13	Selective import of directories whose names match declared nodes is described in Sections 4.1 and 4.2.	Verified

ID	Implementation and verification evidence	Assessment
FR14	Wireshark selection, generated attachment, and a runtime capture are presented in Section 4.2 and Figure 4.2.	Verified
FR15	Refusal, explicit overwrite, and clean-regeneration behaviours are documented in Section 4.1.	Verified
FR16	Output-root selection and coexistence of named generated laboratories are described in Sections 4.1 and 4.3.	Verified
FR17	Start, stop, status, connection, clean, and deletion operations and their effects are reported in Section 4.3 and Table 4.3.	Verified

The matrix makes two boundaries explicit. First, FR12 is verified only for the protocol forms represented by the current YAML model, not for every command offered by FRRouting. Second, runtime execution complements but does not replace static verification: opaque custom commands and complete manual FRR fragments remain outside the semantics that the validator can prove.

2.2 Software Development Lifecycle and Design Evolution

The final architecture should not be interpreted as the implementation of a complete specification that existed before coding began. Development followed an iterative and incremental lifecycle in which a small working solution was extended after each cycle of generation, execution, observation, and correction. This approach was appropriate for the project because several requirements only became evident when real Kathará laboratories were produced and executed. In particular, failures that were invisible at the level of YAML syntax or generated text files often became apparent only when routes, services, or measurement scripts were exercised at runtime.

The initial requirement was deliberately narrow. Manually preparing a Kathará laboratory required the same design information to be distributed among `lab.conf`, node startup files, FRRouting configuration, and node-specific directories. One possible approach would have been to automate edits to these files individually, but that would have preserved the very duplication that motivated the project. A single declarative source was therefore preferred. YAML was selected because its hierarchical structure maps naturally to laboratories, nodes, interfaces, and nested routing properties, while remaining readable and directly editable. The first useful prototype could consequently describe a topology at a higher level and derive ordinary Kathará files from it.

Generation alone soon proved insufficient. A YAML document could be parsed and translated while still describing an inconsistent network: for example, an address could be duplicated, a default gateway could lie outside every local subnet, or a routing relationship could refer to the wrong peer. Allowing such configurations to reach the runtime moved deterministic errors to a much later and less convenient debugging stage. Validation therefore became an explicit architectural responsibility. Structural validation was introduced to protect the implementation from malformed document shapes, while semantic validation was added to reason about relationships among addresses, collision domains, gateways, node roles, and routing declarations before any output was written.

The next iterations were driven by increasingly complex routed laboratories. Simple host–router scenarios required only interfaces and gateways, whereas multi-area OSPF, RIP, and multi-AS BGP scenarios required protocol-specific identifiers, areas, autonomous-system numbers, neighbours, advertised prefixes, and redistribution rules. Rather than replacing the earlier configuration files whenever the representation grew, the implementation retained compatible forms where practical. This reduced disruption to already working laboratories and made them useful as regression inputs, at the cost of some redundancy in the accepted FRRouting syntax. The same pragmatic principle motivated

escape mechanisms such as `commands` and `frr.config`: they allowed the project to express specialised behaviour without first modelling every Linux or FRRouting command in the declarative language.

Execution of the generated laboratories then introduced a different class of requirements. Inspecting files was not enough to determine whether packets were actually following the intended path or whether an application was available. Optional Wireshark integration was added as a diagnostic aid, lifecycle operations were kept separate from generation, and connectivity testing was progressively extended from basic reachability towards application-level verification. During development, a reachable target could still fail a service test when the application process was not running; this made the distinction between network reachability and service availability explicit in the final measurement design. Similarly, sequential client tests were adequate for checking individual hosts but were unsuitable once several clients had to observe the same service condition. The test runner was therefore changed to a round-based concurrent model.

The experimental phase continued the same feedback loop. Earlier runs showed that application measurements could be contaminated by a network that had not yet reached a usable routing state. This motivated explicit readiness checks before collecting the baseline. As the repository accumulated several configurations and generated laboratories, utilities were also adapted to avoid assuming that only one laboratory existed and to use explicit targets or node names where appropriate. Finally, after functional behaviour was stable enough to support the controlled DoS experiment, development moved to verification of the translator itself. The stress and robustness tests in Chapter 6 exposed new trade-offs, including duplicated validation work, permissive unknown YAML keys, incomplete analysis of arbitrary static routes, and the scalability cost of collision-domain overlap checks.

Figure 2.1 summarises the lifecycle used throughout the project. Verification is not represented as a terminal activity: an observed problem can create a new or refined requirement and start another iteration.

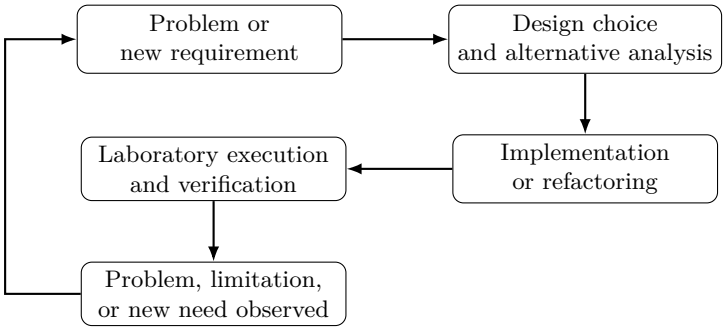


Figure 2.1: Iterative software-development lifecycle followed by the project.

The main iterations can be related directly to concrete features of the final framework, as summarised in Table 2.4. The table is not intended as a strict chronological log of every code change; instead, it identifies the recurring engineering pattern that connected requirements, observed problems, and implementation decisions.

Table 2.4: Main development iterations and resulting design choices.

Development stage	**Need or problem observed**	**Resulting design or feature**
Initial prototype	Repetitive and duplicated information across several Kathará artefacts.	A declarative YAML source from which standard Kathará files are generated.

Development stage	Need or problem observed	Resulting design or feature
Validation	Generated files could remain syntactically plausible while describing an incoherent network.	Separate structural and semantic validation before filesystem modification.
Routing extension	Larger laboratories required OSPF, RIP, BGP, multi-area, and multi-AS information.	Protocol-aware validation and FRRouting generation, while retaining compatible declaration forms.
Extensibility	Some node behaviour could not be represented by the current declarative fields.	Generic `commands`, imported node resources, and the `frr.config` escape mechanism.
Runtime diagnosis	Static inspection did not show actual packet paths or all runtime failures.	Optional Wireshark integration and explicit post-start inspection with Linux and FRR tools.
Service verification	A reachable host or open TCP port did not necessarily mean that the application completed the requested operation.	Application-level request validation and distinct ICMP, TCP, and application metrics.
Concurrent testing	Sequential client execution observed the service at different times.	Round-based concurrent measurements across legitimate clients.
Experiment orchestration	Early measurements could begin before the routed topology was ready.	Readiness checks, isolated experiment outputs, and explicit baseline/adverse phases.
Final verification	Functional success did not reveal all robustness or scalability limits.	Stress testing and analysis of permissive or expensive implementation choices.

2.3 System Architecture and Repository Organisation

The architecture is organised as a sequential pipeline with explicit responsibility boundaries. Figure 2.2 shows the main flow from source configuration to the running laboratory.

The pipeline does not construct a persistent hierarchy of domain objects such as `Node` or `Interface` classes. Introducing a typed internal model was considered a possible way to make invariants more explicit, but it would also have added another conversion layer between YAML and the relatively compact output model required by the prototype. The implemented version therefore keeps the validated configuration as a nested combination of Python dictionaries and lists. This choice reduced implementation overhead and made new YAML properties easy to prototype, but it also means that the validator is responsible for establishing all assumptions later used by the generator. The limitations of this choice become visible again in the discussion of strict schemas and unknown keys in Chapter 6.

The root-level `generate_lab.py` file is intentionally minimal: it imports and invokes `main()` from `src_gen_lab/gen_lab.py`. Configuration discovery and deserialisation are handled by `Lexer.py`. Despite its historical filename, this component does not perform token-by-token lexical analysis; its architectural role is to resolve the supplied configuration and load YAML. `Parser.py` performs the minimum structural checks and the complete semantic analysis. It defines the supported node and protocol sets, the identifier pattern, the validation issue representation, and the report used by both validation-only and normal generation modes. The generation module, `gen_lab.py`, then selects node images, generates `lab.conf`, writes startup files, creates FRRouting configuration, copies matching resource directories, integrates Wireshark when requested, and reports how the resulting laboratory can be started and stopped.

Lifecycle management is intentionally kept outside the generator itself. `start.sh`

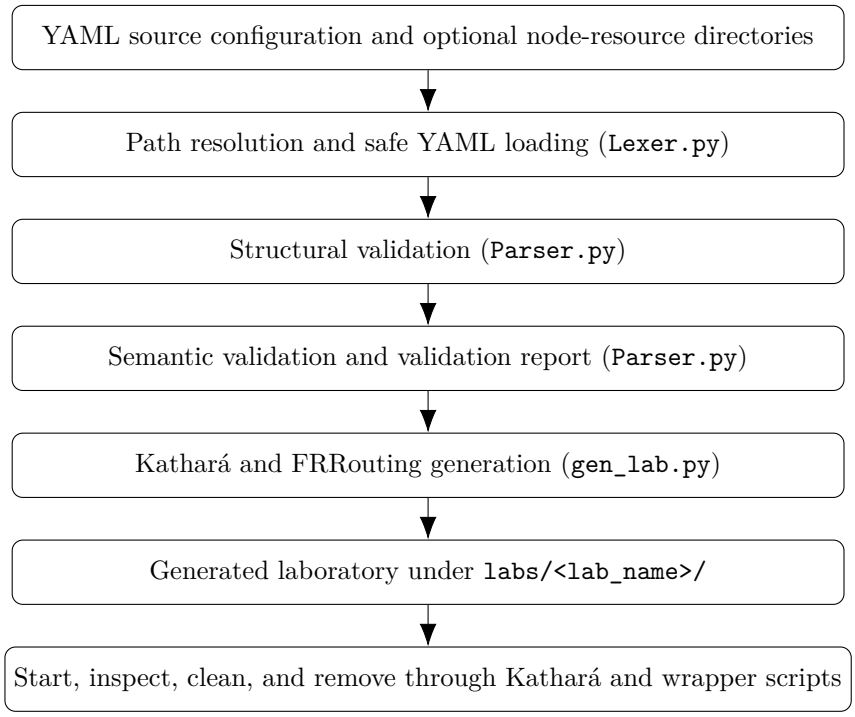


Figure 2.2: High-level architecture of the laboratory-generation framework.

verifies that the supplied laboratory directory exists, changes to that directory, and executes `kathara lstart`. `stop.sh` performs the corresponding `kathara lclean`, while `del_lab.sh` removes one generated directory or, when explicitly requested, the contents of the laboratory output directory. This separation keeps generation, runtime management, and filesystem deletion as distinct operations.

The repository contains both the core framework and the later experimental components. The following view includes only the files relevant to the configuration, generation, and lifecycle architecture discussed in the first part of the thesis.

```
Thesis/
|-- configs/
|   |-- Test1.yml
|   |-- Test_enterprise_multi_as_structured.yml
|   |-- client-router-server.yml
|   |-- dos_lab.yml
|   |-- mitigation.yml
|   |-- multi-area.yml
|   `-- multi_AS.yml
|-- src_gen_lab/
|   |-- Lexer.py
|   |-- Parser.py
|   |-- __init__.py
|   `-- gen_lab.py
|-- labs/
|-- generate_lab.py
|-- start.sh
|-- stop.sh
|-- del_lab.sh
`-- README.md
```

Listing 2.1: Repository structure relevant to the framework core.

The `configs/` directory is the persistent source area. It contains YAML topologies

and may also contain directories whose names match declared nodes. The `src_gen_lab/` package contains the implementation, whereas `labs/` contains generated output and is intentionally not expanded in the source-tree representation. This organisation expresses an important architectural distinction: source configuration is maintained independently from generated files and from the runtime state of the laboratory.

2.4 Operational Flow and Design Principles

The control flow begins when the user invokes the command-line entry point. The argument parser collects the requested configuration, the input and output directories, overwrite behaviour, validation mode, resource-import option, and Wireshark selection. The configuration path is then resolved. If the supplied argument already exists as a path, it is used directly; otherwise, the loader searches the configured directory for matching `.yml` and `.yaml` files. The YAML content is deserialised into ordinary Python objects.

Structural validation runs before semantic analysis so that subsequent functions do not have to operate on a document that lacks mandatory elements such as `nodes` or `interfaces`. Semantic validation then checks the relationships inside the model and deliberately collects all detectable errors and warnings instead of stopping at the first issue. If at least one error exists, generation is not started. If validation succeeds and `--validate-only` is active, the process terminates without modifying the filesystem. Otherwise, the generator prepares the selected output directory and writes all derived artefacts.

The generated directory is independent from the running state. Generation creates files; starting the laboratory creates containers and collision domains; stopping it removes the runtime environment; deleting the generated directory is yet another separate operation. Keeping these stages distinct prevents the project from conflating the declarative source, the generated representation, and the executing instance.

Several design principles follow directly from this flow. First, the YAML file is treated as the declarative source of truth for topology and common node configuration. Generated files are derived output, which reduces duplication and allows the laboratory to be recreated through clean generation. Second, the framework deliberately generates standard Kathará files instead of controlling containers directly through the Docker API or a private orchestration layer. Direct Docker control would have provided more runtime control, but it would also have duplicated responsibilities already implemented by Kathará and tied the project to its own execution path. By generating ordinary Kathará artefacts, the framework remains an input and validation layer rather than a replacement runtime; the output stays compatible with the normal Kathará workflow and can be inspected or executed manually [11].

Validation is also deliberately placed before filesystem modification. A structural or semantic error should not leave a partially generated laboratory, and the validation-only option makes the same checks available for manual review or automated workflows. Destructive operations follow the same principle of explicitness: an existing laboratory directory is not silently replaced. The user must select `--force` to write into existing output or `--clean` to remove and recreate it, making the potentially- destructive behaviour visible at the command line.

Resource import uses a convention rather than an additional mapping file. A node-specific directory is associated with a node when their names match. This choice is simple and readable, but it creates a deliberate naming convention; when import checking is enabled, directories that do not match declared nodes are reported and skipped. Diagnostics are likewise optional rather than mandatory. Wireshark integration can be disabled non-interactively, while the user may attach the monitoring node only to the collision domains required for debugging or inspection. The core topology can therefore be generated without monitoring, and observation can be added only when needed.

Together, these decisions support the non-functional objectives identified in Sec-

tion 2.1. Reproducibility is obtained by maintaining the YAML source and regenerating output from it; transparency and compatibility are preserved through ordinary Kathará and FRRouting files; safety is improved by validation-first and explicit overwrite semantics; modularity is reflected in the separation among loading, validation, generation, monitoring selection, and lifecycle operations; and traceability is provided by validation messages that refer to the affected node, interface, collision domain, neighbour, or field.

2.5 LLM-Assisted Configuration and Deterministic Validation

General-purpose LLMs can assist with configuration authoring, but authoring, validation, and experiment execution are distinct responsibilities. The relevant question is therefore not whether an LLM can emit configuration text, but which guarantees are required before that text is used in a repeatable experiment. In this thesis, RQ5 is addressed as a design and discussion question through architectural analysis and comparison with the literature; no empirical benchmark of LLM-generated configurations is claimed.

An LLM can translate informal intent into a candidate topology, allocate addresses, draft startup commands, produce FRRouting fragments, or create the YAML source accepted by the framework. Recent studies examine these capabilities in network configuration and intent-based management [29, 15, 16]. Such output is nevertheless candidate input rather than evidence that all addresses, gateways, routing relationships, node names, and cross-file references satisfy the laboratory invariants.

The framework provides the complementary deterministic boundary. It applies the same structural and semantic checks on every invocation, derives coordinated Kathará and FRRouting artefacts from one validated representation, and connects the generated laboratory to runtime measurements. This separation is important because technically plausible generated output may still contain functional or security weaknesses and therefore requires explicit checking [21]. The comparison in Table 2.5 summarises these complementary roles.

The comparison does not establish that either component is universally superior. An LLM is suited to interpreting ambiguous intent, drafting candidate configurations, explaining syntax, and helping a user interpret diagnostics. The framework is narrower: it enforces the implemented configuration model, maintains consistency among generated files, and produces the runtime evidence required for evaluation.

Combined authoring and validation workflow. The workflow does not require a model embedded in the framework. A user may ask an LLM to draft or refine YAML and then submit that candidate to the same validation pipeline used for manually written input. Invalid candidates are rejected with structured diagnostics; accepted candidates are translated into standard Kathará artefacts and executed. Logs, `vtysh` output, measurements, and packet captures can subsequently support human or LLM-assisted analysis.

The feedback path in Figure 2.3 allows validation diagnostics to guide a bounded human- or LLM-assisted repair step. The accepted YAML remains the authoritative, versionable input, while measurements and logs from the emulated network remain the evidence used to evaluate the experiment.

This architectural complementarity should not be interpreted as an empirical claim about LLM performance or as proof of complete network correctness. The current parser handles unknown keys permissively, and escape mechanisms such as `commands` and `frr.config` may contain behaviour outside its semantic model. Stricter schemas, repair policies, approval boundaries, and runtime verification are therefore future work, discussed further in Section 7.4.

Activity	**Typical LLM contribution**	**Framework contribution**
Expressing a network requirement	Accepts natural-language intent and can ask or answer clarifying questions.	Requires an explicit YAML representation; it does not interpret unrestricted natural language.
Drafting topology and routing data	Can propose node roles, addresses, protocol parameters, and configuration fragments.	Provides the supported field model and turns accepted fields into standard Kathará artefacts.
Checking known invariants	Can review a draft and suggest likely mistakes, but the result remains model-generated advice.	Applies explicit structural and semantic rules and blocks generation on recognised errors.
Maintaining cross-file consistency	Can generate several files, but consistency depends on the complete prompt, context, and generated output.	Derives multiple files from the same validated source representation.
Handling unusual or unmodelled behaviour	Can suggest arbitrary Linux or FRRouting commands and explain advanced features.	Supports controlled escape mechanisms such as `commands` and `frr.config`, but cannot fully validate their semantics.
Runtime diagnosis	Can interpret logs, command output, packet traces, and measured results supplied by the user.	Provides lifecycle operations, measurement tools, and optional packet capture that produce the evidence to be interpreted.
Reproducibility	Requires preservation of the selected prompt, context, model assumptions, and generated artefacts if the exact interaction is important.	Preserves a versionable source configuration and deterministically regenerates the supported laboratory structure.

Table 2.5: Complementary roles of LLM assistance and the developed framework.

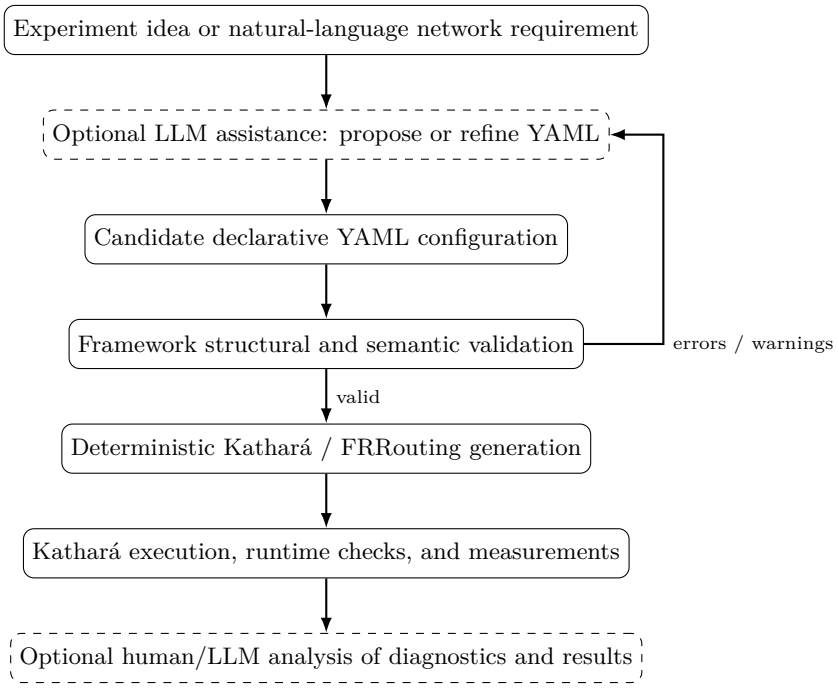


Figure 2.3: Recommended workflow in which LLM assistance remains optional and the framework provides the deterministic validation and generation boundary.

2.6 Architectural Boundaries and Chapter Summary

The architecture deliberately remains specific to Kathará and its file conventions. Supporting a different emulator would require a new output layer, and the current implementation should not be interpreted as a general network orchestration platform. The in-memory configuration is also based on nested mappings rather than explicit domain classes. This keeps the code compact for the present scope, but it makes disciplined validation and consistent key handling essential. A future version could introduce a formal schema or explicit classes for laboratories, nodes, interfaces, and routes.

The YAML model does not attempt to expose every Linux networking option or every FRRouting command. Advanced configurations can still be expressed through custom startup commands or by supplying a complete manual FRR configuration through the supported escape hatch. This retains flexibility, but semantic validation can only reason about the configuration properties it understands explicitly. Generated files may also be edited manually, but such changes do not become persistent source information: a clean regeneration will remove them. Long-term modifications must therefore be made in the YAML source or in the corresponding node-resource directory.

Within these boundaries, the framework implements a validation-first pipeline that converts a declarative source configuration into standard Kathará artefacts while preserving a clear distinction among source files, generated output, and running containers. The comparison with LLM-assisted configuration adds another boundary: natural-language reasoning and draft generation may occur outside the framework, but the accepted YAML source remains the explicit interface to deterministic validation and generation. The architecture defines responsibilities for configuration loading, structural and semantic validation, laboratory generation, optional diagnostics, and lifecycle management, while the functional and non-functional requirements establish the behaviour expected from each stage.

The next chapter examines the configuration language and the validation subsystem in detail. It describes the accepted YAML structures, path resolution and loading, structural checks, semantic rules, and the validation report used to decide whether

laboratory generation may proceed.

Chapter 3

Configuration Model and Validation

This chapter describes the configuration layer of the framework, that is, the representation used to describe a network laboratory before any Kathará file is generated. The framework adopts YAML as a declarative source format [30] and separates the description of the desired topology from the commands required to realise it. A single configuration can therefore contain the laboratory identity, nodes, interfaces, IP addressing, static routes, optional startup commands, and dynamic-routing parameters. The same source is then reused both for validation and, after validation has succeeded, for generation.

The implementation discussed here is contained mainly in `src_gen_lab/Lexer.py` and `src_gen_lab/Parser.py`. The historical filenames are retained in the repository, although their practical responsibilities are closer to configuration loading and validation than to a traditional compiler front end. The YAML document is loaded as nested Python dictionaries and lists; the framework does not construct a separate domain class for every node, interface, or routing protocol. Consequently, the accepted model is defined by the fields interpreted by the generator and by the structural and semantic checks implemented by the parser.

The chapter stops at the validated configuration. The creation of `lab.conf`, startup files, FRRouting files, optional monitoring components, and the laboratory lifecycle are discussed in the following chapter.

3.1 Configuration Model and Supported Operations

A configuration can be selected either by passing an explicit YAML path or by passing only its short name. The function `resolve_config_path()` first checks whether the supplied argument already identifies an existing file. If it does not, the function searches the configured directory, which defaults to `configs/`, and tries the extensions `.yml` and `.yaml`. This permits both forms shown in Listing 3.1.

```
1 python3 generate_lab.py configs/client-router-server.yml
2 python3 generate_lab.py client-router-server
```

Listing 3.1: Equivalent ways of selecting a configuration.

Once the path has been resolved, `load_yaml()` opens the file using UTF-8 encoding and calls `yaml.safe_load()`, following the safe-loading mechanism documented by PyYAML [24]. Safe loading is important for a configuration file because it avoids the general object-construction behaviour associated with unsafe YAML loading. The loader also checks that the top-level value is a Python dictionary; an empty value, scalar, or list is rejected before the network-specific validation begins. If PyYAML is unavailable, the loader reports the missing dependency explicitly.

Every laboratory configuration must contain at least `lab_name` and `nodes`. The remaining fields are optional and are included only when the corresponding behaviour is needed. A compact top-level example is shown in Listing 3.2.

```
lab_name: example_lab
description: Example routed laboratory

metadata:
  description: "Example generated with the framework"
  version: 2.0
  author: "Luca-Bortoluzzi"
  email: "luca.bortoluzzi921@edu.unito.it"
  web: "http://www.kathara.org/"

nodes:
  # node declarations
```

Listing 3.2: General top-level structure of a laboratory configuration.

The `lab_name` value identifies the generated laboratory and becomes the directory name under the selected output directory. The optional `description` and `metadata` mapping do not affect forwarding or addressing. They provide descriptive values later written to `lab.conf`; when a metadata field is omitted, the generator uses a fallback. The `nodes` field is a non-empty mapping in which each key is a node name and each value is the corresponding node declaration. Node names and collision-domain names must satisfy the same identifier rule:

```
^[A-Za-z0-9][A-Za-z0-9_.-]*$
```

Listing 3.3: Identifier pattern accepted for nodes and collision domains.

Therefore, letters, digits, dots, underscores, and hyphens are accepted, while spaces and identifiers beginning with an invalid character are rejected. The node name is not only descriptive: it is reused in generated filenames, `lab.conf`, and, when node resources are imported, in the matching node directory.

Listing 3.4 shows a representative router and annotates the principal fields. The example deliberately combines several features in one declaration; a normal node generally uses only the subset that is relevant to its role.

```
nodes:
  r1:                               # node name
    type: router                    # host | router | switch |
        firewall
    image: kathara/frr              # optional image override
    forwarding: true                # forwarding-consistency
        declaration

    interfaces:                     # ordered list: first entry ->
        eth0
      - network: lan_a              # collision-domain name
        ip: 10.10.1.1/24            # address in CIDR notation
        ospf_area: 10               # optional OSPF area

      - network: r1_r2              # second entry -> eth1
        ip: 10.0.12.1/30
        rip: true                   # optional RIP marker

    default_gateway: 10.10.1.254    # optional default-route next
        hop

    routes:                         # optional static routes
      - to: 192.168.20.0/24
```

```
      via: 10.0.12.2

  commands:                        # optional startup commands
    - sysctl -w net.ipv4.ip_forward=1
    - echo "r1 initialised"

  frr:                             # optional FRRouting
      configuration
    enabled: true
    protocols: [ospf, bgp]
    router_id: 1.1.1.1

    ospf:
      areas:
        - id: 10
          type: stub
          no_summary: false
      redistribute: [bgp]

    bgp:
      asn: 65001
      networks:
        - 10.10.1.0/24
      neighbors:
        - ip: 10.0.12.2
          remote_as: 65002
      redistribute: [ospf]
      ebgp_requires_policy: false
      network_import_check: false
```

Listing 3.4: Annotated node configuration reference.

Table 3.1 is the operational reference for node configuration. Each row corresponds to one operation that the user can express in YAML and states what the framework does with it. Related nested fields are grouped in the same row so that the complete model remains readable without turning each property into a separate subsection.

Table 3.1: Supported configuration operations and their effect.

Operation	**YAML field or syntax**	**Effect in the framework**
Identify the laboratory	`lab_name`	Mandatory top-level value. It identifies the laboratory and is later used as the generated directory name.
Add descriptive metadata	`description`, `metadata.description`, `metadata.version`, `metadata.author`, `metadata.email`, `metadata.web`	Adds human-readable information that is later transferred to `lab.conf`. These values do not change the topology or routing behaviour.
Create and name a node	`nodes.<name>`	Creates the node identifier. The name is reused in Kathará declarations, startup filenames, and optional node resource directories.

Operation	YAML field or syntax	Effect in the framework
Select the node role and image	`type`, `image`	`type` accepts `host`, `router`, `switch`, or `firewall`; omitted types are treated as hosts. `image` overrides automatic Docker-image selection. Ordinary hosts default to `kathara/base`; router, switch, and firewall nodes are treated as FRR-capable and default to `kathara/frr`.
Declare forwarding intent	`forwarding`	Provides a role-consistency declaration. A router explicitly configured with `forwarding: false` is rejected by semantic validation. The field does not itself generate a kernel `sysctl` command.
Create interfaces	`interfaces`	Mandatory non-empty ordered list. Entry zero becomes `eth0`, entry one becomes `eth1`, and so on. This ordering is later reused both in `lab.conf` and startup generation.
Connect an interface to a network	`interfaces[].network`	Mandatory for every interface. The value is the Kathará collision-domain name. Interfaces on different nodes using the same value belong to the same Layer 2 segment.
Assign an interface address	`interfaces[].ip`	Optional IPv4 or IPv6 interface in CIDR notation. During generation it becomes an `ip address add ... dev ethN` command. An interface can remain unnumbered when only Layer 2 attachment is required.
Mark interfaces for dynamic routing	`interfaces[].ospf_area`, `interfaces[].rip`	`ospf_area` associates the interface network with an OSPF area and requires an address. `rip: true` marks the interface network for RIP; if no interface is marked, RIP generation can fall back to all addressed interfaces.
Configure a default route	`default_gateway`	Optional next-hop address. During generation it becomes `ip route add default via ...`. Semantic validation requires the gateway to belong to one of the node's directly connected subnets.
Configure static routes	`routes`, `routes[].to`, `routes[].via`	Adds explicit routes to the startup file. Each mapping states a destination prefix in `to` and a next hop in `via`. The current parser does not perform the same depth of semantic analysis on arbitrary static routes as it does on interfaces and gateways.

Operation	YAML field or syntax	Effect in the framework
Append custom startup commands	`commands`	Appends the supplied strings to the startup file in declaration order. This is the generic escape mechanism for kernel settings, service startup, file manipulation, or other operations not represented by dedicated YAML fields. Arbitrary command semantics are not generally validated.
Activate FRRouting and select protocols	`frr`, `frr.enabled`, `frr.protocol`, `frr.protocols`, `frr.daemons`	Declares FRRouting behaviour. A single protocol can be selected with `protocol`; multiple protocols can be listed or configured through `protocols`; daemon flags can also imply OSPF, RIP, or BGP. The supported protocols are OSPF, RIP, and BGP.
Assign routing identifiers	`frr.router_id`, `frr.ospf.router_id`, `frr.bgp.router_id`	Sets a common router ID or a protocol-specific override. Router IDs are parsed as IPv4 addresses and must be unique across the effective declarations checked by the validator.
Configure OSPF	`frr.area`, `frr.ospf.networks`, `frr.ospf.areas`, `frr.ospf.areas[].id`, `frr.ospf.areas[].type`, `frr.ospf.areas[].no_summary`, `frr.ospf.areas[].totally_stub`, `frr.ospf.redistribute`	Supports the legacy common-area form, interface-level areas, explicit networks, stub or NSSA area declarations, optional no-summary behaviour, and redistribution. If interface-level `ospf_area` markers exist, they determine the generated network-area pairs; otherwise the generator falls back to the common area and FRR network list.
Configure RIP	`frr.rip.version`, `frr.rip.networks`, `frr.rip.auto_summary`, `frr.rip.redistribute`	Selects the RIP version, network statements, automatic summarisation behaviour, and redistribution. When no explicit network list is supplied, the generator uses RIP-marked interfaces or, as a fallback, all addressed interfaces.
Configure BGP	`frr.bgp.asn`, `frr.bgp.router_id`, `frr.bgp.networks`, `frr.bgp.neighbors`, `neighbors[].ip`, `neighbors[].remote_as`, `frr.bgp.redistribute`, `frr.bgp.ebgp_requires_policy`, `frr.bgp.network_import_check`	Defines the local AS, router ID, announced prefixes, peers and their remote AS values, redistribution, and the FRRouting eBGP-policy and network-import switches. The ASN must be in the range 1–4294967295. Peer addresses and remote AS values are checked against nodes declared in the same laboratory.

Operation	YAML field or syntax	Effect in the framework
Provide a complete FRR configuration	`frr.config`	Replaces automatic protocol-block generation with user-supplied FRRouting text written directly to `frr.conf`. It is intended for advanced configurations that exceed the declarative model.

The node types share this common syntax; the framework does not define a separate YAML language for hosts, routers, switches, and firewalls. Their practical differences derive mainly from validation expectations and automatic image selection. A host normally contains one or more interfaces, an optional default gateway, and possibly startup commands. A router normally contains at least two addressed interfaces and either static routes or an FRRouting configuration. In the current implementation, `switch` is only a role label used by validation and image selection: it does not create a Linux bridge, Open vSwitch instance, or any other Layer 2 forwarding function. Kathará's `device[n]` syntax only attaches `ethn` to the named collision domain [9]; consequently, two interfaces on a node do not by themselves join two collision domains. A functioning Layer 2 switch requires explicit bridge or Open vSwitch configuration, for example through imported resources and startup commands or a suitable image such as `kathara/openvswitch` [8]. A firewall uses the same interface model and currently relies on generic commands or imported resources for policy-specific behaviour because the YAML model does not define a dedicated firewall-policy language.

The `image` field allows these defaults to be overridden. If it is absent, ordinary hosts use `kathara/base`; nodes whose type is `router`, `switch`, or `firewall` are considered FRR-capable and use `kathara/frr`. A host can also become FRR-capable by providing an FRR declaration. One implementation detail deserves attention: for router, switch, and firewall types the device type itself is sufficient to select the FRR execution path. Therefore, `frr.enabled` should not be interpreted as a universal switch that disables FRR for every node type; its behaviour is most meaningful for a host whose FRR capability comes from the `frr` section. This is one of the implementation details that a future revision could make more uniform.

Interfaces are the core of the topology model. Each node must contain a non-empty `interfaces` list and every interface must have a `network` field. The network name represents a Kathará collision domain, while the optional `ip` field assigns an IPv4 or IPv6 address in CIDR notation. List order is operational: the first interface becomes `eth0`, the second `eth1`, and so forth. The same order is later used when the generator creates `lab.conf` and node startup commands. Listing 3.5 shows the minimal pattern used by a routed host.

```
pc_a:
  type: host
  interfaces:
    - network: lan_a
      ip: 10.10.1.10/24
  default_gateway: 10.10.1.1
```

Listing 3.5: Host with one addressed interface and a default gateway.

An interface may omit its address, which is useful for an unnumbered or Layer 2-only attachment. Protocol-specific fields can impose stronger requirements: for example, an interface carrying `ospf_area` must also have an IP address because the generator derives the advertised prefix from that address. The optional default gateway is validated as an IP address and must fall inside at least one locally connected subnet, which prevents a syntactically valid but Layer 2-unreachable gateway from reaching generation. Additional routes are declared through `routes`; every route contains a `to` destination and a `via` next hop. The generator translates these mappings directly into `ip route add` commands.

The current validator intentionally devotes more semantic analysis to interfaces, subnets, gateways, and dynamic-routing relationships than to arbitrary static route policies, so malformed or logically inappropriate custom route content is less extensively checked.

The `commands` list was introduced as a pragmatic escape mechanism for behaviour outside the formal model. During development, not every node-specific operation justified a new dedicated YAML field: application startup, temporary kernel settings, and scenario-specific file operations could otherwise have forced the configuration language to grow around individual experiments. Each command string is therefore copied to the node startup file in declaration order. This choice kept the core model small and allowed specialised scenarios to be implemented without blocking on a new parser feature.

The trade-off is that command text is opaque to the general parser, so the framework cannot prove its correctness. It performs only limited consistency checks, such as detecting an explicit `ip_forward=0` setting on a node declared as a router. Similarly, `forwarding: true` is a declaration of intent and validation consistency; it does not by itself emit a forwarding command. When explicit kernel configuration is required, it must be expressed through `commands` or provided by the selected container environment. Chapter 6 later shows why this useful development shortcut also creates a trust and robustness boundary.

Dynamic routing was added as the laboratories evolved beyond static host–router scenarios. Supporting OSPF, RIP, and BGP required the configuration language to represent protocol-specific information that was not needed by the first prototypes. The model was therefore extended incrementally instead of being replaced by a new syntax. For compatibility with configurations developed at different stages of the project, the generator accepts several declaration forms. A single protocol can be written as `frr.protocol`; multiple protocols can be placed in a list under `frr.protocols`; `frr.protocols` can also be a mapping containing protocol-specific options; dedicated `frr.ospf`, `frr.rip`, and `frr.bgp` mappings are recognised; and daemon flags under `frr.daemons` can imply the corresponding protocol. These forms are illustrated in Listing 3.6.

```
# One protocol
frr:
  enabled: true
  protocol: ospf

# Multiple protocols
frr:
  protocols: [ospf, bgp]

# Structured multiprotocol form
frr:
  protocols:
    ospf:
      enabled: true
      redistribute: [bgp]
    bgp:
      enabled: true
      asn: 65001

# Direct daemon selection retained for compatibility
frr:
  daemons:
    ospfd: true
    ripd: false
    bgpd: true
```

Listing 3.6: Compatible FRRouting protocol declaration forms.

Retaining these forms was a deliberate compatibility decision. Requiring an immediate migration to one final representation would have invalidated earlier test topologies precisely when they were useful for regression testing. The cost is a broader parser

and some representational redundancy. For a mature configuration language, a single versioned schema would be preferable; the strict-schema direction discussed in Chapter 6 is intended to reduce this ambiguity without losing the lessons provided by the earlier files.

OSPF can use a common legacy area declared through `frr.area`, or each interface can carry its own `ospf_area`. The latter form is used by the multi-area examples because it keeps area membership next to the interface whose network will be advertised. When one or more interface area markers are present, the generator derives network-area pairs from them. If no such marker exists, the generator falls back to the FRR network list and the common area, which defaults to area 0. The structured OSPF mapping can also contain explicit networks, area declarations, stub and NSSA area types, optional `no_summary` or `totally_stub` flags, and redistribution. The underlying OSPFv2 and NSSA concepts are defined in RFC 2328 and RFC 3101 [19, 20]. Listing 3.7 shows a concise multi-area example.

```
r1:
  type: router
  interfaces:
    - network: lan_a
      ip: 10.10.1.1/24
      ospf_area: 10
    - network: backbone
      ip: 10.0.12.1/30
      ospf_area: 0
  frr:
    enabled: true
    protocol: ospf
    router_id: 1.1.1.1
```

Listing 3.7: Interface-level OSPF area configuration.

RIP follows the same general model and follows the RIPv2 concepts defined in RFC 2453 [17]. It can be selected through the common protocol field or a dedicated `frr.rip` mapping. The configuration can choose the protocol version, control automatic summarisation, provide an explicit network list, and request redistribution. Interface-level `rip: true` markers provide a concise way to select the participating networks. If no marked interfaces or explicit networks are provided, the generator can derive RIP networks from all addressed interfaces.

BGP requires additional relationship information consistent with the BGP-4 model defined in RFC 4271 [25]. The local autonomous-system number can be written at the common FRR level or inside the BGP mapping; the structured value takes precedence. BGP can also define a router ID, announced networks, neighbours, redistribution, and two switches used by the generated FRRouting configuration: `ebgp_requires_policy` and `network_import_check`. The former is directly related to the default external-BGP policy behaviour specified by RFC 8212 [18]. Each neighbour contains an `ip` and a `remote_as`. Listing 3.8 shows the compact form used in the multi-AS configuration.

```
r1:
  type: router
  interfaces:
    - network: lan_a
      ip: 10.10.1.1/24
    - network: r1_r2
      ip: 10.0.12.1/30
  frr:
    enabled: true
    protocol: bgp
    asn: 65001
    router_id: 1.1.1.1
    networks:
```

```
      - 10.10.1.0/24
    neighbors:
      - ip: 10.0.12.2
        remote_as: 65002
```

Listing 3.8: BGP node declaration.

A common or protocol-specific router ID is interpreted as an IPv4 identifier. The common value under `frr.router_id` is used unless the OSPF or BGP section provides its own override. For configurations that cannot be expressed conveniently through these fields, `frr.config` acts as a final escape hatch: its complete multiline value replaces automatically generated protocol blocks and is written directly to `frr.conf`. This preserves extensibility without requiring the YAML model to represent every FRRouting command.

Not every key that may appear in a repository YAML file is part of the core generation language. YAML loading preserves unknown mappings because the configuration remains an ordinary Python dictionary, but only fields explicitly interpreted by the generator have a generation effect. For example, a scenario can contain descriptive or auxiliary sections, and some example files contain additional information used by separate tools. Such data is not automatically converted into Kathará configuration simply because it is present in the YAML document. This distinction is important when reading the repository: the accepted YAML document can be broader than the core node model documented in Table 3.1.

3.2 Validation Process and Semantic Rules

Validation is divided into two stages because malformed structure and network-level inconsistency require different handling. Structural validation is implemented by `validate_config()` and is intentionally fail-fast. It verifies the assumptions that must hold before more detailed analysis can run: `lab_name` must exist; `nodes` must exist and be a non-empty dictionary; every node declaration must itself be a dictionary; every node must contain a non-empty interface list; every interface entry must be a dictionary; and every interface must contain `network`. A violation raises `ValueError` immediately. This prevents a missing basic field from producing secondary implementation errors during semantic analysis. For example, collision-domain grouping cannot produce a meaningful diagnostic if an interface is not a mapping or has no network name.

Semantic validation is implemented by `validate_semantics()` and uses a different policy: rather than stopping after the first network-level problem, it collects all detectable issues in one `SemanticValidationReport`. Each issue is represented by an immutable Python dataclass [22], shown in Listing 3.9.

```
@dataclass(frozen=True)
class ValidationIssue:
    severity: str
    code: str
    message: str
```

Listing 3.9: Representation of a semantic validation issue.

The severity is `ERROR` or `WARN`. The code provides a stable machine-oriented category and the message gives the human-readable context. The report stores the node, interface, and collision-domain counts, records the number of OSPF and BGP nodes, tracks whether resource directories were checked, and holds the complete issue list. Computed properties expose the filtered errors and warnings and a `valid` flag. A report is valid when it contains no error; warnings do not block generation. This distinction allows unusual but potentially intentional topologies to proceed while still drawing attention to suspicious choices.

The first semantic rules concern identifiers and node roles. Node names and collision-domain names are checked against the regular expression introduced in Section 3.1. The supported node types are `host`, `router`, `switch`, and `firewall`; any other value is an error. Routers receive two additional consistency checks. If `forwarding` is explicitly false, or if the custom command list contains a setting equivalent to `ip_forward=0`, the declaration is rejected because the node role conflicts with its stated forwarding behaviour. A router with only one interface is not structurally impossible, but it cannot forward between two network segments, so the validator reports a warning rather than an error.

Interface addresses are parsed with Python's `ipaddress.ip_interface()` function [23]. Both IPv4 and IPv6 interface notation are accepted by this generic parser. Malformed values generate an error. For IPv4 networks with a prefix shorter than `/31`, assigning the network address or broadcast address to an interface is also rejected. Every valid host address is indexed across the complete laboratory, allowing duplicate addresses to be reported with all owning node-interface pairs. The implemented rule is global: two isolated segments are not allowed to reuse the same address, even if they would never communicate. This reflects the framework's assumption of one coherent address space per laboratory and simplifies subsequent target and topology handling.

Collision domains are then checked as Layer 2 segments. The validator groups all endpoints sharing the same `network` name. If a domain has only one endpoint, the condition is reported as a warning because it may be deliberate, but it often indicates a misspelled network name or a missing peer. Addressed interfaces belonging to the same collision domain are expected to use one coherent subnet; if more than one distinct subnet appears on the same domain, the configuration is rejected. The validator also compares the subnets used by different collision domains and rejects overlapping prefixes. This avoids a topology in which separate Layer 2 segments are represented by identical or ambiguous Layer 3 address spaces.

The default gateway is checked after interface parsing. The value must be a valid IP address and must belong to at least one local interface subnet of the same node. A gateway that is syntactically valid but outside all locally connected networks is therefore rejected before generation. This rule is particularly useful for host declarations, where a wrong gateway can otherwise produce a laboratory that starts correctly while being unable to reach any remote subnet.

Some additional role-oriented checks exist in the current parser because the repository contains auxiliary scenarios built on top of the same node model. These checks require an address for certain reserved role naming conventions. They are implementation-specific conventions rather than a general property of Kathará or of the four node types, and they do not change the generic configuration operations documented in this chapter.

FRRouting validation begins by normalising the different compatible protocol declaration forms. The helper that collects declared protocols accepts `protocol`, a list or mapping under `protocols`, the dedicated `ospf`, `rip`, and `bgp` mappings, and daemon flags. Any protocol outside OSPF, RIP, and BGP is reported as unsupported. The parser also requires the `frr` section, when present and active, to be a dictionary. Effective router IDs are collected from the common field and from protocol-specific overrides. Each router ID must parse as an IPv4 address, and duplicate effective IDs across nodes are rejected.

For OSPF, an interface marked with `ospf_area` must contain an IP address. Explicit OSPF network values are parsed as prefixes and must overlap at least one network directly connected to the node. This detects stale or copied network statements that do not correspond to the actual interface addressing. The check is deliberately local: it verifies that the node's declaration is coherent, but it does not attempt to prove global area connectivity or protocol convergence.

For BGP, the local autonomous-system number must be an integer from 1 to 4294967295. Neighbours must be expressed as a list of mappings. Each neighbour IP must be valid, must belong to an interface of a declared node, and cannot refer back to the local node itself. The neighbour's `remote_as` must also be valid and must match the ASN configured on the peer node. If the peer has no valid BGP ASN, the relationship is rejected. After these mandatory checks, the parser also examines reciprocity: when

node A declares an address of node B as a neighbour but node B does not declare any address of node A, a warning is emitted. Reciprocity is treated as a warning rather than an error because an incomplete adjacency may be intentional while a topology is being constructed incrementally.

The validator also performs reference-integrity checks for auxiliary configuration data stored in the same YAML document. A visible generic example is the mapping used for connection tests, named `connection_tests` in the YAML files. A declared client must refer to an existing node, a target node must exist and have an address, and a target IP must be valid and assigned to a declared interface. These checks do not execute a test and do not make the auxiliary subsystem part of laboratory generation; they merely ensure that references embedded in the source configuration remain consistent with the topology. Other auxiliary sections can be checked through the same internal reference mechanism without altering the node model.

When node-directory import validation is requested, the parser examines visible directories beside the selected YAML file. Directories such as `__pycache__` and hidden entries are excluded. A visible directory whose name does not correspond to a declared node is reported as a warning and will be skipped by the generator. This policy is useful because the shared `configs/` directory can contain resources for several laboratories at the same time; an unrelated resource directory should be reported but should not automatically invalidate an otherwise correct topology.

Table 3.2 summarises the principal rules. It combines the checks described above without introducing a separate subsection for every category.

Table 3.2: Representative validation rules and their consequences.

Condition	**Severity**	**Consequence**
Missing `lab_name`, missing or empty `nodes`, malformed node or interface structure, or missing interface `network`	Error	Structural validation stops immediately because later analysis cannot operate safely.
Invalid node or collision-domain identifier	Error	Generation is blocked.
Unsupported node type	Error	Generation is blocked.
Malformed interface address	Error	Generation is blocked.
IPv4 network or broadcast address assigned to an ordinary host interface prefix shorter than /31	Error	Generation is blocked.
Duplicate interface address anywhere in the laboratory	Error	Generation is blocked.
Collision domain with only one endpoint	Warning	Generation may continue; the domain may be deliberately isolated or may contain a naming mistake.
Different subnets used on the same collision domain	Error	Generation is blocked because one Layer 2 segment has inconsistent Layer 3 addressing.
Overlapping subnets used on different collision domains	Error	Generation is blocked because the address model is ambiguous.
Invalid default gateway or gateway outside every local subnet	Error	Generation is blocked.
Router with one interface	Warning	Generation may continue, but the declared router cannot forward between distinct segments.
Router forwarding explicitly disabled	Error	Generation is blocked because the declaration contradicts the router role.
Invalid, duplicated, or unsupported FRRouting declaration	Error	Generation is blocked when the affected routing configuration cannot be considered coherent.

Condition	Severity	Consequence
OSPF area marker without an IP address or explicit OSPF network disconnected from every local interface	Error	Generation is blocked.
Invalid BGP ASN, invalid peer address, self-neighbour, missing peer node, or remote-AS mismatch	Error	Generation is blocked.
BGP adjacency declared in only one direction	Warning	Generation may continue while the incomplete or intentionally asymmetric declaration is reviewed.
Auxiliary target or client reference to a missing node or undeclared address	Error	The source configuration is inconsistent and generation is blocked.
Resource directory without a matching node when import checking is enabled	Warning	The directory is reported and skipped; the topology can still be valid.

A valid report can therefore contain warnings. The `valid` property is based only on the absence of errors. The report's helper methods can append errors and warnings and determine whether specific error-code prefixes are present, which allows the textual summary to suppress an `[OK]` message when a related error already exists. A successful run can produce output of the form shown in Listing 3.10.

```
[OK] 9 nodes validated
[OK] 8 collision domains validated
[OK] IP addresses are valid and unique
[OK] Subnets and default gateways are coherent
[OK] OSPF configuration is coherent on 5 nodes
[OK] Semantic validation passed with 0 warning(s)
```

Listing 3.10: Example of a successful semantic validation report.

This two-stage design is useful from a usability perspective and also reflects an implementation problem observed during development. Structural errors must be reported immediately because the document cannot be interpreted safely, but stopping semantic analysis after its first error made larger topologies slow to correct: fixing one address or routing inconsistency could simply reveal the next independent problem on the following run. Semantic validation was therefore designed to collect multiple issues in one report. A user can correct duplicated addresses, an incorrect gateway, and a BGP remote-AS mismatch together instead of discovering them through three consecutive generator failures. In this sense, the report format is not only diagnostic output; it is a direct optimisation of the edit–validate–correct development cycle.

3.3 Validation Workflow and Limitations

The command-line option `--validate-only` exposes the validation layer without creating or modifying a laboratory. The workflow resolves the configuration path, loads the YAML document, executes structural validation, executes semantic validation, prints the resulting report, and terminates before output-directory preparation. A typical invocation is shown in Listing 3.11.

```
python3 generate_lab.py multi-area --validate-only
```

Listing 3.11: Validating a configuration without generating a laboratory.

This mode is useful while editing a topology because validation can be repeated without changing generated files. It can also be incorporated into a version-control or continuous-integration workflow. Normal generation performs semantic validation again

before writing output. The repetition is intentional: the generation function should not rely on being called only through one command-line path, so it protects itself by validating the configuration it receives.

The repository contains several configurations of increasing complexity that exercise the same core model. `client-router-server.yml` represents a minimal routed topology and is useful for checking the fundamental mapping of hosts, interfaces, collision domains, and default gateways. `Test1.yml` contains several client LANs, a server-side network, and a mesh of OSPF routers; it exercises a larger single-area addressing and routing configuration and also contains auxiliary connection-test references that can be checked for integrity. `multi-area.yml` associates OSPF areas directly with interfaces and demonstrates that the area assignment can remain close to the prefix it affects. `multi_AS.yml` assigns different BGP autonomous systems to routers and declares explicit peers and advertised LAN prefixes, thereby exercising ASN, neighbour-address, remote-AS, reciprocity, and router-ID checks. Finally, `Test_enterprise_multi_as_structured.yml` combines several routing styles and uses the structured multiprotocol representation, including OSPF area options, RIP markers, BGP sessions, and redistribution. Together, these files act both as examples of the accepted language and as regression inputs during framework development.

The current configuration model remains intentionally pragmatic. It is not formalised through a separate YAML Schema or JSON Schema document; the Python implementation is the authoritative definition of accepted structure and semantics. This keeps the project compact, but external editors cannot derive the complete language from a standard schema file. The configuration also remains a hierarchy of dictionaries and lists rather than a typed internal object model, so some assumptions are distributed across the loader, validator, and generator.

Not all aspects of a node can be validated statically. Custom startup commands are arbitrary shell strings, and a complete manual `frr.config` block is opaque to most semantic checks. The framework can validate the surrounding node and interface model, but the correctness of user-supplied command text is ultimately the user's responsibility. Static route entries are similarly less richly analysed than addressing and dynamic-routing relationships. A future version could define stricter schemas for these structures while retaining a controlled escape mechanism for advanced use cases.

Some current rules depend on project conventions rather than on a universal network model. The role-oriented address checks mentioned above rely on node naming conventions. The global duplicate-address rule also assumes one address namespace for the complete laboratory; supporting overlapping addresses in separate routing instances would require a richer representation of routing contexts. The resource-directory check relies on exact correspondence between a directory name and a declared node. These choices are reasonable for the current framework because they simplify generation and diagnostics, but they should be understood as implementation constraints rather than Kathará requirements.

The FRRouting model is intentionally broad enough for the topologies developed in the project but does not attempt to expose the entire FRRouting command language. OSPF, RIP, and BGP are supported, including multiprotocol routers and redistribution, while advanced features can be supplied through `frr.config`. As noted earlier, FRR capability is partly inferred from node type and partly from the `frr` section. This produces a small asymmetry around `frr.enabled` for nodes that are already typed as routers, switches, or firewalls. Making capability selection explicit and uniform would be a useful refinement.

Another important limit is that semantic validation proves configuration coherence, not runtime correctness. A YAML file can have valid identifiers, unique addresses, consistent subnets, reachable gateways, and coherent routing relationships while the resulting network still fails to converge because of a runtime condition, a user-supplied command, a container-specific issue, or a more subtle protocol property not represented by the validator. Application availability is even further outside the scope of this stage. These properties must be checked after generation and execution. The value of the validation

layer is therefore preventive: it eliminates a large class of deterministic configuration mistakes before containers are started, reducing the number of runtime failures that must be diagnosed manually.

In summary, the YAML document is the authoritative description of a laboratory before generation. It can be selected by name or path, loaded safely, and expressed through a compact set of operations covering node identity, type, interfaces, addressing, gateways, static routes, custom startup behaviour, and OSPF, RIP, or BGP configuration. Structural validation protects the framework from malformed input, while semantic validation examines the relationships among those fields and distinguishes blocking errors from non-blocking warnings. The result is a validated Python representation that can be handed to the generation stage with a substantially lower risk of deterministic configuration errors. The next chapter describes how this validated source is translated into standard Kathará artefacts and how the resulting laboratory is managed during its lifecycle.

Chapter 4

Laboratory Generation and Lifecycle Management

The previous chapter described the YAML configuration model and the validation rules applied before a laboratory is created. At that point the framework has a validated Python representation of the intended topology, but no Kathará laboratory has yet been written to disk. This chapter follows the next stage of the workflow: the transformation of that representation into standard Kathará artefacts and the management of the resulting laboratory during its operational lifecycle.

The implementation discussed here is centred on `generate_lab.py`, `src_gen_lab/gen_lab.py`, and the lifecycle wrappers `start.sh`, `stop.sh`, and `del_lab.sh`. The objective is not to introduce another execution environment on top of Kathará. Instead, the framework generates the same files that would normally be written by hand: `lab.conf`, one startup file for each node, optional per-node resource directories, and FRRouting configuration files where required. Once those artefacts have been produced, the laboratory can be started and inspected with ordinary Kathará commands [11].

The chapter is deliberately restricted to topology generation and lifecycle management. It therefore describes how a laboratory is produced, started, inspected, stopped, and removed, while later experimental procedures are kept separate from the generation framework itself.

4.1 Laboratory Generation Process

The public entry point of the generator is the executable Python script `generate_lab.py`. Its implementation is intentionally minimal: it imports the `main()` function from `src_gen_lab/gen_lab.py` and returns its exit status. This separation became more important as generation grew beyond the first prototype. Keeping argument handling, validation, FRRouting generation, resource import, and optional monitoring in one root-level script would have increased coupling and made individual functions harder to reuse or test. The implementation was therefore organised so that the public command remains small while the generation logic lives in a dedicated module. The practical effect is to keep core functions available independently of the wrapper without turning the project into a monolithic command-line script.

```
1 #!/usr/bin/env python3
2 import sys
3 from src_gen_lab.gen_lab import main
4
5 if __name__ == "__main__":
6     sys.exit(main())
```

Listing 4.1: Command-line wrapper used to start the generator.

The complete generation workflow is shown conceptually in Figure 4.1. The first three steps correspond to the loading and validation process described in Chapter 3. Only after both structural and semantic validation have succeeded does the framework prepare the output directory and begin writing Kathará files.

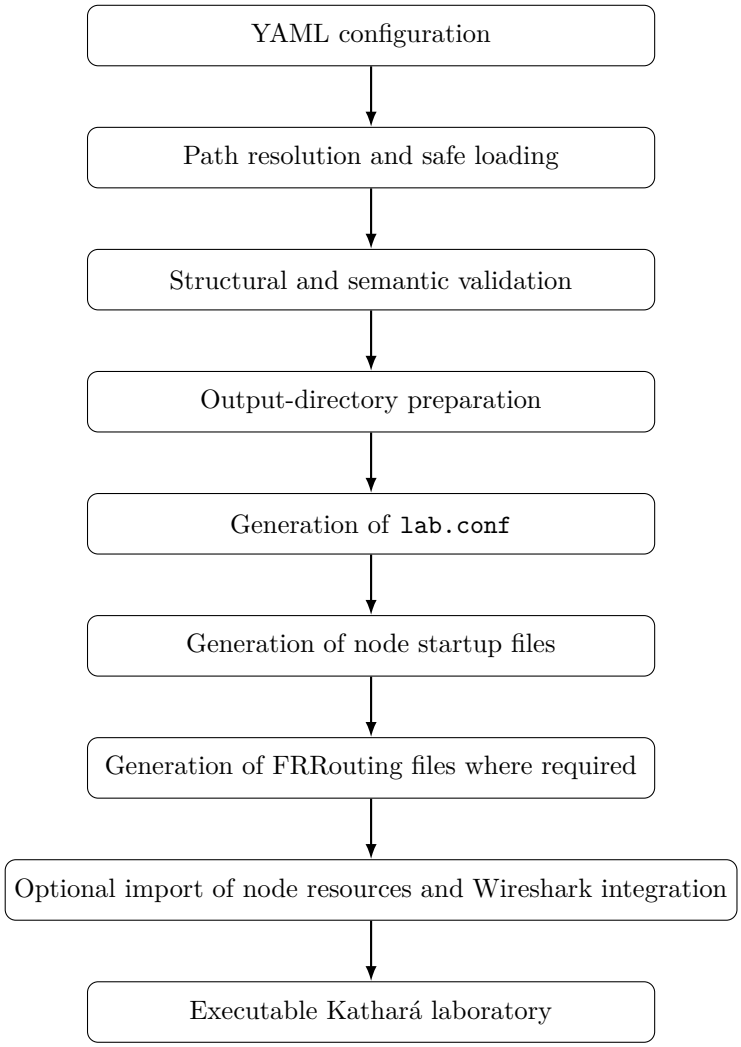


Figure 4.1: Generation pipeline from a YAML source to an executable Kathará laboratory.

The command-line interface exposes the options that control this process. Table 4.1 summarises the options relevant to general laboratory generation. The positional `lab` argument accepts either a configuration name or an explicit YAML path. The default search directory is `configs/`, while generated laboratories are written under `labs/` unless a different output path is selected.

A minimal laboratory can therefore be generated using only its configuration name:

```
python3 generate_lab.py client-router-server
```

Listing 4.2: Generating the minimal client-router-server laboratory.

The same input can be validated without writing any output:

```
python3 generate_lab.py client-router-server --validate-only
```

Listing 4.3: Validation without generation.

For a topology that should be rebuilt from a clean state, the usual invocation is:

Argument or option	Purpose
`lab`	Selects the YAML configuration by name or explicit path.
`--config-dir DIR`	Changes the directory searched when `lab` is not an explicit path. The default is `configs/`.
`--output-dir DIR, -o DIR`	Changes the parent directory in which the generated laboratory is created. The default is `labs/`.
`--validate-only`	Executes loading and validation without creating or modifying laboratory files.
`--clean`	Removes an existing laboratory directory and recreates it from the current configuration.
`--force`	Allows generated files to be written inside an already existing laboratory directory.
`--import-dirs`	Copies directories located next to the YAML file when their names match declared nodes.
`--wireshark MODE`	Controls interactive Wireshark integration. The accepted modes are `ask`, `enabled`, and `disabled`.
`--wireshark-networks LIST`	Explicitly selects one or more collision domains to which the generated Wireshark node is attached.

Table 4.1: General command-line options used during laboratory generation.

```
python3 generate_lab.py multi-area --clean
```

Listing 4.4: Clean regeneration of a laboratory.

The output-directory policy is implemented by `prepare_lab_directory()`. If the target directory does not exist, it is created. If it exists and `--clean` has been requested, the complete directory is removed with `shutil.rmtree()` and then recreated. If it exists and neither `--clean` nor `--force` has been supplied, generation stops with a `FileExistsError`. With `--force`, the existing directory is kept and individual generated files are overwritten as they are produced.

This distinction is important for reproducibility. A clean generation ensures that every file in the resulting directory derives from the current source configuration or from resources explicitly copied during the same run. A forced generation is convenient while developing a topology, because it avoids removing the directory first, but files that are no longer generated may remain from an earlier version. For this reason, `--clean` provides the clearer operation when the objective is to reconstruct a laboratory from its source.

Normal execution validates the configuration twice at different abstraction boundaries. The `main()` function first resolves and validates the input before any output operation is attempted. The lower-level `generate_lab()` function then loads the selected YAML again and repeats structural and semantic validation before writing files. Although this produces some duplicated work, it gives the generation function an independent safety property: it does not assume that every future caller has already performed validation correctly. This was an intentional development trade-off: a clear and defensive module boundary was initially preferred to removing a small amount of repeated work. The stress tests in Chapter 6 later made the cost of this choice measurable and motivated a possible refactoring in which an already validated in-memory representation can be passed to an internal generation function.

Once validation has succeeded, `generate_lab()` derives the laboratory name from `lab_name`, prepares the output directory, writes `lab.conf`, optionally imports node directories, creates every startup file, creates FRRouting files for FRR-capable devices, removes obsolete Wireshark helper scripts, and finally returns the generated path and a summary of optional components. The command-line layer then prints the exact commands that can be used to start and stop the laboratory. A successful generation is therefore not only a file-production step but also the transition from the framework-specific YAML source to a standard Kathará project directory.

4.2 Generated Laboratory Structure and Resources

The most direct way to understand the generator is to examine the artefacts it creates. Consider the `client-router-server.yml` configuration contained in the repository. It defines a client on `lan1`, a router connected to both `lan1` and `dmz`, and a server on the DMZ. The relevant source structure is shown in Listing 4.5.

```
lab_name: client-router-server

nodes:
  client:
    type: host
    interfaces:
      - network: lan1
        ip: 10.0.1.10/24
    default_gateway: 10.0.1.1

  router:
    type: router
    interfaces:
      - network: lan1
        ip: 10.0.1.1/24
      - network: dmz
        ip: 10.0.2.1/24

  server:
    type: host
    interfaces:
      - network: dmz
        ip: 10.0.2.10/24
    default_gateway: 10.0.2.1
```

Listing 4.5: Extract from the client-router-server YAML configuration.

With Wireshark disabled and no node directories imported, generation produces the structure in Listing 4.6. The router directory exists because router nodes are considered FRR-capable and therefore receive an FRRouting directory even when no dynamic protocol has been requested.

```
labs/client-router-server/
|-- lab.conf
|-- client.startup
|-- router.startup
|-- server.startup
`-- router/
    `-- etc/
        `-- frr/
            |-- daemons
            |-- frr.conf
            `-- vtysh.conf
```

Listing 4.6: Generated structure of the minimal routed laboratory.

The first output file is `lab.conf`. Its purpose is to describe the Kathará topology rather than the IP configuration of individual nodes. The function `generate_lab_conf()` begins with metadata and then iterates through the nodes in the validated configuration. Every interface entry is translated into a Kathará attachment of the form `node[index]="collision-domain"`. The index is the position of the interface in the YAML list. Consequently, the first interface becomes `eth0`, the second becomes `eth1`, and the same ordering is subsequently used when the startup file assigns IP addresses.

For the configuration in Listing 4.5, the generated node definitions are:

```
client[0]="lan1"
client[image]="kathara/base"

router[0]="lan1"
router[1]="dmz"
router[image]="kathara/frr"

server[0]="dmz"
server[image]="kathara/base"
```

Listing 4.7: Topology entries generated in lab.conf.

The absence of IP addresses from `lab.conf` is intentional. The file answers the question of which node interfaces are attached to which collision domains. Layer-3 configuration is kept in the corresponding startup files. This separation follows the normal Kathará laboratory model and is preserved by the generator rather than hidden behind a new proprietary representation.

The generator also writes the laboratory metadata at the beginning of `lab.conf`. Values can be provided through the top-level `metadata` mapping. Within that mapping, the `description` field takes precedence over the simpler top-level description; default values are used when individual metadata fields are absent. The output contains fields for the laboratory description, version, author, email address, and web reference. These fields do not influence forwarding, but they make the generated laboratory easier to identify and trace independently from its YAML source.

Image selection is performed by `get_node_image()`. An explicit `image` field always has priority. Without an override, an ordinary host uses `kathara/base`. Nodes whose type is `router`, `switch`, or `firewall` use `kathara/frr`. A host can also become FRR-capable when it contains an active `frr` section. This policy provides useful defaults while still allowing the YAML configuration to select a different image when the node requires additional software.

After `lab.conf`, the generator creates one `.startup` file for every node. The startup-generation function processes the same ordered interface list used by `lab.conf`. For every interface that contains an IP address, it generates an `ip address add` command and associates the address with `ethN`. For example, the client in the minimal topology produces:

```
ip address add 10.0.1.10/24 dev eth0

ip route add default via 10.0.1.1
```

Listing 4.8: Generated startup file for the client node.

The router produces:

```
ip address add 10.0.1.1/24 dev eth0
ip address add 10.0.2.1/24 dev eth1

systemctl start frr
```

Listing 4.9: Generated startup file for the router node.

The generator does not emit explicit `ip link set ethN up` commands. Kathará creates the interfaces as part of the node environment, while the startup file assigns addresses and routing state. This detail is relevant when describing the implementation because the generated file should be documented as it actually exists rather than as a generic Linux-network configuration procedure.

Default gateways and static routes are translated directly into Linux route commands. A YAML route such as

```
routes:
```

```
  - to: 192.168.20.0/24
    via: 10.0.12.2
```

Listing 4.10: Static route represented in YAML.

becomes:

```
ip route add 192.168.20.0/24 via 10.0.12.2
```

Listing 4.11: Command generated from a static route.

The `commands` list is handled differently. Its entries are not translated into a framework-defined representation; they are appended to the startup file as literal strings after the automatically generated network and FRR commands. The model therefore has a clear boundary. Common operations such as address assignment, gateways, routes, and routing-daemon startup are produced automatically, while node-specific behaviour can still be expressed through arbitrary startup commands when needed.

Table 4.2 summarises the relationship between the main configuration operations introduced in Chapter 3 and their generated output.

YAML information	**Generated artefact**	**Effect**
Node name	`lab.conf`, startup filename, optional node directory	Establishes the node identifier used throughout the laboratory.
`interfaces[].network`	`lab.conf`	Connects `ethN` to a Kathará collision domain.
`interfaces[].ip`	`<node>.startup`	Adds the configured address to the corresponding interface.
`default_gateway`	`<node>.startup`	Adds the node's default route.
`routes`	`<node>.startup`	Adds explicit static routes.
`commands`	`<node>.startup`	Appends user-provided shell commands literally.
`image` or inferred node role	`lab.conf`	Selects the container image.
`frr`	`<node>/etc/frr/*` and startup file	Creates FRRouting files and starts the FRR service.

Table 4.2: Relationship between source configuration fields and generated Kathará artefacts.

FRRouting generation is integrated into the same output process. The helper `is_frr_device()` determines whether a node receives FRR files. Device types `router`, `switch`, and `firewall` are always considered FRR-capable. For a host, the presence of an enabled `frr` section is sufficient. This behaviour means that a router without an explicitly selected dynamic protocol still receives a minimal FRR configuration and its startup file still runs `systemctl start frr`.

For each FRR-capable node, `write_frr_files()` creates `etc/frr/` under the node directory and writes three files: `daemons`, `frr.conf`, and `vtysh.conf`, following the configuration model provided by FRRouting [6]. The daemon file always enables Zebra and activates the protocol daemons inferred from the YAML. For a single OSPF router, the relevant part is similar to:

```
zebra=yes
bgpd=no
ospfd=yes
ripd=no
ospf6d=no
ripngd=no
...
vtysh_enable=yes
```

Listing 4.12: Relevant entries in an automatically generated FRR daemons file.

The `frr.conf` file is generated from protocol-specific fields. The `multi-area.yml` configuration, for example, declares router `r1` with three addressed interfaces and associates its LAN with area 10 and its two transit links with area 0. The generated configuration is:

```
frr defaults traditional
hostname r1
service integrated-vtysh-config
!
router ospf
 ospf router-id 1.1.1.1
 network 10.10.1.0/24 area 10
 network 10.0.12.0/30 area 0
 network 10.0.13.0/30 area 0
!
line vty
!
```

Listing 4.13: FRRouting configuration generated for r1 in the multi-area laboratory.

This example illustrates an important property of the generator: the user assigns an address and an OSPF area to the interface, while the framework derives the corresponding network prefix before writing the FRR `network` command; the protocol semantics follow OSPFv2 as specified in RFC 2328 [19]. The same source information therefore drives both Linux interface configuration and routing-protocol configuration, reducing manual duplication between files.

RIP follows the same principle. The generator writes a `router rip` section, selects the requested version, disables automatic summarisation by default, and lists the networks associated with RIP. For BGP, the generator writes the autonomous-system number, optional router ID, neighbour statements, advertised prefixes, and redistribution rules where configured [25]. A typical fragment produced from `multi_AS.yml` is structurally equivalent to:

```
router bgp 65001
 bgp router-id 1.1.1.1
 no bgp ebgp-requires-policy
 no bgp network import-check
 neighbor 10.0.12.2 remote-as 65002
 neighbor 10.0.13.2 remote-as 65003
 network 10.10.1.0/24
```

Listing 4.14: Example of automatically generated BGP configuration.

The two `no bgp` commands are generated when the corresponding YAML options retain their default false value. In particular, `ebgp-requires-policy` reflects the external-BGP policy behaviour standardised by RFC 8212 [18]. They make the behaviour appropriate for the small controlled laboratories used by the framework: eBGP announcements are not blocked by the default policy requirement, and a configured `network` statement is not suppressed only because the exact route is absent from the local routing information base at configuration time. These settings are generation choices for the project environment rather than a replacement for policy configuration in operational networks.

The generator also supports protocol redistribution and multiple routing protocols on the same node. Protocol discovery accepts the compact `frr.protocol` field, a `protocols` collection, nested `ospf`, `rip`, and `bgp` mappings, or direct daemon markers. This backwards-compatible parsing allows the more structured example configurations to coexist with simpler earlier files.

A complete `frr.config` block acts as an escape mechanism: when present, its content is written directly to `frr.conf` instead of constructing the protocol configuration automatically. It is important to distinguish this from daemon activation. The contents

of `frr.conf` may be supplied manually, but `daemons` is still generated from the protocol or daemon information that the framework can identify. A manual configuration that uses a protocol daemon should therefore also declare enough information for that daemon to be enabled. This is one of the points where flexibility transfers more responsibility to the configuration author.

The third FRR file, `vtysh.conf`, contains `service integrated-vtysh-config`. Together with `vtysh_enable=yes` in the daemon file, it allows the generated router to be inspected through the standard `vtysh` interface after the laboratory starts.

Beyond generated network files, the framework can import arbitrary resources associated with a node. The option `--import-dirs` activates `copy_node_directories()`. The source directory is the directory that contains the selected YAML file, not necessarily the repository's global `configs/` directory. For each declared node, the function checks for a sibling directory with exactly the same name. If it exists, that directory is copied recursively into the generated laboratory.

The mapping is therefore:

```
<directory containing YAML>/<node-name>/
                      |
                      |
<output-dir>/<lab-name>/<node-name>/
```

Listing 4.15: Mapping used when node-specific resources are imported.

For a configuration stored in `configs/example.yml`, a directory `configs/server/` is copied to `labs/example/server/`. Kathará exposes the files stored in the node directory through the node's `/hostlab/` path. A startup command can therefore execute an imported script without embedding that script into the YAML configuration itself. This separation is useful because the YAML remains focused on topology and startup declarations, while application code, data files, or node-specific configuration can remain ordinary files.

The import process deliberately ignores `__pycache__`, compiled `.pyc` files, `.git`, and `.DS_Store`. The semantic validation described in the previous chapter also reports visible sibling directories that do not correspond to nodes when directory import is enabled. The generator itself simply skips node names for which no source directory exists. Resource import is therefore optional and non-invasive: a topology can be generated with only the files derived from YAML, or enriched with external per-node resources when required.

Wireshark integration is another optional extension of the generated `lab.conf`; Wireshark provides the packet-capture and protocol-analysis capabilities used by this observation node [27]. The feature was introduced as runtime debugging became more important during development. Inspecting a generated startup file shows what the framework intends to configure, but it does not show whether packets actually traverse the expected collision domain or where communication stops. Packet capture was therefore kept as an optional diagnostic layer rather than made part of every functional topology. The implementation first reconstructs the set of collision domains by scanning all node interfaces. Each domain is associated with the nodes, interface indices, and optional addresses attached to it. This makes it possible to present a meaningful selection such as `lan_a | r1:eth0(...), pc_a:eth0(...)` rather than only a list of internal names.

A capture point can be selected explicitly from the command line:

```
python3 generate_lab.py multi-area --clean \
    --wireshark-networks lan_a,r1_r2
```

Listing 4.16: Generating a laboratory with Wireshark attached to two collision domains.

The parser also accepts `all`, numeric indexes in interactive mode, or the form `node:r1`, which expands to every collision domain attached to the selected node. A top-level `wireshark` mapping in the YAML can also provide an enabled flag and one or

more networks. Explicit command-line network selection takes precedence over interactive selection.

Wireshark is represented as a normal Kathará node added directly to `lab.conf`. For a capture on `lan1`, the generated block is:

```
# Wireshark real-time packet capture
wireshark[0]="lan1"
wireshark[bridged]=true
wireshark[port]="3000:3000/tcp"
wireshark[image]="lscr.io/linuxserver/wireshark"
wireshark[num_terms]=0
```

Listing 4.17: Wireshark node generated in lab.conf.

Connecting Wireshark to collision domains in `lab.conf` is preferable to attaching an external capture process after startup because the observation point becomes part of the generated topology itself. One Wireshark node may be attached to several selected domains. When generated in this form, the browser interface is published on TCP port 3000. The generator also removes the legacy `start_wireshark.sh`, `sniff.sh`, and `stop_wireshark.sh` files if they still exist in the laboratory directory, because they are unnecessary once the capture node is defined directly in `lab.conf`.

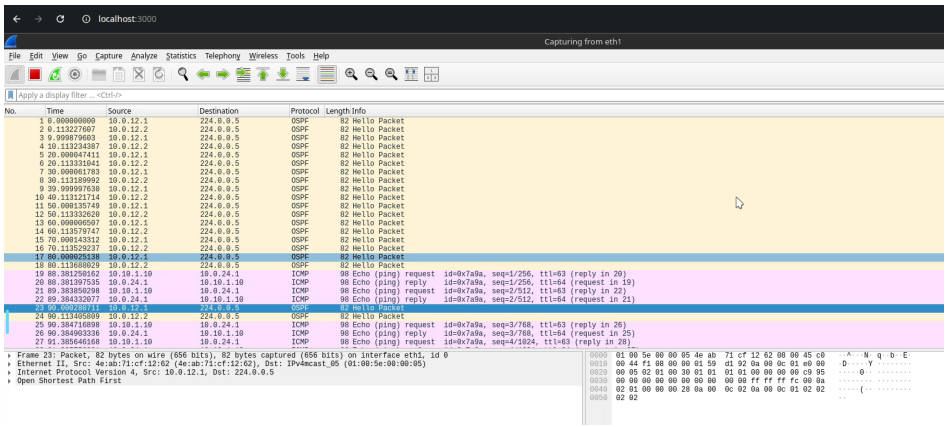


Figure 4.2: Runtime Wireshark capture on the `r1_r2` collision domain of the generated multi-area laboratory.

Figure 4.2 provides a runtime check of the monitoring integration. The generated Wireshark node is attached to the selected collision domain and observes packets exchanged by the emulated nodes without changing their functional roles. This complements inspection of `lab.conf`: the generated file proves that the observation node was configured, while the capture demonstrates that traffic on the selected Kathará segment is actually visible to it during execution.

The output of the generation stage is therefore a conventional Kathará laboratory enriched only by files that Kathará already knows how to consume. The framework's YAML format is used before generation; it is not required by Kathará at runtime. This design is significant because a generated laboratory can be opened, inspected, and started with standard Kathará tools even if the generator is not invoked again.

4.3 Laboratory Execution and Lifecycle Management

Once generation is complete, the operational workflow returns to ordinary Kathará commands. The repository provides shell wrappers for the most common lifecycle operations. Their purpose is intentionally limited: they perform basic argument and directory checks, change into the selected laboratory, and invoke the corresponding

Kathará command. Generation and runtime management were deliberately not merged into one long-running Python process, because the produced laboratory should remain usable even after the generator has exited. The wrappers reduce repetitive shell navigation while keeping the underlying Kathará operation visible to the user.

Figure 4.3 presents the intended lifecycle. The YAML file remains the source configuration. It is validated and generated into a laboratory directory, which can then be started and inspected. When a persistent change is required, the change is applied to the YAML source or to a node resource directory and the laboratory is regenerated rather than relying on a manual edit to derived output.

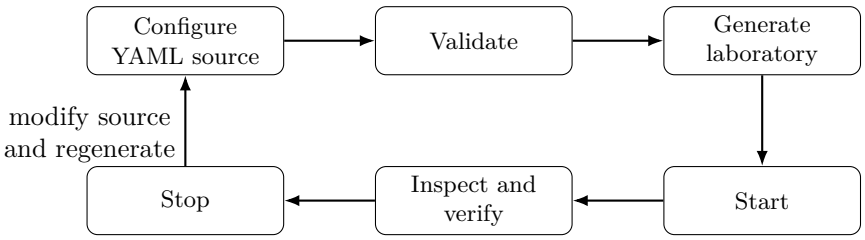


Figure 4.3: Operational lifecycle of a generated laboratory.

A laboratory can be started with the wrapper by passing its generated path:

```
./start.sh labs/client-router-server
```

Listing 4.18: Starting a generated laboratory.

The script requires at least one argument, derives the display name with `basename`, verifies that the supplied directory exists, changes into that directory, and executes:

```
kathara lstart
```

Listing 4.19: Kathará command executed by start.sh.

Starting from the laboratory directory matters because Kathará reads the local `lab.conf` and the associated node files. During startup, the collision-domain definitions in `lab.conf` determine the emulated links, container images determine the node environments, and the `.startup` files apply addresses, routes, FRRouting startup, and user commands. FRR-capable nodes also find their configuration under their local `/etc/frr/`-equivalent hostlab structure according to the standard Kathará file-mapping mechanism.

The wrapper is not required. The generator explicitly prints the equivalent manual procedure after a successful run:

```
cd labs/client-router-server
kathara lstart
```

Listing 4.20: Starting a laboratory without the wrapper.

Keeping both forms available is useful. The wrapper makes the common operation shorter, while the explicit commands make it clear that the generated output is not tied to the framework's own runtime.

After startup, individual nodes can be accessed through normal Kathará commands. For example:

```
kathara connect router
```

Listing 4.21: Connecting to a generated router.

Once inside a node, standard Linux tools provide a first verification of the generated configuration. `ip addr` shows whether addresses have been assigned to the expected interface indices, `ip route` shows connected, default, and static routes, and `ping` can

be used for a basic reachability check. In the minimal client-router-server topology, the client can verify the path to the server address with:

```
ping 10.0.2.10
```

Listing 4.22: Basic reachability check after laboratory startup.

For FRRouting nodes, `vtysh` provides the corresponding routing view. Commands such as `show ip route`, `show ip ospf neighbor`, and `show ip bgp summary` are useful for verifying that the generated FRR files have been loaded and that protocol state is developing as expected. These manual checks are not part of generation itself. They are operational observations performed after the generated files have been consumed by the runtime environment.

This distinction is important because successful generation does not imply immediate runtime convergence. The generator can prove that configured OSPF networks refer to connected prefixes and that BGP neighbour declarations are coherent according to the static model, but it cannot guarantee that a routing protocol has converged at the exact moment the containers start. Runtime state depends on process startup, neighbour availability, timers, imported resources, and any user-supplied commands. Generation correctness and runtime behaviour are therefore related but separate concerns.

The repository provides `stop.sh` for the inverse lifecycle operation:

```
./stop.sh labs/client-router-server
```

Listing 4.23: Stopping and cleaning the runtime environment.

Like `start.sh`, the script verifies that the supplied directory exists and changes into it. It then invokes the standard Kathará cleanup command [11]:

```
kathara lclean
```

Listing 4.24: Kathará command executed by stop.sh.

The word *clean* in this Kathará command should not be confused with the generator's `--clean` option. `kathara lclean` removes the runtime laboratory created by Kathará, including the containers and associated runtime networking, while the generated directory on disk remains available. The generator's `--clean`, by contrast, operates on the file-system output before generation: it deletes the selected laboratory directory and recreates its files from the YAML source.

The difference can be summarised as follows:

Operation	**Generated files**	**Runtime containers and networks**
Generation	Created or overwritten	Not started by the generator
`start.sh` / `kathara lstart`	Preserved	Created and started
`stop.sh` / `kathara lclean`	Preserved	Removed
Generation with `--clean`	Directory removed and regenerated	Independent of runtime state
`del_lab.sh`	Selected generated directory removed	Does not replace runtime cleanup

Table 4.3: Effects of the main lifecycle operations.

When the generated files are no longer required, `del_lab.sh` removes them from disk. It accepts either a path to a laboratory directory or the literal argument `all`. For an individual laboratory:

```
./del_lab.sh labs/client-router-server
```

Listing 4.25: Deleting one generated laboratory.

and for all generated content under `labs/`:

```
./del_lab.sh all
```

Listing 4.26: Deleting all generated laboratories.

The current implementation performs deletion with `sudo rm -rf`. When `all` is supplied, only the contents of `./labs/` are removed and the parent directory itself is preserved. For a specific argument, the script first checks that the path exists as a directory and then removes that path. Because this is a destructive file-system operation, the selected path should be verified before invoking the command. In normal use, runtime cleanup with `stop.sh` should be treated separately from removal of generated files.

The explicit path accepted by the lifecycle scripts also allows several laboratories to coexist under the same output directory. A repository may, for example, contain:

```
labs/
|-- client-router-server/
|-- Test1_multi_area/
|-- multi-AS/
`-- Test_enterprise_multi_as_import_dirs/
```

Listing 4.27: Several generated laboratories coexisting under labs/.

No lifecycle script needs to guess which one is active or intended. This explicit selection also reflects a development issue that appeared as the number of example configurations increased: assumptions that were harmless when only one generated laboratory was present became ambiguous once several outputs coexisted under `labs/`. The user therefore selects the laboratory explicitly:

```
./start.sh labs/multi-AS
```

Listing 4.28: Starting one laboratory among several generated environments.

This explicit selection is consistent with the framework's general design: the tool automates repetitive file creation and common commands, but avoids hidden choices when more than one valid target exists.

The lifecycle also clarifies which artefact should be considered authoritative. The YAML configuration is the source description, while the directory under `labs/` is derived output. A persistent topology change should therefore be applied to the YAML source. A persistent node-resource change should be applied to the resource directory located next to that YAML file. After the change, the laboratory can be regenerated, preferably with `--clean` when a fully reconstructed output is required.

Directly editing `lab.conf`, a generated startup file, or a generated FRR file is possible because they are ordinary text files, but such edits are not part of the reproducible source model. A later clean generation replaces them. Treating generated files as disposable output reduces configuration drift between the declarative source and the laboratory that is actually executed. The relationship can be expressed as:

```
configs/<topology>.yml       -> authoritative topology source
configs/<node>/              -> authoritative optional node
    resources

labs/<lab-name>/             -> generated laboratory output
```

Listing 4.29: Source and derived-output relationship.

There are several practical boundaries to this workflow. Docker and Kathará must already be installed and correctly configured for the current user; the generator does not provision those dependencies. The selected container images must also provide the commands expected by the generated startup files. Forced generation can leave obsolete files in an existing directory, whereas clean generation intentionally removes manual changes made to the output. Imported resources depend on the naming convention that

associates a source directory with a YAML node of the same name. Advanced Linux behaviour outside the configuration model must be supplied through startup commands, and advanced FRRouting behaviour can be supplied through manual FRR content, which reduces the amount of semantic checking the framework can perform.

A further implementation detail concerns FRR-capable node roles. Because `router`, `switch`, and `firewall` are recognised as FRR devices by type, those nodes receive FRR startup and configuration files even when the configuration does not select OSPF, RIP, or BGP. In that case the framework writes a minimal `frr.conf` containing the hostname, integrated VTY configuration, and a VTY section, while protocol daemons remain disabled. This keeps the behaviour predictable for routing-oriented images, but it also means that node type influences generated runtime configuration in addition to documenting the conceptual role of the node.

These boundaries do not change the central lifecycle model. The framework takes responsibility for deterministic transformations it can express explicitly, then hands a normal Kathará laboratory to the standard runtime. This makes the generated output inspectable and reduces the amount of project-specific logic required once generation has finished.

4.4 Chapter Summary

This chapter described how the validated configuration model is transformed into an executable Kathará laboratory. The command-line wrapper delegates the process to `src_gen_lab.gen_lab`, which validates the input, prepares the output directory, creates `lab.conf`, writes node startup files, generates FRRouting configuration for FRR-capable devices, and optionally imports node-specific resources or adds a Wireshark observation node.

The generated artefacts preserve the standard Kathará structure. Collision domains and image assignments are written to `lab.conf`; addresses, default gateways, static routes, FRR startup, and custom commands are written to per-node startup files; and routing devices receive the appropriate `daemons`, `frr.conf`, and `vtysh.conf` files. The same validated YAML information can therefore influence several output files without requiring the user to duplicate the configuration manually.

After generation, the framework does not replace the Kathará runtime. `start.sh` and `stop.sh` are thin wrappers around `kathara lstart` and `kathara lclean`, while `del_lab.sh` manages the generated directories on disk. Nodes remain accessible through standard Kathará commands and can be inspected with Linux networking tools and FRRouting's `vtysh`. Multiple generated laboratories can coexist because lifecycle operations receive an explicit laboratory path.

Most importantly, the YAML configuration remains the authoritative description of the topology and the generated laboratory remains derived output. This separation supports reproducibility: changes are made at the source level and propagated through validation and regeneration, while the generated files stay compatible with the ordinary Kathará workflow.

Chapter 5

Experimental Evaluation and Controlled DoS Experiment

The previous chapters described the framework as a software system: the input model, the validation rules, the generation of Kathará artefacts, and the lifecycle used to start and inspect a generated laboratory. This chapter moves from implementation to experimental use. Its purpose is to verify that the framework can support a repeatable network experiment in which several legitimate hosts access the same service concurrently and their observations are collected before and during a controlled adverse workload.

The experiment is intentionally bounded and is executed entirely inside the local Kathará environment. No traffic is directed towards external systems. The adverse workload is therefore used as an experimental condition rather than as a general-purpose attack mechanism. The objective is to observe how a finite-capacity TCP application behaves when many connections compete for its available service slots, and to determine whether client-side and server-side measurements can describe that transition quantitatively.

Real-world DDoS context. The controlled experiment should be interpreted against the broader evolution of Distributed Denial-of-Service attacks. Real incidents can exhaust different resources and can rely on compromised botnets, reflection and amplification, protocol-specific techniques, or application-layer request floods [7]. The laboratory developed in this thesis does not attempt to reproduce the scale, source diversity, or attack infrastructure of such incidents. Instead, selected cases are useful because they identify several distinct mechanisms that a small emulated experiment can abstract only in part.

DDoS is also used outside purely criminal or commercial contexts. Europol's 2026 Internet Organised Crime Threat Assessment describes the pro-Russian NoName057(16) network as a prominent example of ideologically motivated DDoS activity. In July 2025, Operation Eastwood disrupted more than 100 servers associated with the infrastructure; Europol linked the network to an average of 73 DDoS attacks per day against Ukrainian public and private actors and reported that activity resumed after an initial decline [5]. This example is relevant because it shows that the availability objective of a DDoS attack can remain similar even when the operational motivation and infrastructure are very different.

These cases define an important boundary for the evaluation. The experiment does not claim to reproduce an Internet-scale botnet, reflection/amplification, HTTP/2 protocol exploitation, or multi-terabit flooding. Because several controlled agents contribute simultaneously, the workload has a distributed-source structure, but the term *controlled DoS* is retained to emphasise the deliberately bounded laboratory scope. What the experiment does reproduce is a narrower and measurable resource-contention mechanism: multiple sources compete with legitimate clients for finite service capacity. Consequently, the most relevant variables are not absolute traffic volume but application completion, lower-layer reachability, response time, and the server-side capacity state. Sections 5.2

Incident	Main characteristic	Relation to this thesis
Mirai / Dyn, 2016	A Mirai-variant botnet composed of compromised Internet-of-Things devices was used in DDoS activity that impacted Dyn's DNS infrastructure and disrupted access to multiple major Internet services [28].	This is the closest structural analogy to the controller and multiple load agents used in the Kathará testbed: several distributed sources converge on a common service. The laboratory agents, however, are deliberately controlled, bounded, and non-compromised.
GitHub, 2018	A memcached UDP reflection/amplification attack reached a peak of 1.35 Tbps and 126.9 million packets per second [13].	This case illustrates a mechanism that is intentionally not reproduced by the framework. The attack relied on reflectors, source-address spoofing, and amplification rather than on direct TCP connection generators.
Google HTTP/2 Rapid Reset, 2023	A Layer 7 campaign using the HTTP/2 Rapid Reset technique reached a peak of 398 million requests per second. The underlying issue was tracked as CVE-2023-44487 [12].	The event shows that DDoS intensity cannot be described only in bit/s. Request rate and server or proxy processing capacity may also become the limiting resource, which is conceptually closer to the finite application capacity studied in this chapter.
Cloudflare, 2025	Cloudflare reported blocking a 7.3 Tbps multi-vector DDoS attack that delivered 37.4 TB in 45 seconds; approximately 99.996% of the traffic was classified as UDP floods [31].	This case illustrates the modern hyper-volumetric and short-duration extreme. The present experiment deliberately studies a different question: the relationship between offered workload and available service capacity, rather than Internet-scale throughput.

Table 5.1: Selected real-world DDoS incidents used to contextualise the controlled experiment.

and 5.3 measure these variables directly.

The archived project contains five complete repetitions for each topology, collected on 13 October 2026 with the same eight clients, ten concurrent measurement rounds, and controlled workload schedule. The direct-server runs are stored under `results/dos_lab/simulation_0001/` through `simulation_0005/`; the five complete reverse-proxy runs are stored under `results/mitigation/simulation_0002/` through `simulation_0006/`. Each run contains a baseline and a controlled-load phase, and every phase file contains 80 client observations. The resulting dataset therefore comprises 1,600 client observations: 800 per topology and, within each topology, 400 for the baseline and 400 for the controlled-load phase. The final dataset was defined as executions containing both client-phase CSV files with 80 observations per phase together with the server-side metric files; no observation from a run satisfying this completeness criterion was excluded. The directory `results/mitigation/simulation_0001/` is a preliminary pilot collected before the routing-readiness criterion was added. It contains server metrics but no client-result CSV and was therefore never a member of the final dataset. The first complete run of each topology is retained below as a detailed representative execution; Section 5.5 then treats the ten complete runs as the descriptive dataset. All numerical results were recalculated directly from the archived CSV files rather than transcribed from console output.

5.1 Experimental Methodology and Testbed

The experimental testbed is generated from `configs/dos_lab.yml`. It is a multi-LAN enterprise-style topology composed of two legitimate-user networks, a routed core, a separate network containing the controlled load agents, and a DMZ containing the application server. All routers use OSPF in area 0, so the experiment also depends on the routing configuration produced by the framework rather than on a single flat collision domain.

The two legitimate LANs contain four hosts each. The first LAN uses the prefix `10.10.1.0/24` and contains `pc_a1` through `pc_a4`; the second uses `10.10.2.0/24` and contains `pc_b1` through `pc_b4`. The target service is hosted by `server` at `10.30.1.10/24` in the DMZ. A separate workload network uses `10.20.1.0/24` and contains the controller `attacker` together with three bounded load agents, `zombie_1`, `zombie_2`, and `zombie_3`. The corresponding collision-domain names are reported in the configuration and the functional roles are summarised in Table 5.2.

Component	**Nodes / address range**	**Role**
Legitimate LAN A	`pc_a1`–`pc_a4`, `10.10.1.0/24`	Concurrent legitimate clients connected through `r1`.
Legitimate LAN B	`pc_b1`–`pc_b4`, `10.10.2.0/24`	Concurrent legitimate clients connected through `r2`.
Routing core	`r1`, `r2`, `r3`, `r4`, `core`	OSPF infrastructure connecting the client LANs, the DMZ, and the isolated workload LAN.
Service DMZ	`server`, `10.30.1.10/24`	Instrumented TCP application exposed on port `9000`.
Controlled-load LAN	`attacker`, `zombie_1`–`zombie_3`, `10.20.1.0/24`	Controller and bounded connection generators used only inside the laboratory.

Table 5.2: Main components of the experimental topology.

Figure 5.1 shows the logical structure of the testbed. The central point is that legitimate traffic and controlled workload traffic reach the same server through the routed infrastructure. This makes the experiment a test not only of the application but also of the complete laboratory generated by the framework: addressing, gateways, inter-router

links, OSPF routes, node resources, and service startup commands all have to operate consistently.

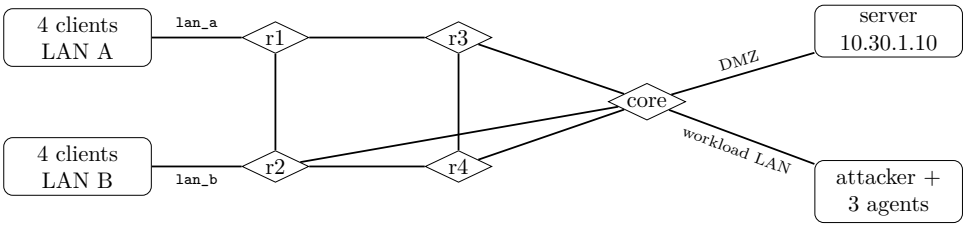


Figure 5.1: Logical structure of the controlled experimental testbed.

The experiment is orchestrated by `simulate.sh`. By default the script selects `dos_lab`, targets the node named `server` on TCP port `9000`, discovers all nodes whose names begin with `pc_` as legitimate clients, and creates a new numbered output directory under `results/dos_lab/`. Earlier connectivity utilities were more dependent on literal addresses; accepting a logical target name where possible reduced the need to repeat addressing information already represented by the laboratory configuration and made the same test workflow easier to reuse after address changes. A full run first performs the baseline measurement and then performs the controlled-load measurement using the same laboratory and the same measurement parameters.

The default parameters used by all archived complete runs are ten simultaneous rounds, a three-second measurement timeout, one second between rounds, and a 25-second startup wait when the orchestrator starts the lab. The script also checks reachability of the target from every legitimate client before starting the measurements. This routing-readiness stage repeatedly sends an ICMP probe from each client and allows up to 120 seconds for the routes to become usable. The check was introduced after earlier executions showed that a measurement could start while dynamic routing was not yet usable from every client. A fixed startup delay alone was therefore not considered a sufficient readiness criterion. By checking the observed network state, an OSPF convergence problem can be detected before it contaminates the service baseline. This is particularly relevant in light of the preliminary `mitigation/simulation_0001` pilot described above: because it predates this criterion and contains no client-result dataset, it is retained only as a development artefact and is not analysed as an experimental run.

Table 5.3 records the software and host environment used for the archived experiments. These values were captured on the experimental host when the results were archived on 13 October 2026. The Git working tree contained uncommitted changes, so the base commit alone is not a byte-for-byte identifier of the executed source; the archived project files, configuration files, raw CSV data, and container-image digests are therefore the authoritative experimental artefacts.

The results of each run are isolated by a run number. Client measurements are separated by scenario in `connection_results_baseline.csv` and `connection_results_dos.csv`; service-side counters are stored in `server_metrics.csv`; and host-level Docker samples are stored in `server_container_metrics.csv`. The same run identifier is also used by the logging hierarchy, which makes it possible to associate a client failure with the corresponding server state and orchestration run. The mitigation runs retain the same schema: the `target_input` field changes from `server` to `proxy`, while the public address and port remain `10.30.1.10:9000`.

5.2 Measurement Infrastructure and Baseline Evaluation

The target application is implemented by `configs/server/service.py`. It is a multithreaded TCP service that listens on port `9000`. A bounded semaphore, implemented

Component	Recorded value
Host processor	11th Gen Intel Core i7-11800H at 2.30 GHz; 16 logical CPUs
Host memory	15 GiB RAM; 2 GiB swap
Operating system	Linux Mint 22.2; Linux kernel `7.0.0-34-generic`
Python / PyYAML	Python 3.12.3; PyYAML 6.0.1
Docker	Client and server 29.8.1
Kathará	Version 3.8.3
`kathara/base:latest`	`sha256:8a2e70ac8f51bb283b8549fd686f65f9 a1ef4487fe2c81849cb74d301e61970b`
`kathara/frr:latest`	`sha256:3b7a3f630d8ecd9efddd4586ba2f0a41 e2c462f13bc81dee8e3c6ebac183f57c`
Framework source	Git base commit `3a1d65189981`; working tree dirty

Table 5.3: Host, software, and container-image metadata recorded for the experimental archive.

with `threading.BoundedSemaphore`, limits the number of active application sessions. In the experimental YAML and in the orchestration fallback command, `SERVICE_MAX_ACTIVE` is set to 20 and `SERVICE_BACKLOG` is set to 20. A connection that cannot acquire an application slot is answered with `BUSY` when possible and then closed. A connection that obtains a slot receives a small text menu; selection `1` returns the token `pong`.

This behaviour allows the experiment to distinguish network reachability, TCP connectivity, and application availability. This distinction grew out of a concrete development problem: a client could reach the server node while the application process was not running or was unable to complete the expected transaction. Treating reachability as a successful service test therefore produced misleading results. A successful ICMP reply proves only that the routed path is currently reachable, while a successful TCP handshake proves that the kernel accepted a connection to the server port. Neither event alone proves that the application completed the requested operation. For this reason, `run_connection_tests.py` treats a request as successful only when the client sends `1` and subsequently observes `pong` in the server response.

Each measurement is executed inside the client container rather than on the Docker host. The script records five timing fields, listed in Table 5.4. This separation is important because the host-side time required to invoke `docker exec` is not a network delay and should not be interpreted as one.

Metric	Meaning
`icmp_rtt_ms`	Round-trip time of a single ICMP echo request executed in the client network namespace.
`tcp_connect_ms`	Time from the beginning of `connect()` to successful establishment of the TCP connection.
`application_response_ms`	Time between sending selection `1` and receiving a response containing `pong`.
`request_completion_ms`	Complete TCP application transaction from the start of connection establishment to reception of `pong`.
`host_command_completion_ms`	Host-observed duration of the complete measurement command, including Docker invocation overhead.

Table 5.4: Client-side metrics recorded by `run_connection_tests.py`.

The execution model was also changed during development. A sequential strategy is sufficient when the question is simply whether each client can reach a service, but it becomes problematic when clients are intended to observe a shared and time-varying server condition. Executing all attempts from one host before moving to the next would compare measurements collected in different windows and could make an overload transition appear different for each client. For every request index, the final implementation therefore

schedules the eight legitimate clients concurrently through a `ThreadPoolExecutor`. The execution model is round based rather than client based: in round 1 all eight clients perform one measurement; after the configured delay, round 2 repeats the same operation, and so on. With eight clients and ten attempts, each scenario produces 80 application requests. Concurrency is therefore an experimental requirement for comparability, not merely a performance optimisation.

Application availability is defined as

$$A = \frac{N_{\text{pong}}}{N_{\text{requests}}} \times 100, \tag{5.1}$$

where N_{pong} is the number of requests for which the expected application response was received. Failed requests remain part of the denominator. Latency summaries, on the other hand, can only be calculated from requests for which the corresponding timing value exists. This distinction is essential under overload: a low mean response time among the small subset of successful requests does not imply a healthy service when most requests fail.

As a representative execution, the baseline of `dos_lab/simulation_0001` contains 80 requests and all 80 completed successfully, corresponding to 100% application availability. All ICMP probes and all TCP connection attempts were also successful. The mean ICMP RTT was 1.686 ms and the mean TCP connection time was 1.803 ms. Among the successful application requests, the mean response time was 3.218 ms, the median was 3.202 ms, the 95th percentile was 4.922 ms, and the maximum was 5.309 ms. Table 5.5 reports the main values.

Baseline metric	**Value**
Legitimate application requests	80
Successful application requests	80
Application availability	100.00%
ICMP successes	80 / 80
TCP connection successes	80 / 80
Mean ICMP RTT	1.686 ms
Mean TCP connection time	1.803 ms
Mean application response time	3.218 ms
Median application response time	3.202 ms
95th percentile application response time	4.922 ms
Maximum application response time	5.309 ms
Approximate measurement duration	10.77 s

Table 5.5: Direct-server baseline results from `dos_lab/simulation_0001`.

Figure 5.2 reports the corresponding per-client measurements. The availability panel remains at 100% in every round, while the other panels show the short-timescale variation in ICMP, TCP, and application response times summarised in Table 5.5.

The per-client baseline is also uniform. Every client completed ten out of ten requests successfully. No client exhibits an availability disadvantage. This is relevant because the clients are attached to two different access LANs and traverse different first-hop routers; the baseline therefore establishes that the generated routing topology does not itself introduce a visible service failure before the adverse workload is introduced.

The server-side metrics reinforce the same conclusion. During the baseline window, the instrumented service recorded 80 accepted connection events and no `BUSY` or rejected events. The maximum number of simultaneously active connections observed by the server was one and the maximum active thread count was four. The container sampler collected six Docker-stat samples for the baseline window; these samples reported a mean CPU utilisation of approximately 0.53% and a maximum of 0.88%. Since the baseline measurement lasts only about ten seconds and the container dataset contains a small number of samples, these resource figures are used descriptively rather than as statistically

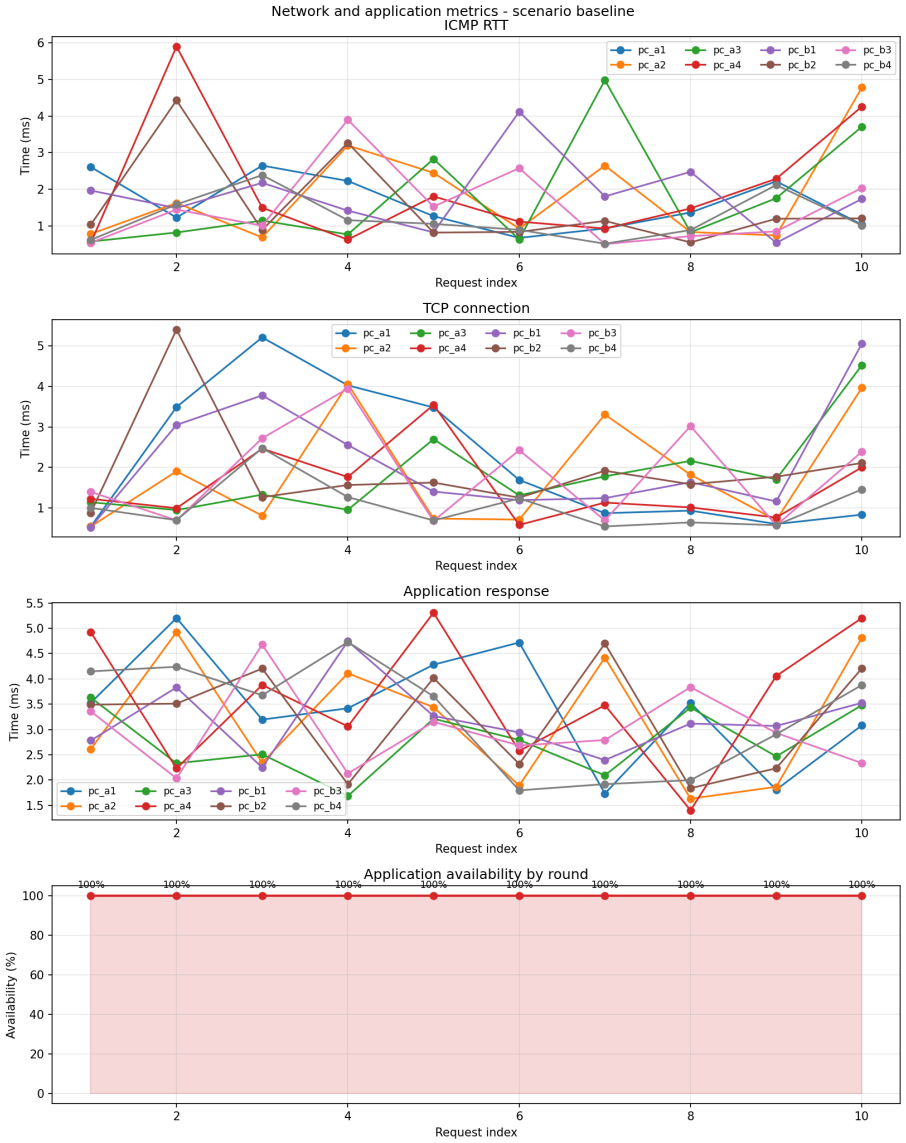


Figure 5.2: Per-client network and application measurements for the direct-server baseline in `dos_lab/simulation_0001`.

stable estimates of host consumption.

The mitigation baseline in `mitigation/simulation_0002` independently satisfies the same validity criteria: all eight clients are present in every round, all 80 ICMP and TCP checks succeed, and all 80 application transactions complete. Its application response is slower because the transaction traverses the proxy: the mean is 6.058 ms, compared with 3.218 ms for the direct-server baseline. This baseline cost is considered together with the protection result in Section 5.4.

5.3 Controlled DoS Experiment and Observed Results

The controlled DoS phase was not part of the initial topology-generation requirement. It was introduced only after generation, routing, and service measurement were sufficiently stable to support a controlled comparison between two operating conditions. Reusing the same topology and measurement path was preferred to building a separate attack-specific environment, because it keeps the baseline and adverse phases comparable and turns the experiment into an extension of the existing verification workflow.

The second phase introduces a bounded connection-based workload while the same eight legitimate clients continue to use the same application interface. The experiment is best interpreted as connection-capacity contention rather than as a bandwidth-flooding benchmark. Each controlled agent opens several TCP connections to the service and attempts to keep them established for a bounded period. The target application, by contrast, exposes only 20 concurrent application slots. The expected effect is therefore contention for finite server-side session capacity, not saturation of a physical link.

The control architecture consists of one `attacker` node and three agents. Each agent listens on TCP port `9999`. The controller reads the agent addresses from `zombies.txt` and sends a command containing the target address, target port, requested number of connections, and duration. The orchestrator uses 40 requested connections per agent and a 60-second duration by default, with an additional global cap that prevents a configured duration above 90 seconds. With three agents, the requested workload therefore consists of up to 120 concurrent connection workers competing with a service configured for 20 active application sessions.

The agents are themselves bounded. The implementation limits the number of connections created by one agent and limits the duration of a load episode. Each worker reconnects when necessary so that pressure can be maintained across the configured interval even if a server-side timeout closes an idle connection. These restrictions make the mechanism suitable for a repeatable laboratory experiment: load intensity and duration are explicit parameters and the agent remains confined to the addresses provided by the controller inside the generated topology.

The full orchestration does not simply start the workload and then begin measuring. Client measurement starts first, while workload activation is scheduled asynchronously after the configured start delay. In the direct-server run this creates a visible transition within the same result file. The first two request rounds complete successfully for all eight legitimate clients; from round 3 onward, none of the legitimate requests receives a valid `pong`. Table 5.6 shows this progression.

Across the complete controlled-load measurement, 16 of 80 legitimate requests succeeded, giving an overall availability of 20%. This percentage includes the first two rounds, which completed before the controlled workload became active. During the attack-active interval, from rounds 3 through 10, none of the 64 legitimate application requests completed successfully; application availability while the workload was active was therefore 0%. The 20% value must consequently be interpreted as the availability of the complete transition run, whereas 0% describes the period in which the adverse workload was actually active. Importantly, all 80 ICMP probes succeeded and all 80 TCP handshakes were recorded as successful. Of the 64 application failures, 47 did not receive the expected `pong` and 17 reported a connection reset as `socket_error:104` after the TCP connection had already been established. The experiment therefore exhibits a clear

Round	Requests	Successful	Availability
1	8	8	100%
2	8	8	100%
3	8	0	0%
4	8	0	0%
5	8	0	0%
6	8	0	0%
7	8	0	0%
8	8	0	0%
9	8	0	0%
10	8	0	0%

Table 5.6: Application availability by request round during the direct-server controlled-load measurement.

separation between network-level reachability and application-level availability.

This result is one of the central observations of the evaluation. If the test had relied on `ping` alone, the service would have appeared fully reachable during the entire workload window. If it had relied only on TCP connection establishment, the same conclusion would have been reached, because all handshakes succeeded. Only the application-level request identifies that 64 of the 80 legitimate transactions did not complete successfully. In other words, a reachable host and an open TCP port do not imply that the service is currently able to satisfy legitimate work.

The timing values show the same distinction. Mean ICMP RTT increased from 1.686 ms in the baseline to 2.127 ms during the controlled-load run. Mean TCP connection time increased from 1.803 ms to 2.198 ms. Among the 16 requests that still succeeded, mean application response time was 3.432 ms, the median was 3.347 ms, the 95th percentile was 4.567 ms, and the maximum was 4.735 ms. These successful-request latencies alone would suggest only modest change. The major degradation is therefore represented by the fall in completion probability, not by a large increase in the latency of transactions that happen to complete.

Metric	Baseline	Controlled run
Legitimate requests	80	80
Successful application requests	80	16
Overall application availability	100.00%	20.00%
Attack-active application availability	–	0.00%
Successful ICMP probes	80	80
Successful TCP handshakes	80	80
Mean ICMP RTT	1.686 ms	2.127 ms
Mean TCP connection time	1.803 ms	2.198 ms
Mean application response time*	3.218 ms	3.432 ms
Median application response time*	3.202 ms	3.347 ms
95th percentile application response*	4.922 ms	4.567 ms
Maximum application response time*	5.309 ms	4.735 ms

Table 5.7: Client-side comparison for `dos_lab/simulation_0001`. Values marked with * are calculated only from requests that produced an application response.

The transition is visible directly in Figure 5.3: application availability is 100% in the first two rounds and falls to 0% from round 3 onward. Application-response curves consequently stop after round 2, although the ICMP and TCP panels retain measurements for all ten rounds.

Figure 5.4 separates the complete transition from the workload-active interval and anticipates the mitigation comparison in Section 5.4. The plot focuses on completed application requests rather than on ICMP or TCP reachability, because those lower-layer

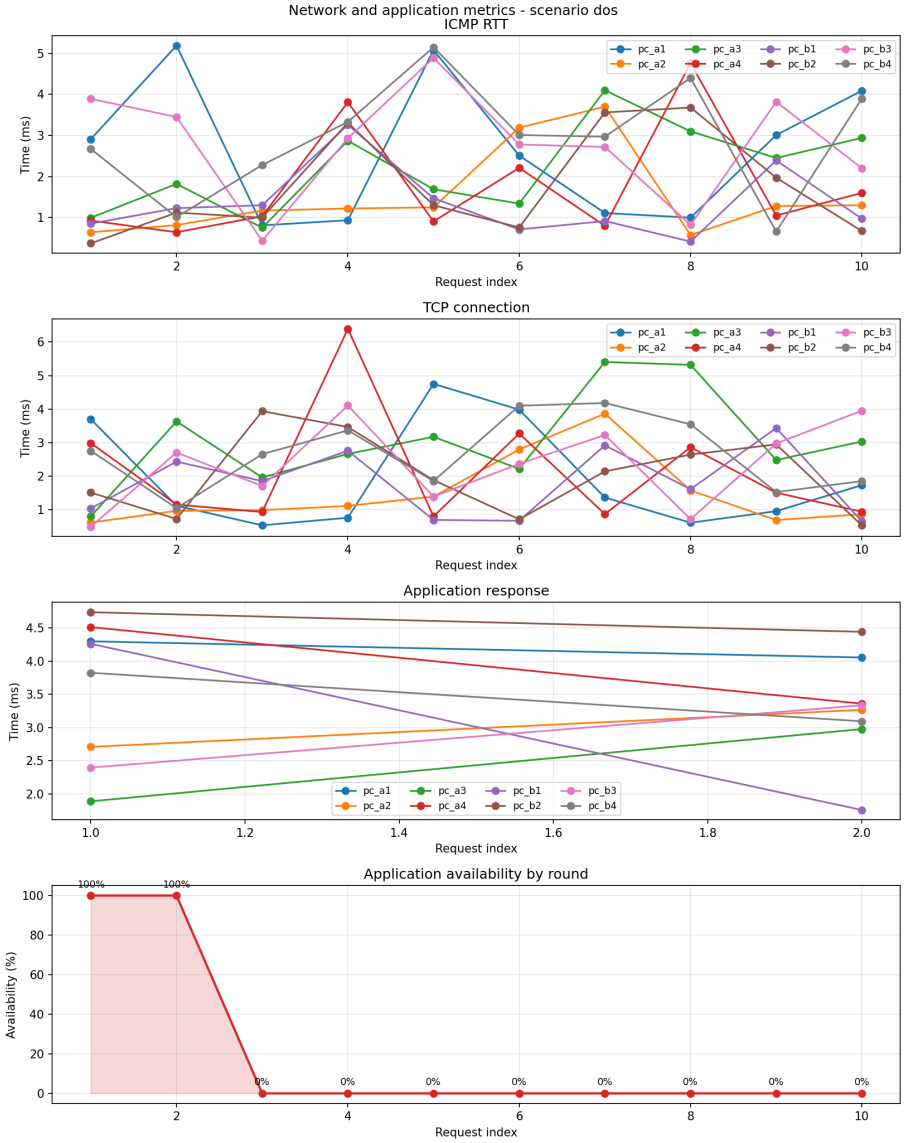


Figure 5.3: Per-client network and application measurements during the direct-server controlled-load phase in `dos_lab/simulation_0001`.

checks remained successful throughout both experiments.

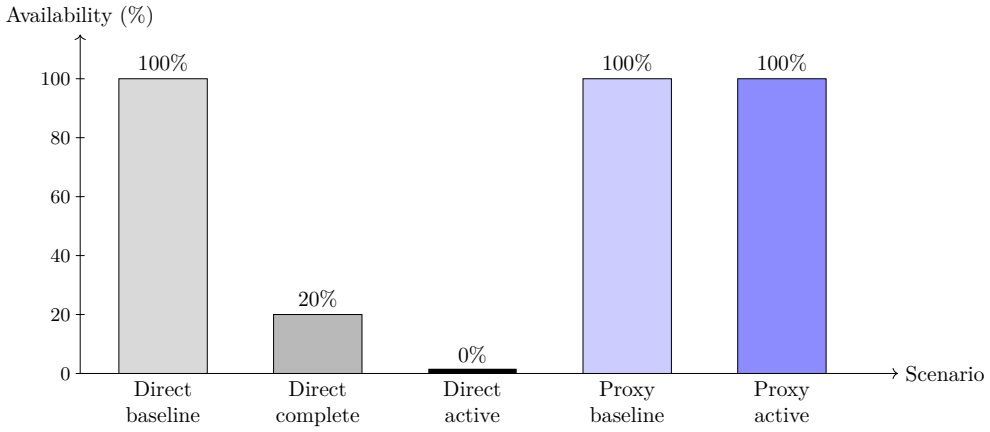


Figure 5.4: Application availability without and with the reverse-proxy mitigation. "Complete" includes the two pre-activation rounds; "active" denotes rounds 3–10.

The service-side counters explain the client observation. During the baseline, the service recorded 80 accepted connection events and no rejected or `BUSY` events. During the controlled-load window it recorded 56 accepted connection events together with 264 rejected/`BUSY` events. The maximum number of simultaneously active application connections reached exactly 20, which is the configured semaphore limit, and the maximum active thread count rose from four in the baseline to 25 during the load phase. Periodic server samples recorded a mean of 15.00 active connections during the controlled-load scenario, compared with approximately zero at the one-second sampling instants during the short baseline transactions.

Server-side observation	**Baseline**	**Controlled load**
Accepted connection events	80	56
Rejected / `BUSY` events	0	264
Maximum active connections	1	20
Mean active connections in periodic samples	0.00	15.00
Maximum active threads	4	25
Mean container CPU†	0.53%	2.13%
Maximum container CPU†	0.88%	7.14%
Maximum container memory†	0.05%	0.07%
Maximum container PIDs†	3	23

Table 5.8: Server and container observations for `dos_lab/simulation_0001`. Values marked with † are based on six Docker-stat samples per scenario and are therefore descriptive.

The resource samples show higher CPU activity and many more processes/threads under load, but they should not be over-interpreted. Only six Docker-stat rows were archived for each scenario, and CPU utilisation on a shared host is sensitive to scheduling and unrelated host activity. The stronger evidence is the application-specific instrumentation: the service reached its configured limit of 20 active connections and generated hundreds of overload rejections while overall availability fell to 20% and attack-active availability fell to 0%. These observations are directly related by the service implementation and do not depend on a precise host CPU baseline.

The per-client distribution is also useful. Every legitimate client completed two out of ten requests, corresponding to 20% over the complete controlled run; for every client, both successful requests occurred before workload activation, so availability during the

attack-active interval was 0%. There is therefore no indication that a single access LAN or routing path was responsible for the failures. Instead, the loss of availability appears across both client networks at the same time, consistent with contention at the common server endpoint. Combined with the fact that all ICMP probes remained successful, this supports the interpretation that the routing infrastructure continued to operate while the application capacity was exhausted.

5.4 Reverse-Proxy Mitigation Results

The representative reverse-proxy execution evaluates the mitigation topology under the same client schedule. A rate-limiting reverse proxy exposes the public endpoint at `10.30.1.10:9000` and forwards admitted sessions to a backend at `10.30.2.10:9000`. The proxy applies global and per-source connection limits, while the clients continue to use the same application protocol and the controlled agents retain the same activation point. Consequently, the intentional experimental change is the protected service path rather than the client measurement procedure.

The baseline and controlled-load files in `mitigation/simulation_0002` each contain 80 requests. Every request succeeded at the ICMP, TCP, and application layers in both phases. In particular, all 64 legitimate requests in rounds 3–10 completed while the controlled workload was active, compared with 0 of 64 in the direct-server experiment. Each of the eight clients therefore achieved ten successes out of ten, with no difference in availability between the two access LANs.

Client-side metric	**Proxy baseline**	**Proxy load**	**Direct load**
Successful application requests	80 / 80	80 / 80	16 / 80
Overall application availability	100.00%	100.00%	20.00%
Attack-active availability	–	100.00%	0.00%
Successful ICMP probes	80 / 80	80 / 80	80 / 80
Successful TCP handshakes	80 / 80	80 / 80	80 / 80
Mean ICMP RTT	1.550 ms	1.674 ms	2.127 ms
Mean TCP connection time	1.808 ms	1.781 ms	2.198 ms
Mean application response time	6.058 ms	5.545 ms	3.432 ms*
Median application response time	5.896 ms	5.133 ms	3.347 ms*
95th percentile application response	9.177 ms	9.502 ms	4.567 ms*
Maximum application response time	11.696 ms	11.019 ms	4.735 ms*

Table 5.9: Client-side results for the mitigation experiment and the corresponding direct-server load phase. Direct-load application latencies marked with * describe only the 16 successful requests.

Figures 5.5 and 5.6 show the complete per-client series for the mitigation experiment. Unlike the direct-server load plot, the application panel contains observations for every client and round, and the availability panel remains at 100% after workload activation.

The proxy introduces a measurable baseline cost: mean application response time is 6.058 ms through the proxy and 3.218 ms in the direct-server baseline, a difference of 2.840 ms. This latency difference should be interpreted as a descriptive observation from one run, not as a stable estimate of proxy overhead. More importantly for this experiment, the distribution remains bounded under load: the mitigated mean is 5.545 ms and the 95th percentile is 9.502 ms while all requests complete. The lower mean under load than in the proxy baseline is another reason not to infer small performance differences from a single short execution.

The backend instrumentation is consistent with the client-side result. During the mitigated load phase the backend records 92 accepted connections, no rejected or `BUSY` event, and a maximum of eight simultaneous connections, well below its limit of 20. The count exceeds the 80 legitimate requests because a limited subset of controlled-workload

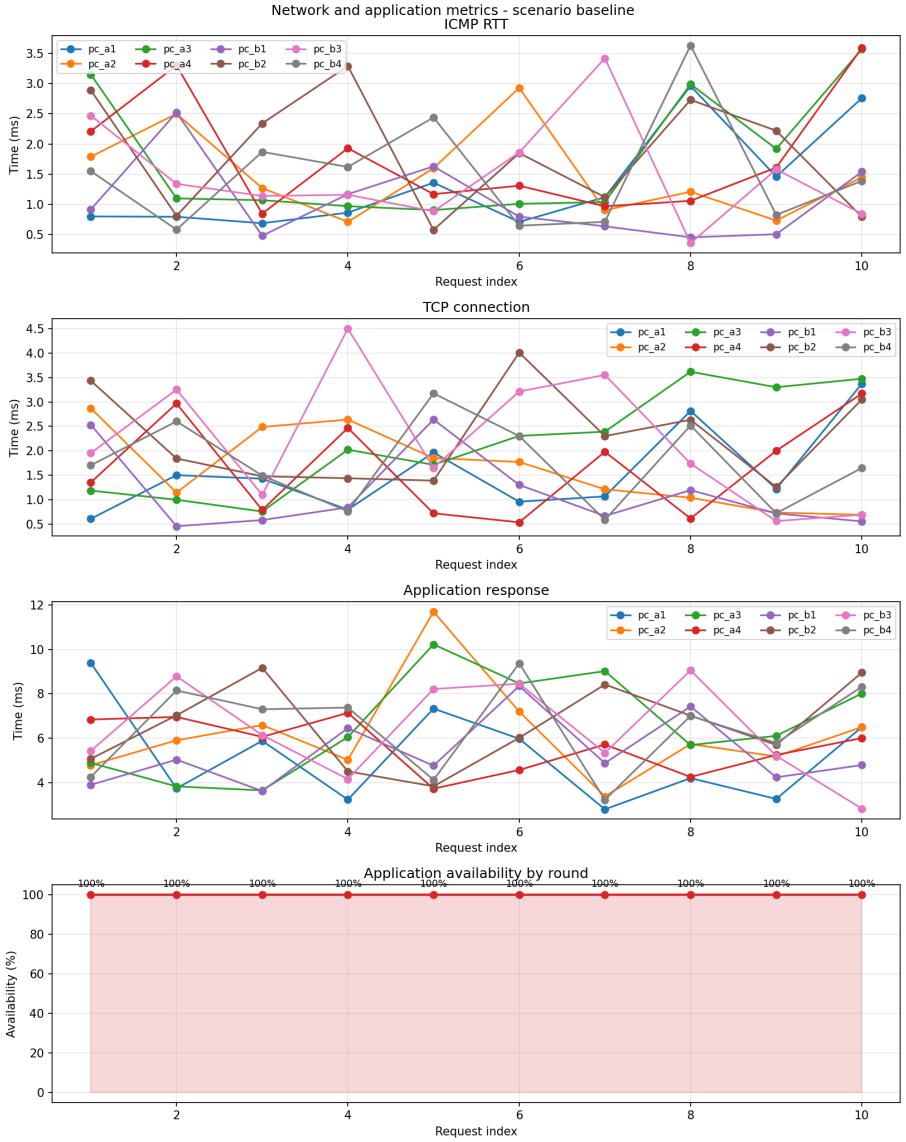


Figure 5.5: Per-client network and application measurements for the reverse-proxy baseline in `mitigation/simulation_0002`.

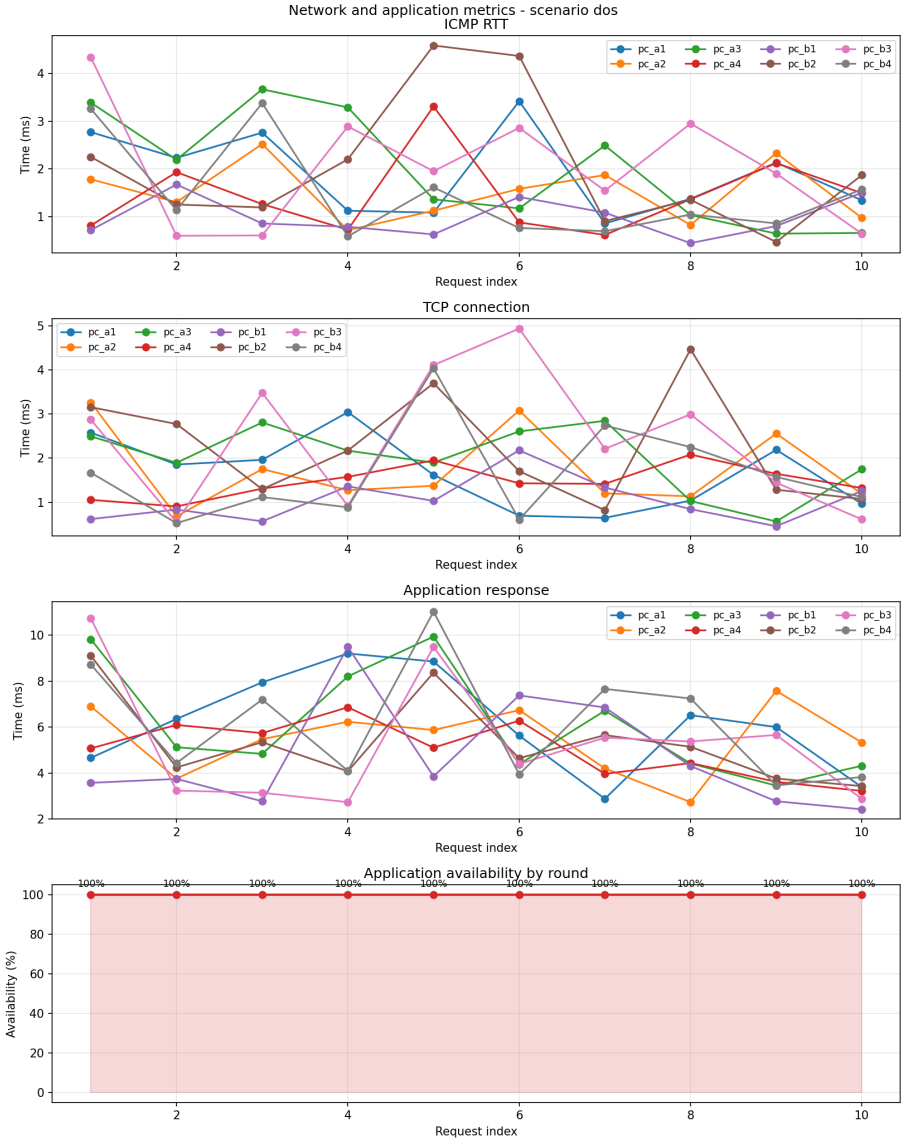


Figure 5.6: Per-client network and application measurements during the reverse-proxy controlled-load phase in `mitigation/simulation_0002`.

sessions also reaches the backend. The reverse proxy therefore does not remove all adverse traffic; in this run it prevents that traffic from exhausting the backend's application slots.

Backend observation	**Proxy baseline**	**Proxy load**	**Direct load**
Accepted connection events	80	92	56
Rejected / `BUSY` events	0	0	264
Maximum active connections	2	8	20
Mean active connections in periodic samples	0.00	4.62	15.00
Maximum active threads	4	10	25
Mean container CPU†	0.49%	0.78%	2.13%
Maximum container CPU†	0.75%	1.56%	7.14%
Maximum container memory†	0.05%	0.06%	0.07%
Maximum container PIDs†	3	9	23

Table 5.10: Backend and container observations with mitigation, compared with the unmitigated load phase. Values marked with † are based on six Docker-stat samples per phase.

These results demonstrate effectiveness only for the bounded, connection-oriented workload and rate limits used in this laboratory. They do not establish resistance to volumetric traffic, source-address rotation, distributed sources that defeat per-source limits, proxy resource exhaustion, or the application-layer techniques discussed earlier in the chapter. The representative pair shows that attack-active application availability increases from 0% without the proxy to 100% with it, while the backend maximum falls from 20 to eight simultaneous connections and overload rejections fall from 264 to zero. Whether these values persist across clean laboratory restarts is evaluated explicitly in Section 5.5.

The experiment demonstrates why the measurement framework records several layers independently. ICMP RTT answers whether the host is reachable; TCP connection time describes connection establishment; application response time measures successful transactions; application status captures whether the transaction actually completed; and server-side counters reveal the internal capacity state. No single metric is sufficient to describe the experiment. The combination makes it possible to identify a state in which the network remains healthy, TCP remains reachable, but the application is no longer reliably available to legitimate clients.

5.5 Repeated-Run Repeatability Analysis

To evaluate run-to-run repeatability, the complete baseline and controlled-load procedure was executed five times for each topology. The runs were organised as five acquisition blocks, each containing one direct-server run and one reverse-proxy run; the laboratory was cleaned and restarted between executions. Client count, round count, timeouts, workload activation point, number of controlled connection workers, and workload duration were unchanged. The direct run preceded the mitigated run inside each block, so the pairing controls the acquisition period but does not remove every possible host-order effect.

The statistical unit is one complete execution, not one of its 80 client requests. Request-level observations within a run are dependent because they share the same emulated network, server process, host scheduler, and workload episode. Each run was therefore summarised first; descriptive statistics were then calculated across the five run-level values. Table 5.11 reports the median, first and third quartiles, and sample variance s^2 with denominator $n-1$. Percentiles within each run use the same nearest-rank method as `show_connection_results.py`. For availability under active load, only rounds 3–10 are included, yielding 64 legitimate requests per run.

Figure 5.7 displays the observed run-level values directly, without fitting a distribu-

	Direct server		Reverse proxy	
Run-level metric	**Median [Q1, Q3]**	s^2	**Median [Q1, Q3]**	s^2
Baseline availability (%)	100 [100, 100]	0	100 [100, 100]	0
Active-load availability (%)	0 [0, 0]	0	100 [100, 100]	0
Baseline application median (ms)	3.335 [3.290, 3.336]	0.088	5.808 [5.437, 5.896]	0.144
Baseline application p95 (ms)	5.399 [5.008, 5.900]	0.338	9.514 [9.170, 10.915]	1.899
Active-load application median (ms)	–	–	5.315 [5.236, 5.700]	0.271
Active-load application p95 (ms)	–	–	9.481 [9.151, 9.540]	0.663
Rejected / `BUSY` events	264 [264, 264]	0	0 [0, 0]	0
Maximum active backend connections	20 [20, 20]	0	8 [8, 8]	0

Table 5.11: Run-level descriptive statistics across five complete executions per topology. Variance is the sample variance across runs. Dashes denote an undefined latency statistic because no direct-server request completed during active load.

tion to the five executions. The paired layout preserves the acquisition-block structure and separates quantities with different units into distinct panels.

All five direct-server baselines and all five reverse-proxy baselines achieved 80 successes out of 80 requests. During the complete transition phase, every direct-server run achieved 16 successes out of 80, because only the two pre-activation rounds completed. Consequently, all five direct-server runs had 0 successes out of 64 during active load. In contrast, every reverse-proxy run completed all 64 active-load requests. The paired availability difference was therefore +100 percentage points in each of the five acquisition blocks: median +100, interquartile interval $[+100, +100]$, and sample variance zero. This exact replication provides evidence of consistent behaviour for the tested configuration beyond the original single execution, although five runs remain a small sample and do not justify extrapolation to other workload models.

Latency is reported only where an application transaction completed. No direct-server application latency exists for rounds 3–10, so assigning a timeout value or comparing the proxy only with the 16 pre-activation direct successes would bias the result. The protected path instead produced an active-load median response between 4.934 and 6.288 ms across runs and a p95 between 7.661 and 9.590 ms. In the baseline, the paired increase in the per-run application median introduced by the proxy had a median of 2.167 ms and an interquartile interval of [2.147, 2.472] ms. This provides a run-level estimate of the protection overhead while keeping availability and latency as separate outcomes.

The server-side evidence is equally stable. Every direct run reached the 20-connection application limit and recorded 264 rejected or `BUSY` events during the controlled-load phase. Every mitigated run limited the backend maximum to eight connections and recorded zero overload rejection. Thus each paired block reduced the maximum backend occupancy by 12 connections and the rejection count by 264. The zero variances in these discrete outcomes describe deterministic repetition under the fixed laboratory parameters; they must not be interpreted as proof that the same values hold for untested rates, source populations, protocols, or host environments.

5.6 Threats to Validity

The results support a comparison between the two implemented architectures under the stated parameters, but several threats constrain their interpretation.

Construct validity. Application availability is defined as receipt of the expected response within the configured timeout. This is stronger than ICMP reachability or completion of a TCP handshake, but it represents one small TCP service rather than every notion of service quality. Response time is defined only for completed transactions; failed requests are therefore reported through availability and are not assigned an artificial latency. The controlled workload reproduces a bounded application-layer resource-exhaustion condition generated by known nodes. It is useful for comparing the direct and protected paths, but it is not a measurement of Internet-scale DDoS resistance.

Internal validity. Both emulated networks, their containers, and the measurement processes share one physical host. CPU scheduling, filesystem activity, and Docker overhead can consequently affect clients, workload generators, proxy, and server at the same time. Pairing the two architectures within acquisition blocks reduces changes between distant measurement periods, and cleaning and restarting the laboratory reduces residual runtime state. However, the direct-server run always preceded the proxy run inside a block, so an order or warm-cache effect cannot be excluded. The proxy topology also adds both rate limiting and an extra processing hop; the experiment estimates the effect of the complete architectural change and does not isolate every proxy mechanism individually.

External validity. The experiment uses Kathará and Docker on one workstation, a fixed routed topology, eight legitimate clients, one application protocol, one server capacity limit, and one controlled workload profile. Different machines, container runtimes, network delays, loss patterns, request mixes, proxy rules, or adversarial strategies may produce different results. In particular, a distributed workload with many source identities could interact differently with rate limiting. The observed 0% and 100% active-load availability values therefore characterise the tested configurations and must not be extrapolated to production services.

Conclusion validity. Five paired executions provide stronger evidence than a single demonstration, and treating each complete run as the statistical unit avoids presenting the dependent client requests as independent samples. Nevertheless, five pairs are too few for precise population estimates or a persuasive analysis of small latency differences. The repeated binary and counter outcomes are reported descriptively; zero sample variance records what occurred in these runs, not an absence of variability in the wider population. Further evaluation should randomise execution order, increase the number of independent restarts, and vary workload intensity and host conditions before making broader causal or performance claims.

5.7 Chapter Summary

This chapter evaluated the generated framework through a controlled experiment executed entirely inside Kathará. The testbed combined eight legitimate clients, an OSPF-based routed core, a separate workload network, and an instrumented TCP application service. The orchestration layer verified target reachability before measurement, executed all legitimate clients concurrently for each round, and stored client, application, server, and container metrics under a run-specific directory.

Across all five direct-server runs, every baseline produced 80 successful application transactions out of 80 attempts. Every complete transition run had 20% availability because its first two rounds preceded workload activation; during rounds 3–10, every run had 0 successes out of 64 despite successful ICMP probes and TCP handshakes. In all five executions the service reached its configured limit of 20 active connections and recorded 264 overload rejections. The repeated result demonstrates a distinction that is central to end-to-end service evaluation: network reachability and TCP acceptance do not necessarily imply application availability.

The five reverse-proxy runs provide the corresponding architectural comparison. Every baseline and controlled-load phase completed all 80 application requests, including all 64 requests issued during active load. In each run the protected backend recorded no overload rejection and reached at most eight simultaneous connections. The paired improvement in active-load availability was 100 percentage points in every acquisition block, while the median baseline application-response overhead was 2.167 ms. For the bounded workload used here, the proxy therefore preserved application availability at the cost of higher baseline response time. This is repeated evidence for the tested configuration, not a claim of general DDoS resistance.

The real-world cases discussed at the beginning of the chapter clarify the scope of this result. Mirai/Dyn provides a structural example of distributed sources, the GitHub incident illustrates reflection and amplification, HTTP/2 Rapid Reset illustrates application-layer request pressure, and the 2025 Cloudflare case illustrates hypervolumetric short-duration flooding. The Kathará experiment does not reproduce those Internet-scale mechanisms. Rather, it isolates a common availability principle in a controlled setting: when offered workload exceeds a finite service resource, legitimate transactions can fail even while lower-layer connectivity remains available.

The results also validate the usefulness of the framework as an experimental platform. The topology is generated from YAML, the same clients and target can be reused across scenarios, measurements are executed with a consistent schema, and runs can be checked using explicit reachability and completion criteria. Five repeated acquisition blocks show that the principal availability and backend-capacity outcomes are stable across clean laboratory restarts, while the reported quartiles and variances make the remaining latency dispersion explicit.

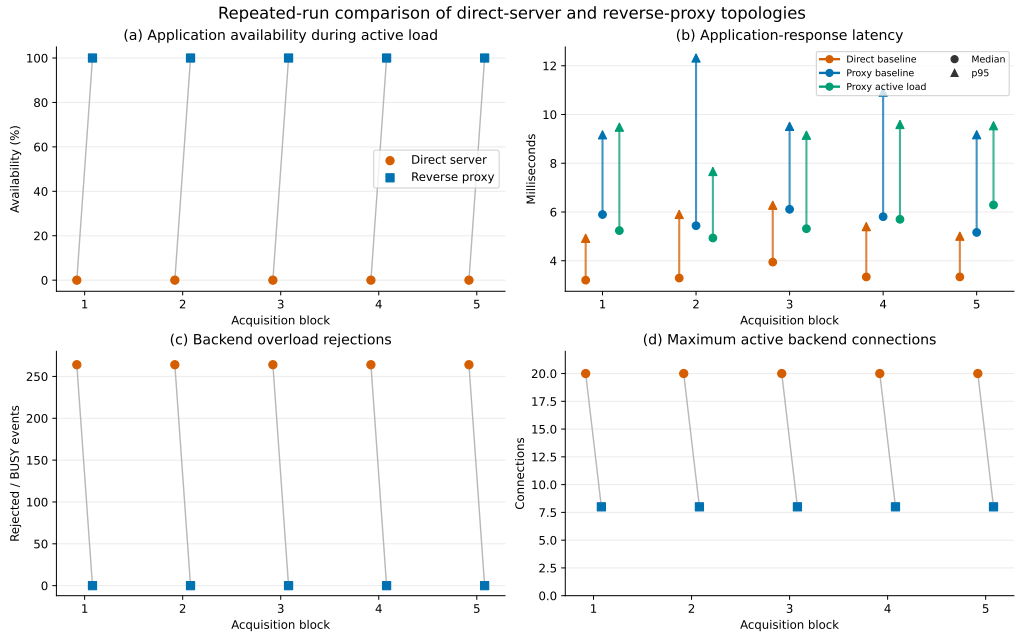


Figure 5.7: Run-level comparison across the five paired acquisition blocks. Grey lines connect the direct-server and reverse-proxy observations within each block. Panel (a) reports attack-active application availability; panel (b) reports application-response medians and p95 values where a transaction completed; panels (c) and (d) report backend overload rejections and maximum active connections. Direct-server active-load latency is absent because no request completed during that interval.

Chapter 6

Robustness, Security Limitations and Scalability Evaluation

The previous chapters presented the configuration model, the validation pipeline, the generation of Kathará artefacts, and the experimental use of the framework. This chapter evaluates the framework from a different perspective. Rather than considering only whether a valid laboratory can be generated, it investigates how the translator behaves when its input becomes large, structurally complex, ambiguous, or potentially unsafe.

From the perspective of the software-development lifecycle introduced in Section 2.2, this chapter represents a final verification stage. Earlier iterations primarily asked whether the framework could generate and execute the intended laboratories. The tests below instead challenge assumptions made during implementation and ask what happens when the same mechanisms are exercised with large, malformed, ambiguous, or potentially unsafe inputs. The resulting observations are therefore feedback on the design choices described in Chapters 2–4, not only a list of independent limitations.

The analysis has two objectives. The first is to identify scalability limits through reproducible stress tests. The second is to discuss implementation choices that may become robustness or security weaknesses when configuration files are malformed or originate from an untrusted source. These two aspects are related. A configuration language is not only a convenient input format: it is also an interface through which a user can request the creation of thousands of nodes, routes, routing statements, files, and startup commands.

It is important to distinguish three categories throughout this chapter. A *security vulnerability* is a behaviour that can become exploitable across an explicit trust boundary. A *robustness weakness* is a condition that can result in an incorrect or misleading laboratory without necessarily creating a security compromise. A *model limitation* is a deliberate or implicit restriction in the set of networks that the translator can represent. Not all limitations described below should therefore be interpreted as exploitable vulnerabilities.

6.1 Stress-Test Methodology

The descriptive scalability benchmarks exercise the translator itself rather than the runtime scalability of Docker or Kathará. Each input is therefore processed in two modes. The first uses the framework's validation-only option and measures YAML loading, structural validation, and semantic validation without creating a laboratory. The second performs complete generation and additionally writes `lab.conf`, node startup files, and FRRouting files where required. Starting the resulting containers is intentionally outside

the scope of these measurements because it would mix translator performance with Docker, Kathará, kernel, and host-resource limitations.

The benchmark data used in this chapter are the results recorded in `final_benchmark.csv` and `final_summary.csv`. The CSV data contain two measured repetitions for each available scenario and mode. No measured execution reached a timeout. Every valid configuration returned exit code 0 in both repetitions. The intentionally invalid configuration returned exit code 1 in both repetitions, as expected. Because two repetitions cannot support strong distributional or inferential claims, the measurements are used only as descriptive observations of the tested executions.

The scenarios were selected so that different dimensions of input complexity could be isolated rather than increasing only the number of nodes:

- `flat_10000.yml`: 10,000 nodes sharing a large flat topology, intended to stress node and address processing without creating thousands of distinct collision domains;
- `star_4000.yml`: approximately 4,000 nodes arranged around a central router with a separate point-to-point collision domain for each peripheral connection, intended to stress subnet and collision-domain validation;
- `chain_1000.yml`: 1,000 routers connected in a chain, stressing repeated routed-node and interface generation;
- `enterprise_100r_10h.yml`: 100 routers with ten hosts per routed LAN, providing a more realistic mixed topology;
- `dense_500.yml`: 500 routers with multiple inter-router links, increasing interface density;
- `ospf_1000.yml`: 1,000 OSPF routers, stressing FRRouting validation and configuration generation;
- `routes_20000.yml`: a configuration dominated by 20,000 static-route declarations;
- `commands_100000.yml`: a configuration dominated by 100,000 custom startup commands;
- `large_yaml_15mb.yml`: a 15.64 MiB YAML configuration, intended to stress deserialisation and in-memory representation;
- `invalid_5000.yml`: 5,000 intentionally invalid nodes, intended to verify that large erroneous inputs are rejected without producing a laboratory.

For each measured run the benchmark records elapsed time, peak resident memory, exit status, input size, number of generated files, and output size. Table 6.1 reports the aggregated values contained in the provided summary CSV. Times are medians of the two recorded repetitions, while memory is the maximum peak RSS observed in those repetitions.

Scenario	Validate (s)	Generate (s)	Val. RSS (MiB)	Gen. RSS (MiB)	Input (MiB)	Files	Output (MiB)
Flat 10,000 nodes	4.065	8.840	97.04	109.16	1.127	10004	1.146
Star 4,000 nodes	14.565	29.570	60.69	67.36	0.688	4004	0.727
Chain 1,000 routers	1.320	2.835	27.03	28.66	0.165	4001	0.579
Enterprise 100r/10h	0.535	1.095	25.53	26.96	0.141	1401	0.185
Dense 500 routers	1.235	2.520	24.79	26.29	0.135	2001	0.347
OSPF 1,000 routers	1.730	3.695	35.38	37.88	0.292	4001	0.675
20,000 static routes	2.900	5.920	82.34	92.83	1.020	6	0.867
100,000 commands	4.410	8.645	76.81	87.23	3.709	2	2.946
Large YAML	7.610	15.240	74.47	124.36	15.640	2	15.030
Invalid 5,000 nodes	2.025	–	57.04	–	0.595	0	0

Table 6.1: Descriptive stress-test results from the supplied benchmark CSV files. Times are medians of two raw measurements and should not be interpreted as stable population estimates.

The results show that node count alone is not a sufficient predictor of translation cost. The clearest example is the comparison between the flat and star topologies. The flat topology contains 10,000 nodes and validates in 4.065 seconds. The star topology contains only about 4,000 nodes but requires 14.565 seconds for validation. It is therefore approximately 3.6 times slower despite containing substantially fewer nodes. Complete generation exhibits the same effect: 8.840 seconds for the 10,000-node flat case and 29.570 seconds for the 4,000-node star case.

This difference is explained by the semantic checks performed on collision domains. The validator compares the subnets associated with different collision domains in order to reject overlapping Layer 3 address spaces. When a topology contains a very large number of distinct point-to-point domains, the number of pairwise comparisons can grow approximately as

$$\frac{D(D-1)}{2},$$

where D is the number of collision domains being compared. In the worst case this corresponds to quadratic growth, $O(D^2)$. A flat topology, by contrast, may contain many nodes while keeping D small. The stress results therefore identify the number of distinct collision domains as a more important scalability factor than raw node count for this part of semantic validation.

The tests dominated by configuration volume reveal a different limit. The 20,000-route configuration validates in 2.900 seconds and reaches 82.34 MiB peak RSS, while the 100,000-command configuration validates in 4.410 seconds and reaches 76.81 MiB. The 15.64 MiB YAML file requires 7.610 seconds for validation and reaches 74.47 MiB during validation. During complete generation its peak RSS rises to 124.36 MiB, the highest value in the supplied summary. These cases indicate that large nested YAML structures impose a cost even when the logical topology is simple, because the complete document must be deserialised into Python dictionaries and lists before semantic processing can begin.

Another consistent result is the relationship between validation and complete generation time. Across all valid scenarios, the ratio between the median generation time and the median validation time ranges from approximately 1.96 to 2.17, with a mean close to 2.06. This behaviour is consistent with the current implementation, in which normal execution validates the configuration at the command-line boundary and the lower-level generation function loads and validates it again before writing files. The duplicated validation gives `generate_lab()` an independent safety property, because the function does not assume that its caller has already validated the input. However, the stress tests show that this design also becomes measurable overhead for large configurations.

Finally, `invalid_5000.yml` provides a useful negative result. Both recorded executions terminated in about 2.02–2.03 seconds with exit code 1, without timeout and without generating output files. This demonstrates that the validation pipeline can reject a large erroneous configuration without continuing into the generation phase. From a robustness perspective this is a positive property.

6.2 Security and Robustness Weaknesses

The stress tests identify performance boundaries, but other weaknesses arise from the expressive features of the configuration language itself. Several of these features are deliberate escape mechanisms. They are useful when YAML files are trusted and authored by the laboratory designer, but they create a clear trust boundary if configurations are downloaded, shared, or generated by another user.

Arbitrary startup commands. The most important security-sensitive feature is the `commands` list. Each command string is copied literally to the node startup file. This is intentionally flexible: it allows the framework to launch applications, configure kernel

parameters, create files, or perform operations not covered by the declarative model. It also means that a YAML file can carry executable shell commands.

A harmless proof-of-concept configuration is:

```
nodes:
  pc_a:
    type: host
    interfaces:
      - network: lan_a
        ip: 10.0.1.10/24
    commands:
      - echo "COMMAND_EXECUTED" > /tmp/translator_test
```

Listing 6.1: Benign demonstration of command execution through a startup entry.

The generated `pc_a.startup` file contains the command literally. When the laboratory starts, the command executes inside the node container. This should not be described as an automatic host compromise: execution occurs in the container context, and escalation beyond that boundary depends on the container configuration and runtime privileges. Nevertheless, if YAML is accepted from an untrusted source, the feature becomes a command-execution boundary and should be treated accordingly.

A theoretical mitigation is to introduce two operating modes. A strict or safe mode would reject `commands` entirely and permit only structured operations represented by dedicated YAML fields. An explicit `--allow-commands` option could retain the current behaviour for trusted laboratories. Common operations currently expressed as shell strings could progressively be converted into typed fields. Additional protection could include command allowlists, clear warnings when a configuration contains opaque commands, and reduced Linux capabilities for containers that do not need privileged operations.

Opaque commands can bypass semantic intentions. The parser performs only limited consistency checks on command strings. For example, it can recognise some explicit attempts to disable IPv4 forwarding on a node declared as a router. General shell semantics, however, cannot be reliably inferred from text. Equivalent operations can be written in many forms. A configuration such as

```
r1:
  type: router
  forwarding: true
  interfaces:
    - network: lan_a
      ip: 10.0.1.1/24
    - network: lan_b
      ip: 10.0.2.1/24
  commands:
    - sh -c 'echo 0 > /proc/sys/net/ipv4/ip_forward'
```

Listing 6.2: Equivalent command that changes a forwarding runtime property.

illustrates the general problem. The declarative field states that the node is intended to forward traffic, while a later opaque command can change the runtime state. In addition, `forwarding: true` currently expresses intent and validation consistency but does not itself emit a kernel `sysctl` operation.

The general solution is not to build an increasingly complex regular expression for shell commands. A stronger design would make forwarding a generated postcondition: when `forwarding: true` is selected, the generator could explicitly configure the kernel state at a well-defined stage and optionally verify it after startup. In strict mode, commands that directly modify protected networking parameters could be disallowed.

Raw FRRouting configuration bypasses most semantic guarantees. The `frr.config` field provides another intentional escape mechanism. Instead of generating protocol blocks from structured YAML fields, the framework writes user-provided FRRouting text directly to `frr.conf`. This is useful for advanced FRR features not represented by the declarative model, but the translator cannot fully validate the semantics of arbitrary FRR syntax.

For example:

```
frr:
  config: |
    router ospf
     this-command-does-not-exist
```

Listing 6.3: Raw FRR content that cannot be fully validated by the YAML model.

can be syntactically valid YAML while still containing an invalid FRR command. Successful YAML translation therefore does not imply successful FRR startup.

A possible mitigation is to distinguish generated FRR configuration from raw FRR configuration in the validation report and mark the latter as "unverified". A stricter implementation could invoke the FRR parser in a disposable validation environment before accepting the laboratory. Another option is to disable `frr.config` in strict mode and require the structured protocol model whenever reproducibility and static validation are more important than maximum expressiveness.

Static-route next hops are less deeply validated. Interfaces and default gateways receive strong semantic checks, including address parsing and directly connected gateway validation. Arbitrary static routes are less extensively analysed. Consider a router connecting two different networks:

```
r1:
  type: router
  interfaces:
    - network: lan_a
      ip: 10.0.1.1/24
    - network: lan_b
      ip: 10.0.2.1/24

  routes:
    - to: 172.16.0.0/16
      via: 192.168.200.1
```

Listing 6.4: Static route with a next hop outside the router's connected subnets.

The router is directly connected only to the `10.0.1.0/24` and `10.0.2.0/24` networks. However, the static route specifies `192.168.200.1` as its next hop, even though that address is not reachable through either configured interface.

The route can still be translated into:

```
ip route add 172.16.0.0/16 via 192.168.200.1
```

Consequently, the YAML configuration may be accepted and translated even though the resulting route is operationally incorrect.

A possible improvement would be to apply to every `routes[].via` field a reachability check similar to the one already used for `default_gateway`. The validator could verify that the next-hop address belongs to at least one subnet directly connected to the router. More advanced cases, such as explicitly configured `onlink` routes or recursive next-hop resolution, could later be supported through dedicated configuration options.

Unknown YAML keys can become silent misconfigurations. YAML itself permits arbitrary mapping keys. During rapid feature development, retaining the loaded document as ordinary dictionaries made it easy to add new properties without a separate schema-migration step. The same permissiveness has a cost: the framework interprets the keys that belong to its model, while an unknown node-level key is not automatically translated into Kathará or FRRouting behaviour. This creates a robustness risk when a user makes a typographical error:

```
pc_a:
  type: host
  interfaces:
    - network: lan_a
      ip: 10.0.1.10/24
  defualt_gateway: 10.0.1.1
```

Listing 6.5: Typographical error that resembles a valid configuration field.

The user's intention is obvious, but `defualt_gateway` is not `default_gateway`. A strict configuration schema should therefore reject unknown fields rather than silently ignore them. This can be implemented with explicit allowed-key sets or a schema-validation library. Error messages could additionally suggest the closest valid field name.

FRR enable/disable semantics are not completely uniform. Router, switch, and firewall node types are considered FRR-capable by their type, while a host becomes FRR-capable through its FRR declaration. As a result, `frr.enabled: false` does not act as one universal "disable FRR" switch across all node types. This is primarily an API and robustness weakness rather than a security vulnerability, because a configuration may express a state whose operational interpretation is not immediately intuitive.

A future version could define a single precedence rule: an explicit `frr.enabled` value should override type-based defaults, while absence of the field should preserve automatic capability detection. This would make the meaning of the configuration independent of the path by which FRR capability was inferred.

Container-image override introduces a trust decision. The optional `image` field overrides the default image selected by the framework. This is necessary for custom laboratories, but a configuration from an untrusted source can therefore request an image that has not been reviewed by the laboratory author. The security consequences depend on Docker and Kathará runtime policy, so this should be described as an image-trust or supply-chain risk rather than as a demonstrated container escape.

For controlled environments, image overrides could be limited to an allowlist of repositories or immutable image digests. Strict mode could reject unapproved images, while normal development mode would retain the current flexibility.

6.3 Translator Model Limitations

Some behaviours that appear restrictive are consequences of the current network model rather than vulnerabilities.

The validator assumes a single coherent Layer 3 address space for the complete laboratory. IP addresses are treated as globally unique and overlapping subnets on distinct collision domains are rejected. This simplifies target resolution and catches many accidental addressing errors, but it prevents legitimate multi-tenant or VRF-style laboratories in which isolated routing domains intentionally reuse the same prefixes.

For example, the following conceptual topology is reasonable in a multi-tenant system:

```
tenant_a_pc:
  interfaces:
```

```
    - network: tenant_a_lan
      ip: 192.168.1.10/24

tenant_b_pc:
  interfaces:
    - network: tenant_b_lan
      ip: 192.168.1.10/24
```

Listing 6.6: Address reuse that is meaningful only when routing domains are isolated.

The current translator rejects this configuration because the same address is used twice, even though the two logical tenants might never share a routing domain. The corresponding mitigation would not be to weaken duplicate-address validation globally. A better extension would introduce an explicit `routing_domain`, namespace, or VRF concept and scope address uniqueness and subnet-overlap checks within that domain.

The switch and firewall node types have similar expressiveness limitations. They reuse the common node and interface model, but the YAML language does not currently provide a dedicated policy language for VLANs, trunks, spanning tree, firewall rules, NAT policy, or complex filtering. Such behaviour must be implemented through custom commands or imported node resources. This design is sufficient for the current project scope but also increases reliance on the opaque escape mechanisms discussed above.

6.4 Possible Improvements

The identified weaknesses can be addressed incrementally without redesigning the complete framework.

First, the collision-domain overlap check should avoid an all-pairs comparison when the number of domains becomes large. IPv4 and IPv6 networks can be indexed separately and ordered by network address, or stored in a prefix or interval structure. The objective is to reduce the common-case complexity from quadratic pairwise comparison toward $O(D \log D)$ indexing and lookup, while preserving the current ability to report the two conflicting domains.

Second, the normal generation path could reuse the already loaded and validated Python representation instead of parsing and validating the source a second time. The lower-level generation function can retain its independent safety property by exposing two interfaces: a public function that accepts a path and performs complete validation, and an internal function that accepts a validated configuration object. The command-line path would then use the second function after its initial validation. The benchmark result that generation takes approximately twice as long as validation across all scenarios suggests that this change could have a measurable effect.

Third, explicit resource limits should be considered if configuration files can originate from untrusted sources. Limits may include maximum YAML file size, number of nodes, number of interfaces, number of collision domains, number of static routes, number of custom commands, and maximum generated-file count. These limits would convert extreme resource consumption into an early, predictable validation error. The limits should be configurable so that research users can deliberately raise them for large experiments.

Fourth, the configuration language could provide a strict mode. In this mode, unknown fields would be errors, `commands` and raw `frr.config` would be rejected or require explicit opt-in, image overrides would be restricted, and structured route validation would be stronger. The existing permissive mode could remain available for trusted laboratory development.

Fifth, translation success could be separated from runtime correctness through an optional post-generation verification stage. Such a stage could start the laboratory, verify expected interface addresses and forwarding state, inspect FRR daemon status, and run selected connectivity tests. This would not replace static validation, but it would detect the class of errors that cannot be proven by inspecting opaque shell or FRR commands.

6.5 Discussion

The experiments show that the translator does not have a single meaningful "maximum number of nodes". Its effective scalability depends on which part of the configuration grows. A 10,000-node flat topology validates in 4.065 seconds, while a 4,000-node star topology requires 14.565 seconds because it contains many more independent collision domains that must be compared. Large lists of routes and commands stress YAML deserialisation and memory rather than topology analysis. Complete generation introduces additional filesystem work and, in the current architecture, repeats loading and validation.

The supplied results are encouraging in one respect: all valid stress configurations completed without timeout or generation error, and the large invalid configuration was rejected consistently without producing files. The results therefore do not indicate a small hard capacity limit. Instead, they reveal specific scaling dimensions that can be improved.

From a security perspective, the principal boundary is trust in the YAML source. The framework already uses safe YAML loading, which prevents YAML tags from directly constructing arbitrary Python objects. Safe deserialisation, however, does not make the complete configuration harmless: the configuration language deliberately contains fields that later become shell commands, raw FRRouting text, and container-image choices. For a locally authored research laboratory this is an acceptable and useful design decision. For a service that accepted third-party configurations, the same features would require strict sandboxing, explicit opt-in, resource limits, and stronger schema validation.

The benchmark should also be interpreted with an important methodological limitation. The supplied CSV records two measured repetitions per scenario. The two runs are close to each other and are sufficient to identify large differences such as the star-versus-flat behaviour, but they are not enough for strong statistical claims about small performance differences. A future evaluation could repeat each test five or more times on a documented reference machine and report median, percentile, and variance. It would also be useful to run a second, separate experiment that starts the generated laboratories in Kathará. That experiment would measure emulation scalability rather than translator scalability and should not be mixed with the results presented here.

Overall, the evaluation identifies a clear boundary between the parts of the framework that are strongly validated and those deliberately left open for advanced user control. The most relevant improvement areas are the collision-domain overlap algorithm, duplicate loading and validation, strict schema handling, static-route next-hop validation, and explicit controls for opaque commands, raw FRR configuration, and custom images. These changes would preserve the framework's main advantage—the ability to generate readable standard Kathará laboratories from a compact YAML source—while making its behaviour more predictable under large or untrusted inputs.

Chapter 7

Framework Evolution and Future Developments

The framework developed in this thesis was intentionally designed as a lightweight layer on top of Kathará. Its current scope is therefore limited and explicit: it provides a declarative representation of a laboratory, validates that representation, generates standard Kathará artefacts, assists with laboratory lifecycle operations, and supports a set of repeatable measurements. The experimental evaluation in Chapter 5 shows that this foundation is already sufficient to build and observe a non-trivial routed environment under substantially different operating conditions. At the same time, the same experiment also identifies several directions in which the framework could be generalised.

The purpose of this chapter is not to describe functionality that is already implemented. Instead, it presents a possible evolution path from the current prototype towards a broader framework for network experimentation. The main idea is to preserve the characteristics that motivated the original design—a readable declarative configuration, explicit validation, standard Kathará output, and simple command-line operation—while extending the framework with reusable experiment definitions, runtime verification, richer experiment management, and a cleaner separation between the logical network model and the execution backend. These developments are discussed as future work and should therefore be interpreted as architectural proposals rather than as features of the present implementation.

7.1 Extension to Network Attack Testing

The controlled scenario presented in the previous chapter demonstrates that the framework can support more than topology generation. A generated laboratory can also host application services, execute coordinated client measurements, apply a bounded adverse workload, and collect observations that distinguish network reachability from application-level availability. In the current repository, however, this workflow is tied to a specific experiment and to a set of scripts written for that experiment. A natural evolution would be to extract the reusable concepts from that implementation and introduce a generic subsystem for controlled security and resilience testing.

The objective of such an extension would not be to turn the project into a general-purpose attack tool. Instead, it would support bounded experiments that study denial-of-service and related resilience conditions inside an isolated environment [7]. The value of Kathará in this context is precisely that potentially disruptive network conditions can be reproduced inside an isolated and explicitly configured environment. The future framework should therefore model adverse conditions as *experimental scenarios*: bounded, repeatable changes applied to nodes that belong to the current laboratory and observed through a predefined measurement procedure. This interpretation also broadens the scope beyond malicious traffic. From the point of view of the framework, a security test,

an accidental overload, a temporary link problem, and a routing fault are all controlled deviations from an expected baseline that can be introduced, measured, and compared.

A first extension could support several families of adverse network conditions. Examples include connection-pressure scenarios, controlled high-rate UDP or ICMP traffic, service discovery and scanning inside the emulated network, short traffic bursts, artificial congestion, temporary link failures, routing misconfiguration, and selected routing anomalies. These examples represent different causes, but the orchestration problem is similar in each case. The framework must identify the participating nodes, validate that they belong to the laboratory, apply a bounded action for a known period, continue measuring the relevant services, and stop the action deterministically. The specific traffic generator or fault mechanism would become an implementation detail of a scenario type rather than the organising principle of the whole experiment.

This suggests separating three concepts that are only partially separated in the current project: the *topology*, the *experiment*, and the *measurements*. The topology should continue to describe the relatively stable network structure: nodes, interfaces, collision domains, addressing, routing, and node resources. An experiment should describe a temporary condition applied to that topology. A measurement specification should state which observable properties must be recorded. Keeping these concepts separate would allow the same topology to be reused for different experiments and the same experiment definition to be applied, when compatible, to more than one topology.

A possible future configuration could therefore use a dedicated experiment document such as the conceptual example in Listing 7.1. The syntax is illustrative and is not implemented by the current framework.

```
experiment:
  name: service_resilience_test
  duration: 60

  source_nodes:
    - node_a
    - node_b

  target:
    node: server
    port: 9000

  workload:
    type: connection_stress
    profile: medium

  measurements:
    - availability
    - tcp_connect_time
    - application_response_time
```

Listing 7.1: Conceptual declarative definition of a controlled experiment.

The important aspect of this proposal is not the exact YAML syntax but the change in abstraction. Instead of encoding an experiment primarily through shell variables and script-specific command-line arguments, the user would provide a structured description that could be validated before execution. The framework could check that `node_a`, `node_b`, and `server` are nodes in the selected topology, that the target port is valid, that the requested workload type is installed, and that the selected metrics are supported. The resulting experiment definition would then become part of the reproducible input together with the topology configuration.

Scenario profiles could further reduce repetition. Rather than expressing every parameter explicitly for every run, the framework could expose named profiles such as `baseline`, `low`, `medium`, and `high`. A profile would not imply an absolute notion of severity; it would simply be a versioned collection of bounded parameters for a particular

scenario type. Users could then execute the same topology under multiple profiles while the framework records the exact values used by each run. The current baseline and controlled-load comparison in Chapter 5 can be seen as a first manual instance of this more general pattern.

A generic experiment subsystem should also make safety constraints part of the configuration model rather than relying only on conventions in individual scripts. At minimum, future workload definitions should have explicit maximum durations, maximum concurrency or rate parameters, and target restrictions. Most importantly, a target should be expressed through a node reference in the selected laboratory whenever possible, rather than as an arbitrary external address. The validation phase could resolve the reference to the addresses assigned to that node and reject destinations that do not belong to the emulated environment. The framework could also terminate all experiment processes when a configured deadline is reached or when the laboratory is stopped. These constraints would preserve the intended use of the system as an isolated experimentation platform.

Generalising the scenario model would also make mitigation testing more systematic. The rate-limiting reverse-proxy experiment evaluated in Section 5.4 is a first instance: a public endpoint at `10.30.1.10` forwards admitted sessions to a private backend at `10.30.2.10` while applying global and per-source connection limits. The archived comparison reuses the same clients, round count, activation schedule, and measurement schema across five executions per topology. These paired repetitions were orchestrated as separate runs; future work should encode their common parameters in a shared experiment definition and execute the complete batch automatically. More generally, a mitigation should not be evaluated by changing the workload and the topology at the same time without recording those changes. The framework could define experimental variants that share the same workload and measurement specification while modifying one clearly identified network or service configuration. Rate limiting, filtering, traffic shaping, service-capacity changes, firewall policies, and architectural isolation are examples of mechanisms that could be compared in this way.

Conceptually, the execution model would evolve from a scenario-specific script into the pipeline shown in Figure 7.1.

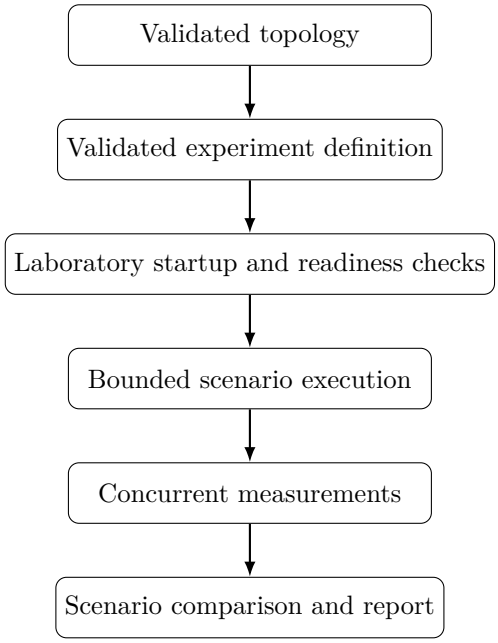


Figure 7.1: Possible future pipeline for reusable controlled network experiments.

This extension would build directly on the present framework. The node names, addressing information, output directories, concurrent client execution, and CSV-based

measurements already provide several of the required building blocks. Future work would mainly need to formalise their relationships and replace scenario-specific orchestration with a reusable experiment model.

7.2 Runtime Verification and Automated Testing

The current validation subsystem operates before laboratory generation. It can identify malformed structures, duplicated addresses, inconsistent subnets, invalid gateways, routing configuration problems, and other errors that can be deduced from the declarative input. This is useful because it prevents many problems from reaching the deployment stage. Nevertheless, a statically valid configuration does not guarantee that the running network has reached the state intended by the user. The difference between *configuration correctness* and *runtime correctness* is therefore an important area for future development.

Several causes can produce this difference. A container may fail to start, a startup command may return an error, an expected interface may not be present, a service process may terminate immediately, or a dynamic routing protocol may require additional time to converge. Similarly, two routers can contain syntactically valid BGP configurations while their session never reaches the `Established` state. These conditions cannot be fully detected by analysing the YAML document alone. A future framework should consequently add a verification phase after `kathara lstart` and before an experiment is considered ready.

The first level of runtime verification could compare the generated model with the basic state of each container. The framework already knows which nodes should exist and which interfaces and addresses were generated for them. It could therefore verify that all expected containers are running, inspect `ip addr` to check interface addressing, inspect `ip route` to verify default and static routes, and confirm that required processes are active. These checks would transform information already available during generation into explicit post-start assertions.

Routing protocols require more specialised checks. For an OSPF laboratory, the framework could inspect the neighbour table and verify that expected adjacencies are present after a configurable convergence interval [19]. For BGP, it could check that selected peers reach the expected session state and that required prefixes appear in the routing table [25, 6]. The same principle applies to application services: a node configured to provide a TCP service could be considered ready only after the corresponding socket is listening and, where appropriate, after a minimal application-level request succeeds. Such checks would make startup deterministic from the point of view of later experiments.

The intended state could be partially inferred from the topology, but not every expectation can be derived automatically. A future extension could therefore allow users to declare selected behavioural expectations. The conceptual configuration in Listing 7.2 illustrates this idea.

```
expected:
  connectivity:
    - source: pc_a
      destination: server
      protocol: tcp
      port: 9000
      reachable: true

  routing:
    r1:
      ospf_neighbors: 2

    edge_router:
      bgp_peers_established: 1
```

Listing 7.2: Conceptual declaration of expected runtime properties.

The framework could translate such expectations into runtime checks and produce a structured comparison between the expected and observed state. A failed expectation should not necessarily be treated as a software error: in some experiments, an unreachable destination may be intentional. The relevant point is that the expected outcome is explicit and machine-readable, allowing the same assertion to be evaluated across repeated runs.

This mechanism would also improve the orchestration logic. In the current experimental workflow, fixed waiting periods and explicit reachability checks are used to reduce the risk of starting measurements before the laboratory is ready. A more general implementation could replace most fixed delays with state-based readiness conditions. For example, instead of waiting an arbitrary number of seconds after startup, the framework could poll the required OSPF adjacencies until they appear or until a maximum convergence timeout is reached. The experiment would start only when all mandatory readiness checks succeed. If they do not, the run would be marked as invalid rather than being mixed with application measurements. The distinction made in Chapter 5 between a valid baseline and a run affected by initial reachability problems shows why this separation matters.

Runtime verification should be complemented by automated testing of the framework itself. The present implementation could be covered by unit tests for configuration loading, structural validation, semantic validation, collision- domain analysis, startup generation, `lab.conf` generation, and FRRouting output. Each test could provide a small input configuration and compare the resulting data or files with an expected representation. Negative tests would verify that invalid configurations produce the correct validation issues without modifying the output directory.

A second layer could consist of integration tests that create very small Kathará laboratories. An integration test could generate a two- or three-node configuration, start it, execute one or more runtime assertions, and clean it again. Such tests would verify not only the Python generation logic but also its compatibility with the Kathará and Docker environment. Because these operations are more expensive than unit tests, they could be executed separately or only on systems where the required runtime is available.

The combination of static validation, runtime verification, and automated software tests would produce a clearer lifecycle:

`configuration` → `static validation` → `generation` → `startup` → `runtime verification` → `experiment`

This evolution would also make the framework more useful in teaching. A student laboratory could include both the intended topology and a set of expected properties. The framework could then report whether the generated or modified network satisfies those properties without hiding the underlying Kathará commands and files. The user would still be able to connect to nodes and inspect the network manually, while automated checks would provide a repeatable reference point.

7.3 Experiment Management and Usability

The current project deliberately keeps its tools small and independent. Generation, startup, connectivity measurements, metric visualisation, and cleanup are implemented through separate Python or shell entry points. This modularity was useful during development because each component could be tested and changed independently. As the number of supported workflows grows, however, the same modularity can create a usability problem: the user must know which script to invoke, in which order, with which laboratory path, and where the resulting files will be stored. A future version should preserve the internal modules while presenting a more unified external interface.

A single command-line application could act as an orchestration layer. Its interface could expose commands such as those shown in Listing 7.3. The names are illustrative, but they express a consistent resource-oriented workflow.

```
framework validate topology.yml
framework generate topology.yml
framework start my_lab
framework verify my_lab
framework experiment my_lab experiment.yml
framework report my_lab experiment_0042
framework stop my_lab
framework delete my_lab
```

Listing 7.3: Conceptual unified command-line interface.

Internally, these commands could call the same generation, validation, testing, and reporting components that exist as separate scripts. The change would therefore concern orchestration rather than a monolithic rewrite. A unified interface would also make help messages, error handling, configuration paths, and naming conventions consistent across the project.

Experiment identity is another area that deserves stronger support. The current repository already uses numbered experiment-result directories, which is an effective first step because it prevents one run from overwriting the results of another. A future implementation could formalise this idea by assigning every execution a unique experiment identifier and storing all inputs and metadata required to interpret that run. The directory structure could resemble Listing 7.4.

```
results/
‘-- experiment_0042/
    |-- metadata.json
    |-- topology.yml
    |-- experiment.yml
    |-- measurements.csv
    |-- server_metrics.csv
    ‘-- report.pdf
```

Listing 7.4: Possible future experiment output layout.

The metadata file could record the experiment identifier, timestamp, framework version, topology identifier, topology hash, experiment parameters, Kathará version, Docker version, and selected host information. Capturing this information would make it easier to determine whether two result sets were produced under comparable conditions. Storing a copy of the topology and experiment definition alongside the measurements would also ensure that later changes to files in `configs/` do not make an old result ambiguous.

A cryptographic hash of the input configuration could provide a simple link between source and output. The hash would not replace the copied YAML document, but it would make accidental configuration changes immediately visible. The same principle could be extended to the framework version, for example by recording a Git commit identifier when available. Reproducibility would then be based not only on the numerical measurements but also on a traceable record of how the laboratory and experiment were produced.

Reporting could also be integrated into the experiment lifecycle. The current tools already calculate summary statistics and visualise selected measurements. A future report generator could systematically combine raw measurements, metadata, validation status, runtime readiness checks, and plots. It could produce an HTML or PDF report containing success rate, availability, mean, median, percentile statistics, error categories, resource utilisation, and comparisons among profiles or experiment variants. The raw CSV files should remain available so that the automated report is an additional representation rather than the only form of the data.

A related improvement is automatic topology visualisation. The generator already has enough information to construct a graph: node names, node types, interfaces, collision domains, addresses, and routing relationships are known before the laboratory is started.

A visualisation component could transform this model into Graphviz, Mermaid, SVG, or another portable representation. The same diagram could be embedded in generated experiment reports and used as a quick check before deployment. For larger laboratories, automatic layout would be especially useful because a textual YAML file becomes progressively harder to interpret as the number of nodes and collision domains increases.

Visualisation should remain derived from the declarative source rather than becoming a second authoritative configuration. An editor that allows users to modify the graph directly could be considered later, but the underlying change should still be represented in the same data model and serialised back into a versioned configuration. This avoids creating two independent descriptions of the same laboratory.

A lightweight web interface could eventually expose generation, validation, lifecycle control, topology visualisation, and result browsing through a browser. Such an interface could be useful in teaching environments where users are less familiar with command-line tools. Nevertheless, the CLI should remain a first-class interface because it is more suitable for automation, version-controlled workflows, remote execution, and integration testing. A web interface would therefore be a front end to the same core services rather than a replacement for them.

Finally, experiment management could support batches of related runs. Instead of manually executing one profile at a time, the user could declare a matrix of scenarios, repetitions, or topology variants. The framework would assign a separate identifier to each run, execute them sequentially or with explicitly controlled parallelism, and generate an aggregate report. This would be particularly useful when studying variability. The repeated-run method used in Chapter 5 already bases its principal conclusions on a distribution of run-level summaries, but its five acquisition blocks were launched as separate commands. A batch facility would make the same method declarative, automatic, and reusable for other experiment matrices.

7.4 Using LLM-Based Coding Alongside the Framework

The comparison in Section 2.5 first suggests an immediately usable workflow rather than a requirement to integrate a model into the software. A user can employ any general-purpose LLM-based coding assistant to draft or modify the YAML file, run the existing validator, and return its diagnostics to the assistant when another revision is useful. The model does not write directly to the Kathará runtime and is not treated as the authority on whether a configuration is valid: its output enters the existing pipeline as untrusted candidate input. A future front end could automate this exchange, but the main synergy already exists through ordinary files, command output, and measured results.

The first implementation step would be to formalise the currently implemented configuration model. The current accepted language is defined by Python code and documented in Chapter 3; future work already identifies a strict schema as desirable. An LLM integration strengthens that requirement because a machine-generated draft benefits from an explicit description of allowed keys, types, enumerations, and nested structures. A JSON Schema or an equivalent versioned schema could be supplied to the model as context and also enforced by the framework independently. The model could then be instructed to produce only a structured object conforming to that schema instead of free-form Kathará files.

Schema conformance would remain only the first check. The existing semantic validator would still be needed because many network properties depend on relationships rather than local field types. Two addresses can each be valid CIDR values while still being duplicates; a gateway can be a valid IP address while being unreachable from the node; and two BGP declarations can each be well formed while disagreeing on the remote autonomous system. These are precisely the kinds of checks for which deterministic code

is preferable to asking the same model that produced the configuration whether its own answer is correct.

The resulting interaction could use the validation report as a repair protocol. A first LLM response would produce a candidate YAML document. The framework would return either success or a list of structured issues. If errors were present, a controller could send the candidate configuration and the relevant issue codes back to the model and request a revised version. The loop would be bounded by a small maximum number of attempts and would preserve every candidate for traceability. A conceptual exchange is shown in Listing 7.5.

```
user intent
  -> LLM: candidate topology.yml
  -> framework validate topology.yml
       -> VALID: generate laboratory
       -> ERROR: return structured diagnostics
                 -> LLM/human revises topology.yml
                 -> validate again
```

Listing 7.5: Conceptual LLM-assisted validation loop.

This design would make the LLM replace part of the manual authoring effort, not the validator. It would also preserve a useful property of the present project: a user who does not want to use an LLM could continue to write YAML directly and follow exactly the same validation and generation path. The AI component would therefore be an adapter to the declarative interface rather than a new required dependency.

A second use of LLMs could occur after execution. Runtime verification proposed in Section 7.2 could produce structured facts such as missing routes, inactive services, failed OSPF adjacencies, or unexpected application responses. An LLM could combine those facts with the source configuration and selected logs to explain likely causes or suggest a next inspection command. The same principle applies to experiment results: raw CSV files, packet captures, and metadata should remain authoritative evidence, while an LLM may generate a human-readable interpretation or propose hypotheses for a subsequent experiment. This separation avoids making an automatically generated summary the only record of what happened.

The integration also introduces security and control requirements. The present configuration language intentionally exposes powerful escape mechanisms. A `commands` entry becomes shell text in a startup file, `frr.config` can contain arbitrary FRRouting configuration, and an explicit image can change the software executed by a node. These fields are useful for expert users but should not be executed automatically merely because an LLM proposed them. A safer assisted mode could prohibit opaque fields by default, require explicit human approval for them, or apply allow-lists for commands and images. Generation could also run under resource limits and with a clearly isolated output directory before any laboratory is started.

Traceability should include the AI-assisted part of the process. If an LLM is used, experiment metadata could record that fact together with the model family or endpoint, the prompt template or task description, the schema version, and a hash of the accepted YAML source. The exact conversational history need not become part of every experiment, but the final source and framework version must remain sufficient to regenerate the laboratory. This preserves the thesis' main reproducibility principle: the accepted configuration, not the transient conversation, is the authoritative experiment input.

There is also a useful teaching perspective. An LLM can explain why a route or BGP relationship is required and can help a student produce an initial configuration, while deterministic validation prevents the interaction from ending at a plausible-looking answer. The student can compare the LLM proposal, the validator diagnostics, the generated Kathará files, and the observed runtime state. The AI assistant then becomes part of the learning process without hiding the concrete networking mechanisms that the laboratory is meant to expose.

Research on LLM-assisted networking already points toward such hybrid designs. NetConfEval considers the interaction between LLMs and existing network synthesis mechanisms, while LLM-NetCFG combines intent-driven generation with verification in a network-management workflow [29, 15]. The proposed extension follows the same general principle but keeps the scope specific to this thesis: the LLM would improve how a user reaches the YAML representation and how diagnostics are interpreted, whereas the framework would continue to own validation, generation, and the transition to the Kathará runtime.

7.5 Towards a More General Network Emulation Framework

The previous proposals can be implemented incrementally without abandoning Kathará. In fact, preserving direct compatibility with Kathará is one of the main strengths of the current project: generated laboratories remain standard, inspectable environments rather than opaque artefacts that require a custom runtime. If the framework grows substantially, however, a stronger separation between its logical network model and its Kathará-specific generation code would improve maintainability and make the architecture easier to extend.

The current pipeline can be simplified as

$$\texttt{YAML configuration} \rightarrow \texttt{Kathará files}.$$

A more general architecture would insert an explicit internal model and a backend boundary. The existence of different emulation approaches, including Kathará and container-/namespace-based systems such as Mininet, illustrates why the logical model and backend-specific representation should remain separate [26, 14]:

$$\texttt{configuration} \rightarrow \texttt{network model} \rightarrow \texttt{backend} \rightarrow \texttt{runtime artefacts}.$$

The internal model would represent concepts such as laboratories, nodes, interfaces, collision domains, addresses, routes, and routing configuration without embedding the syntax of `lab.conf` or startup files. The Kathará backend would then translate this model into the artefacts described in Chapter 4. This separation would make it easier to test the model independently from file generation and would provide a clear place for future backend-specific behaviour.

Such an abstraction does not imply that support for other emulation platforms would be simple. Different systems expose different network models, lifecycle semantics, privilege requirements, and configuration mechanisms. A future backend interface would therefore be valuable primarily because it separates responsibilities, not because it guarantees portability. Kathará would remain the reference backend, while alternative targets could be investigated only where their capabilities map cleanly onto the internal model.

A plugin architecture could complement this separation. The present `gen_lab.py` module contains several responsibilities related to routing, resource import, Wireshark integration, and output generation. If more protocols, experiment types, validators, and reporting components are added, continuing to extend a small number of central modules would increase coupling. A future layout could instead organise optional capabilities around explicit extension points, for example:

```
framework/
|-- core/
|-- model/
|-- validators/
|-- backends/
|   `-- kathara/
|-- experiments/
|-- measurements/
```

```
|-- reporting/
`-- plugins/
    |-- ospf/
    |-- bgp/
    `-- wireshark/
```

Listing 7.6: Conceptual modular organisation for a larger framework.

A plugin would not necessarily correspond to a dynamically installed Python package. The important design principle is that an optional capability should have a defined way to contribute configuration fields, validation rules, generation logic, runtime checks, and documentation. For example, a routing extension could register both semantic validation and backend generation rules without requiring unrelated experiment-management code to know its internal details.

Chapter 6 already characterises the scalability of the translator with synthetic inputs, including configurations containing up to 10,000 nodes. Those measurements cover loading, semantic validation, and file generation, but deliberately stop before the generated laboratories are started. The remaining scalability question is therefore runtime execution: future work should measure Kathará startup time, host memory and CPU consumption, container stability, and routing convergence as the number of executed nodes and collision domains increases. This would separate the scalability of the Python translation stage, which has been benchmarked, from the scalability of the Docker/Kathará runtime, which has not.

Large experiments also raise questions about orchestration. A single host is a simple and reproducible environment for the present project, but CPU scheduling, memory pressure, and the number of containers eventually influence results. The framework should therefore record host resource information and make clear when measurements are specific to one execution environment. Distributed execution could be studied in the future, but it would represent a substantial change in scope because synchronisation, clock behaviour, network links between hosts, and result aggregation would become part of the experimental model. For this reason, distributed operation should be considered a possible long-term direction rather than a direct next step.

The same caution applies to a graphical topology editor and other usability features. They can make the framework easier to approach, but they should not weaken the properties that make the current design useful for reproducible experiments. The declarative source must remain versionable, inspectable, and sufficient to reconstruct the laboratory. Runtime state and generated files should remain derived artefacts. Any additional interface should therefore operate on the same underlying model rather than creating an independent source of configuration.

Taken together, the developments discussed in this chapter describe a gradual path from a laboratory generator towards a more complete experimentation framework. The first stage would generalise scenario execution and runtime verification using the existing Kathará backend. The second would improve experiment identity, metadata, reporting, and automated testing. An optional LLM-assisted front end could then use the same declarative model and validation report without becoming a replacement for them. Only after these foundations are stable would a richer internal model, plugin system, or additional backends be necessary. This staged evolution preserves the simplicity of the present implementation while providing clear architectural directions for extending its use in network emulation, resilience testing, security experimentation, AI-assisted configuration, and teaching.

Chapter 8

Conclusions

This thesis presented the design, implementation, and experimental use of a lightweight framework for the generation and management of Kathará network laboratories. The work originated from a practical problem: even relatively small emulated networks require several artefacts to remain consistent with one another. A topology must define the correct collision domains, every node must receive the intended interfaces and IP addresses, gateways and static routes must match the addressing plan, routing daemons must be enabled and configured, and application-specific files must be placed in the correct node directories. When these tasks are performed manually, the resulting laboratory is often correct only after several iterations of editing and troubleshooting. The framework developed in this thesis addresses this problem by moving the primary description of a laboratory to a declarative YAML configuration and deriving the Kathará artefacts from that source.

The contribution is deliberately modest in scope. The project does not attempt to replace Kathará, Docker, or FRRouting, and it does not introduce a new network-emulation runtime; instead, it preserves the standard Kathará workflow described by the project documentation [10, 11]. It adds a small abstraction layer that makes a selected class of laboratories easier to describe, validate, regenerate, and reuse while preserving compatibility with the normal Kathará workflow. This design choice shaped the complete implementation and is also central to the assessment presented in this concluding chapter.

8.1 Summary of Contributions

The first contribution of the work is the definition of a declarative configuration model for Kathará laboratories. A single YAML document can represent the laboratory name and metadata, node roles, interfaces, collision domains, IP addressing, default gateways, static routes, startup commands, and routing-related information. The same model supports several node categories and can express both simple host-router-server laboratories and more structured routed environments. The use of YAML keeps the source configuration readable and versionable, while the generated Kathará directory remains a derived artefact rather than the authoritative description of the topology.

A second contribution is the validation stage introduced before generation. The framework first checks the minimum structural requirements of the document and then performs semantic checks on the network description. These checks include node and collision-domain naming, node types, IP syntax, duplicate addresses, use of network or broadcast addresses, inconsistent or overlapping subnets, default gateways that are not locally reachable, routing-device consistency, router identifiers, OSPF network declarations, and BGP parameters and neighbour relationships. The distinction between blocking errors and warnings allows the framework to reject configurations that cannot be generated safely while still reporting suspicious situations that may be intentional. The `--validate-only` mode also makes validation useful independently of generation.

The third contribution is the automatic production of standard Kathará artefacts. Starting from a validated configuration, the generator creates the laboratory directory, writes `lab.conf`, produces the startup files for individual nodes, selects suitable container images, and creates the FRRouting files required by routing-capable devices. OSPF, RIP, and BGP are supported by the current generation logic, while the configuration model also provides an escape mechanism for cases in which a complete custom FRRouting configuration is more appropriate than the declarative subset implemented by the framework. Node-specific directories can additionally be imported into the generated laboratory, allowing application scripts and configuration files to coexist with the automatically generated network artefacts.

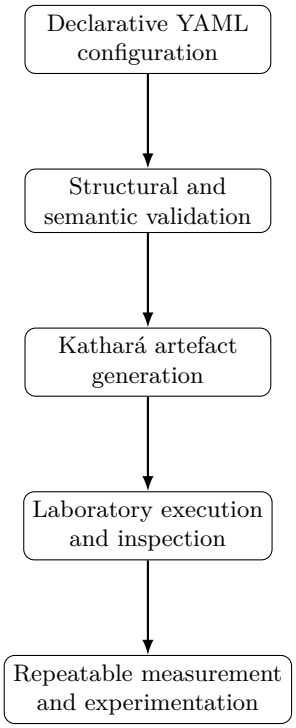
The generator also provides optional Wireshark integration. The framework can identify the collision domains defined by the topology and attach a dedicated capture container to selected networks. This avoids requiring separate Docker or packet-capture setup after generation and keeps the observation point part of the laboratory itself. Together with the lifecycle wrappers for starting, stopping, and removing laboratories, these features create a complete workflow from configuration to execution while still relying on standard Kathará commands underneath.

A further contribution is the explicit separation between configuration, generation, execution, and experimentation. The architecture described in Chapter 2 is reflected by the implementation: YAML files describe the desired laboratory, the loader and validator analyse that description, the generator produces the runtime artefacts, and separate tools operate on the generated environment. This separation prevents the topology description from becoming entangled with one specific experiment. It also makes it possible to regenerate a laboratory after changing the configuration instead of applying persistent modifications directly to generated files.

Finally, the framework was exercised in a non-trivial experimental setting rather than being evaluated only through minimal examples. The case study in Chapter 5 combined eight legitimate clients, multiple routers, OSPF-based connectivity, an application service, concurrent measurements, server-side metrics, and a bounded adverse workload. The purpose of that scenario was not to make attack generation the central contribution of the thesis. It was used to verify that laboratories produced by the framework could support coordinated, repeatable measurements in substantially different operating conditions. A second topology placed a rate-limiting reverse proxy in front of the service, demonstrating that the same measurement schema can also support an explicit mitigation comparison.

From a software-engineering perspective, these contributions are also the result of successive refinement rather than independent features selected in advance. The project moved from basic topology generation to pre-generation validation, richer routing support, diagnostic and lifecycle facilities, application-level testing, concurrent measurements, controlled experimentation, and finally robustness and scalability analysis. Several final behaviours are direct consequences of problems observed during earlier iterations: service checks were separated from reachability after runtime testing showed that a reachable node could still provide no usable application response; concurrent rounds replaced sequential client execution when temporal comparability became necessary; readiness checks were strengthened after an early baseline was contaminated by unavailable routes; and the final stress tests exposed the performance cost of defensive duplicated validation. The implemented architecture therefore records both successful design decisions and trade-offs identified through verification.

These contributions can be summarised as a progression from a declarative network description to an executable and observable environment:



The resulting framework remains intentionally small, but the complete chain is important. Automation is useful only if the input can be checked, the output is understandable, and the generated environment remains compatible with the tools that users already expect to use. For this reason, the project favours explicit files and standard Kathará conventions over a more opaque orchestration layer.

8.2 Main Findings and Overall Assessment

The implementation confirms that a declarative front end can reduce the amount of repetitive work required to construct Kathará laboratories without hiding the resulting network configuration. The generated `lab.conf`, startup files, and FRRouting configuration remain directly inspectable. This is particularly useful in an educational or experimental context, where the configuration itself is often as important as the running network. The framework therefore automates file construction while preserving the ability to connect to a node, inspect interfaces and routes, use `vtysh`, capture packets, and modify or troubleshoot the environment with ordinary Kathará and Linux tools.

The validation layer proved equally important. Several classes of mistakes can be identified before any container is started, including addressing conflicts, gateways outside directly connected networks, and inconsistent routing parameters. Detecting these conditions at configuration time does not guarantee that the running network will behave exactly as intended, but it reduces a set of errors that would otherwise appear later as connectivity or routing failures. This distinction between static correctness and runtime correctness is also one of the reasons why runtime verification was identified in Chapter 7 as a natural direction for future work.

The framework also demonstrated that the same generation process can be used for topologies of different complexity. Beyond the example laboratories, the translator was stress-tested in Chapter 6 with synthetic configurations containing up to 10,000 nodes and with inputs dominated by large route or command sets. These results characterise loading, validation, and file-generation behaviour, but they do not establish that laboratories of comparable size can be executed efficiently in Kathará. The runtime scalability of large generated laboratories therefore remains outside the evaluated scope.

The experimental evaluation provides a more concrete assessment of the running environment. Five complete executions were collected for each topology. Every direct-server baseline completed all 80 legitimate application requests. In every subsequent controlled run, 16 of 80 requests completed, corresponding to 20% over the complete transition; all 16 occurred in the first two rounds before workload activation. During rounds 3–10, every direct execution therefore completed 0 of 64 legitimate requests, corresponding to 0% attack-active availability. At the same time, all ICMP probes and TCP connection attempts succeeded. In each run the instrumented server reached its configured limit of 20 active application connections and recorded 264 rejected or `BUSY` events during the load phase.

The five reverse-proxy executions produced a different and equally stable outcome under the same round structure. Every baseline and controlled-load phase completed all 80 legitimate requests, so attack-active availability was 100% in every run, compared with 0% without the proxy. The protected backend reached at most eight simultaneous connections and recorded no rejected or `BUSY` event in any repetition. Across paired runs, the increase in the per-run baseline median application response had a median of 2.167 ms and an interquartile interval of [2.147, 2.472] ms, illustrating the availability–overhead trade-off in the tested configuration.

The direct-server result is significant mainly because of the distinction it exposes. The emulated routing infrastructure remained reachable and the TCP service port continued to accept connections, yet the application was unable to complete most legitimate requests. The experiment therefore demonstrates that ICMP reachability, TCP connection establishment, and application availability are separate observations. Measuring only ping latency or whether a port can be opened would have missed the principal effect observed by the legitimate clients. For this reason, the framework's measurement tools record several timing and success dimensions instead of collapsing the experiment into a single latency value.

The experimental evidence must nevertheless be interpreted within its scope. The five paired acquisition blocks provide evidence of consistent run-to-run behaviour for the implemented testbed under the stated parameters and clean laboratory restarts, as detailed in Section 5.5. They remain a small sample, were collected on one host and day, and used a fixed direct-then-mitigated order. Moreover, the mitigation result applies to the bounded connection-oriented workload used in the laboratory and does not establish protection against volumetric or more diverse application-layer attacks.

This limitation is useful when assessing the framework as a research prototype. The project succeeds in automating configuration, validation, generation, and a repeatable experimental workflow, but it does not yet provide all of the controls expected from a mature experimentation platform. There is no formal schema independent of the Python implementation, no complete typed internal network model, no automatic verification that the runtime state matches the expected state, and no systematic benchmark of runtime execution at large scale. These points do not invalidate the implemented workflow; they define the boundary of the contribution and motivate the evolution path presented in the previous chapter.

From an architectural perspective, the most important outcome is therefore not a single generated topology or a single experimental result. It is the separation of concerns established by the project. The source configuration expresses intent, validation checks that intent against the supported model, generation converts it into standard runtime artefacts, and measurement tools observe the behaviour of the resulting network. Because each stage has a clear responsibility, the framework can be extended without requiring the current workflow to be discarded. Declarative experiment definitions, runtime verification, richer reporting, topology visualisation, or additional backend abstractions can all build on the same separation.

8.2.1 Answers to the Research Questions

RQ1 — Declarative translation. The framework translates the YAML description through a staged pipeline: loading, structural and semantic validation, and generation. The output is an ordinary Kathará directory containing `lab.conf`, per-node startup files, imported resources, and FRRouting files where required. The generated artefacts remain inspectable and usable with standard Kathará commands.
RQ2 — Generated configuration and deployed resources. Address assignments, gateways, static routes, and the supported OSPF, RIP, and BGP forms are translated into startup or FRRouting configuration. Application resources and startup commands can be deployed from node-specific directories, and a Wireshark node can be attached to selected collision domains. The current model does not, however, turn `forwarding: true` into a kernel command or turn a `switch` role into an automatic Layer 2 bridge; such behaviour requires explicit commands or resources.
RQ3 — Repeated service measurement. The client workflow executes concurrent rounds from several legitimate nodes and records ICMP reachability, TCP establishment, application completion, and timing fields in CSV form. Success requires the complete application exchange of selection `1` followed by `pong`, so a reachable host or a successful handshake is not misreported as an available service.
RQ4 — Controlled comparison and mitigation. The same client procedure and result schema were used for the baseline, controlled-load, and reverse-proxy experiments. Under the tested active-load condition, availability was 0% in all five direct-server runs and 100% in all five protected runs, while backend occupancy and rejection counters supplied independent server-side evidence. This supports the mitigation only for the bounded workload and parameters evaluated here.
RQ5 — Deterministic framework and LLM assistance. An LLM can draft YAML, shell, or FRRouting text and help interpret results, but plausible text is not equivalent to a checked, reproducible experiment. The framework supplies a validated configuration interface, repeatable semantic checks, coordinated artefact generation, and machine-readable evidence. The two approaches are therefore complementary: flexible drafting and explanation surround a deterministic validation–generation–measurement boundary, in line with work combining LLM-based configuration with explicit synthesis or verification [29, 15].

8.3 Final Remarks

The objective of this thesis was to investigate whether a simple framework could make the construction and use of Kathará laboratories more systematic without replacing the tools and file formats on which Kathará is based. The implemented prototype shows that this is feasible. A readable YAML description can be validated and transformed into an ordinary Kathará laboratory, including node startup configuration and dynamic-routing files, while optional resources and packet-capture support can be incorporated into the same generation process. The resulting environment remains accessible through the normal Kathará lifecycle and can be regenerated from its declarative source.

The experimental case study further shows why this type of automation is useful. Once the laboratory structure is reproducible, the focus of an experiment can move from repeatedly preparing the environment to observing its behaviour. In the evaluation performed for this thesis, concurrent measurements captured a clear difference between normal service operation and operation under a bounded adverse workload, while simultaneously showing that network reachability alone was insufficient to describe application behaviour. This provides a practical example of how the generated infrastructure can be used for networking and security experiments rather than only for topology deployment.

The framework should therefore be considered a foundation rather than a final platform. Its current implementation addresses a deliberately limited set of network primitives and operational tasks, but it establishes a coherent path from configuration

to execution and measurement. The developments proposed in Chapter 7—generalised security and resilience testing, runtime verification, improved experiment management, an optional LLM-assisted configuration loop, and a cleaner separation between the logical model and the execution backend—would extend this foundation while preserving its original emphasis on explicit configuration and reproducibility.

In its present form, the project demonstrates that useful automation does not require hiding the network from the user. By generating standard Kathará artefacts from a validated declarative source, the framework reduces repetitive configuration while keeping the laboratory transparent, inspectable, and compatible with existing tools. The same principle remains valid in an LLM-assisted workflow: natural-language assistance can make configuration more accessible, but the accepted experiment should still pass through explicit, inspectable, and reproducible validation and generation stages. This balance between flexible assistance, deterministic control, and visibility is the main result of the work and provides the basis for further development in network emulation, teaching, controlled security testing, and reproducible experimentation.

Acknowledgements

I would first like to thank my supervisor, **Prof. Gianluca Rizzo**, for his guidance, availability, and valuable feedback throughout the development of this thesis. His suggestions helped me improve both the technical aspects of the project and the way in which the development process, design decisions, and experimental results were presented and analysed.

A special thank you goes to **my family**, for their constant support throughout my university studies and for always encouraging me, especially during the most demanding periods.

I would also like to thank **all my friends** for their support, for the time spent together and for creating moments that made this journey an unforgettable experience.

Among them, I would like to give special thanks to **Federico De Stefani, Alessandro Abate, and Federico Rovei**, my closest friends. They are the people I know I can always count on, especially during the most difficult moments of my life. Their friendship has played an important role in helping me grow, change, and mature over the years.

I would also like to thank **Zoe Orlandi** for her constant support, understanding, and valued friendship throughout this period.

Finally, I thank my **fellow students** for their companionship and collaboration throughout my studies. Their help before examinations and during project work made the most demanding periods easier to face.

References

[1] Gaetano Bonofiglio, Veronica Iovinella, Gabriele Lospoto, and Giuseppe Di Battista. Kathará: A container-based framework for implementing network function virtualization and software defined networks. In *2018 IEEE/IFIP Network Operations and Management Symposium (NOMS 2018)*, pages 1–9. IEEE, 2018. DOI: https://doi.org/10.1109/NOMS.2018.8406267.

[2] Docker, Inc. What is a container? https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/, n.d. Accessed: 13 October 2026.

[3] Docker, Inc. What is an image? https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/, n.d. Accessed: 13 October 2026.

[4] Docker, Inc. What is Docker? https://docs.docker.com/get-started/docker-overview/, n.d. Accessed: 13 October 2026.

[5] Europol. Internet organised crime threat assessment (IOCTA) 2026. https://www.europol.europa.eu/cms/sites/default/files/documents/IOCTA-2026.pdf, 2026. See pp. 28–29 for DDoS activity and Operation Eastwood. Accessed: 11 August 2026.

[6] FRRouting Project. FRRouting user guide. https://docs.frrouting.org/en/latest/, n.d. Accessed: 13 October 2026.

[7] Mark Handley, Eric Rescorla, and Internet Architecture Board. Internet denial-of-service considerations. RFC 4732, RFC Editor, November 2006. Available: https://www.rfc-editor.org/rfc/rfc4732.html; DOI: https://doi.org/10.17487/RFC4732.

[8] Kathará Project. Docker Images: Official image definitions. https://github.com/KatharaFramework/Docker-Images, n.d. Accessed: 13 October 2026.

[9] Kathará Project. kathara-lab.conf(5): Network scenario configuration file. https://www.kathara.org/man-pages/kathara-lab.conf.5.html, n.d. Accessed: 13 October 2026.

[10] Kathará Project. Kathará: Lightweight container-based network emulation system. https://www.kathara.org/, n.d. Accessed: 13 October 2026.

[11] Kathará Project. kathara(1): A network emulation tool. https://www.kathara.org/man-pages/kathara.1.html, n.d. Accessed: 13 October 2026.

[12] Emil Kiner and Tim April. Google mitigated the largest DDoS attack to date, peaking above 398 million rps. https://cloud.google.com/blog/products/identity-security/google-cloud-mitigated-largest-ddos-attack-peaking-above-398-million-rps, October 2023. Google Cloud Blog, 10 October 2023. Accessed: 11 August 2026.

[13] Sam Kottler. February 28th DDoS incident report. https://github.blog/news-insights/company-news/ddos-incident-report/, March 2018. GitHub Blog, 1 March 2018. Accessed: 11 August 2026.

[14] Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: Rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, Hotnets-IX, pages 19:1–19:6. ACM, 2010. DOI: https://doi.org/10.1145/1868447.1868466.

[15] Oscar G. Lira, Oscar M. Caicedo, and Nelson L. S. da Fonseca. Large language models for zero touch network configuration management. *IEEE Communications Magazine*, 63(7):146–153, 2025. Available: https://doi.org/10.1109/MCOM.001.2400368.

[16] Fan Liu, Behrooz Farkiani, and Patrick Crowley. Large language models for computer networking operations and management: A survey on applications, key techniques, and opportunities. *Computer Networks*, 271:111614, 2025. Available: https://doi.org/10.1016/j.comnet.2025.111614.

[17] Gary Malkin. RIP version 2. RFC 2453, RFC Editor, November 1998. Available: https://www.rfc-editor.org/rfc/rfc2453.html; DOI: https://doi.org/10.17487/RFC2453.

[18] Jared Mauch, Job Snijders, and Greg Hankins. Default external BGP (EBGP) route propagation behavior without policies. RFC 8212, RFC Editor, July 2017. Available: https://www.rfc-editor.org/rfc/rfc8212.html; DOI: https://doi.org/10.17487/RFC8212.

[19] John Moy. OSPF version 2. RFC 2328, RFC Editor, April 1998. Available: https://www.rfc-editor.org/rfc/rfc2328.html; DOI: https://doi.org/10.17487/RFC2328.

[20] Peter Murphy. The OSPF not-so-stubby area (NSSA) option. RFC 3101, RFC Editor, January 2003. Available: https://www.rfc-editor.org/rfc/rfc3101.html; DOI: https://doi.org/10.17487/RFC3101.

[21] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768. IEEE, 2022. Available: https://doi.org/10.1109/SP46214.2022.9833571.

[22] Python Software Foundation. `dataclasses` — data classes (Python 3.12). https://docs.python.org/3.12/library/dataclasses.html, n.d. Accessed: 13 October 2026.

[23] Python Software Foundation. `ipaddress` — IPv4/IPv6 manipulation library (Python 3.12). https://docs.python.org/3.12/library/ipaddress.html, n.d. Accessed: 13 October 2026.

[24] PyYAML Project. PyYAML documentation. https://pyyaml.org/wiki/PyYAMLDocumentation, n.d. Version 6.0.1 used in the experiments. Accessed: 13 October 2026.

[25] Yakov Rekhter, Tony Li, and Susan Hares. A border gateway protocol 4 (BGP-4). RFC 4271, RFC Editor, January 2006. Available: https://www.rfc-editor.org/rfc/rfc4271.html; DOI: https://doi.org/10.17487/RFC4271.

[26] Mariano Scazzariello, Lorenzo Ariemma, and Tommaso Caiazzi. Kathará: A lightweight network emulation system. In *2020 IEEE/IFIP Network Operations and Management Symposium (NOMS 2020)*, pages 1–2. IEEE, 2020. DOI: https://doi.org/10.1109/NOMS47738.2020.9110351.

[27] Richard Sharpe, Ed Warnicke, and Ulf Lamping. Wireshark user's guide. https://www.wireshark.org/docs/wsug_html/, n.d. Wireshark Foundation. Accessed: 29 July 2026.

[28] U.S. Department of Justice, Office of Public Affairs. Individual pleads guilty to participating in internet-of-things cyberattack in 2016. https://www.justice.gov/archives/opa/pr/individual-pleads-guilty-participating-internet-things-cyberattack-2016, December 2020. Press release, 9 December 2020; updated 5 February 2025. Accessed: 11 August 2026.

[29] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, and Marco Chiesa. NetConfEval: Can LLMs facilitate network configuration? *Proceedings of the ACM on Networking*, 2(CoNEXT2), 2024. Available: https://doi.org/10.1145/3656296.

[30] YAML Language Development Team. YAML Ain't Markup Language (YAML), Version 1.2.2. https://yaml.org/spec/1.2.2/, 2021.

[31] Omer Yoachimik. Defending the internet: How cloudflare blocked a monumental 7.3 Tbps DDoS attack. https://blog.cloudflare.com/defending-the-internet-how-cloudflare-blocked-a-monumental-7-3-tbps-ddos/, June 2025. Cloudflare Blog, 19 June 2025. Accessed: 11 August 2026.